MATLAB Stress-Strain Analysis — Chat Transcript

M003 tensile tests · 2026-10-07 to 2026-10-15 · 112 messages

You 2026-10-07 21:39 UTC

📎 `specimen.dat` I need you to set up matlab code so I can easily load data from a specific folder structure without having to manually specify every subfolder. The main folder is called "MAE Solids datasets". Inside that folder, I want the code to go into the "M003" folder. Inside M003, there will be multiple folders with each folder containing data for a different material. I want to be able to specify the material/folder I want directly in the MATLAB code. Based on that input, the code should automatically find the correct folder and locate the necessary .DAT file inside it. The code should then read the data from the DAT file and use it to create the required plot. I want the final figure to contain both the graph and a table of the relevant data in the same figure. The goal is to make this easy to use: I should only need to change the material/folder name in the code, and MATLAB should automatically find the correct folder, locate the DAT file, read the data, and generate the graph and table. ive attached a sample file. I need the plot to be a stress/ strain curve using axial force as stress and using axial strain as strain. the plot needs to contain a 0.2% offset line, error bars, and ultimate, yield, and rupture points .

Claude 2026-10-07 21:42 UTC

I wrote the script and am committing it now. I can't run it because there's no MATLAB or Octave here.

Claude 2026-10-07 21:42 UTC

I added `plotStressStrain.m` and pushed it to `claude/brave-johnson-5v00jz`. I couldn't run it because MATLAB isn't available here, so it's untested in MATLAB. I did check the file reading and the yield/ultimate math in Python against your sample file.

What you change: just the top line:

```
material = 'Aluminum';   % folder name inside M003 (case doesn't matter; part of the name works)
```

`baseFolder` defaults to `C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets`. I guessed that from your other script, so set it once if your folder is somewhere else.

What it does:

- **Finding the files:** it goes into `MAE Solids datasets\M003\<material>`, searches every subfolder for a `.dat` file and reads it.
- If the material name doesn't match, the error lists the folders that are there.
- If there's more than one `.dat`, it picks `specimen.dat`, otherwise the newest one. You can name the file yourself with `datFileName`.
- **Reading the data:** it finds the Axial Force and Axial Strain columns by their names, so the extra header lines in the MTS file don't matter.
- **The figure:** the stress–strain curve is on the left and a results table is on the right, in the same window.
- **Offset line:** the slope E comes from a straight-line fit over 10–40% of the ultimate stress (change it with `fitRange`), and the 0.2% line is drawn with that slope.
- **Yield:** the point where the curve crosses the offset line.
- **Ultimate:** the highest stress.
- **Rupture:** the last point before the load drops.
- **Error bars:** spaced along the curve, ±1% of stress and ±5e-5 strain by default.
- **Table contents:** material, file, E, yield, ultimate and rupture stress and strain, and the error settings. It also prints in the Command Window.
- **Saving:** set `saveFigure = true` to save a `.png` and `.fig` into the material folder.

Problems with your sample file:

1. **The Axial Strain column doesn't change.** It stays between about −2.7e-5 and −0.3e-5 the whole test, which looks like the extensometer wasn't recording. With that data the

offset line and yield point mean nothing, and the script will warn you. If that happens, set `strainSource = 'displacement'` and your `gaugeLength`, and it will calculate strain as displacement divided by gauge length.
2. **The sample file stops early.** It's only 51 seconds and 101 points, and the force is still going up at the end (2530 lbf). On this file the "ultimate" and "rupture" points will just be the last point. A full test file will give real values.
3. **The y-axis is force, not stress.** As you asked, it plots axial force in lbf. If you enter the cross-section area, e.g. `specimenArea = pi*0.505^2/4;`, it switches to real engineering stress in psi.

The error-bar sizes (`stressErrPct`, `strainErrAbs`) are placeholders. Set them to your load cell's and extensometer's accuracy.

You 2026-10-07 21:43 UTC

make it copy and pasteable

Claude 2026-10-07 21:44 UTC

Here's the full script. Paste it into a new MATLAB script and save it as `plotStressStrain.m`. Run it from the Editor, not the Command Window, because the helper functions at the bottom only work inside a saved script (MATLAB R2016b or newer). Before you run it, change `material` on the first settings line, and `baseFolder` if your folder is somewhere else.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line, error
% bars, and the yield, ultimate and rupture points, plus a table of the
% results in the same figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Stress ----
% [] = plot axial force (lbf) directly as the stress axis.
% Give the cross-sectional area [in^2] to get true engineering stress (psi),
% e.g. round bar: specimenArea = pi*0.505^2/4;  flat bar: specimenArea = 0.5*0.125;
specimenArea = [];

% ---- Strain ----
% 'strain'       = Axial Strain column (extensometer, in/in)
% 'displacement' = Axial Displacement / gaugeLength (use if the extensometer
%                  was not attached and the strain column is flat)
strainSource = 'strain';
gaugeLength  = 2.0;            % [in], only used when strainSource = 'displacement'
zeroStart    = true;           % shift strain so the curve starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [0.10 0.40];       % fit elastic slope between these fractions of ultimate stress
breakDrop = 0.10;              % after the ultimate, points below this fraction of
                               % ultimate are post-break and are removed

% ---- Error bars ----
stressErrPct = 1.0;            % +/- percent of reading (load cell accuracy)
strainErrAbs = 5e-5;           % +/- strain, same units as the strain axis
nErrorBars   = 15;             % number of points along the curve that get error bars

saveFigure = false;            % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
F  = data(:, iF);
fUnit = units{iF};

if strcmpi(strainSource, 'displacement')
    iD = findCol(names, 'Axial Displacement');
```

```
    e  = data(:, iD) / gaugeLength;
    eUnit = 'in/in';
else
    iE = findCol(names, 'Axial Strain');
    e  = data(:, iE);
    eUnit = units{iE};
end

good = isfinite(F) & isfinite(e);
F = F(good);
e = e(good);
if numel(F) < 5, error('Not enough data points in %s.', datFile); end
if zeroStart, e = e - e(1); end

% Strain recorded in percent instead of in/in
if contains(eUnit, '%')
    strainScale = 100;
else
    strainScale = 1;
end
offsetVal = offset * strainScale;

if max(e) - min(e) < 1e-4 * strainScale
    warning(['The strain barely changes (range %.3g %s). The extensometer may not have ' ...
        'been recording; try strainSource = ''displacement''.'], max(e) - min(e), eUnit);
end

%% Stress
if isempty(specimenArea)
    s = F;
    sUnit  = fUnit;
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
else
    s = F / specimenArea;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';
        case 'kip', sUnit = 'ksi';
        case 'n',   sUnit = 'MPa';    % area in mm^2
        otherwise,  sUnit = [fUnit '/area'];
    end
    sLabel = sprintf('Stress (%s)', sUnit);
end

%% Ultimate and rupture
[sU, iU] = max(s);
iBreak = find(s(iU:end) < breakDrop * sU, 1);
if ~isempty(iBreak)                % drop post-break points
    n = iU + iBreak - 2;
    s = s(1:n);
    e = e(1:n);
end
eU = e(iU);
iR = numel(s);
sR = s(iR);
eR = e(iR);

%% Elastic modulus (linear fit) and 0.2% offset yield
idxFit = find(s(1:iU) >= fitRange(1)*sU & s(1:iU) <= fitRange(2)*sU);
if numel(idxFit) < 2
    error('Fewer than 2 points in the elastic fit range. Widen fitRange.');
end
p = polyfit(e(idxFit), s(idxFit), 1);
E = p(1);
b = p(2);

sY = NaN;
eY = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offsetVal) + b; yield is where the curve
    % first drops below it
    g  = s - (E*(e - offsetVal) + b);
    k0 = idxFit(1);
    k  = find(g(k0:end) <= 0, 1) + k0 - 1;
    if ~isempty(k) && k > 1
        t  = g(k-1) / (g(k-1) - g(k));
        eY = e(k-1) + t*(e(k) - e(k-1));
        sY = s(k-1) + t*(s(k) - s(k-1));
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    warning('The curve never crosses the 0.2% offset line; no yield point.');
end

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [80 80 1400 650]);
ax = axes(fig, 'Position', [0.06 0.11 0.56 0.80]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Error bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
sErr = stressErrPct/100 * abs(s(ib));
eErr = strainErrAbs * ones(size(ib(:)));
hErr = errorbar(ax, e(ib), s(ib), sErr, eErr, eErr, 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offsetVal;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end
```

```
% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
    'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
    'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', sprintf('Error (\\pm%g%% stress, \\pm%g strain)', stressErrPct, strainErrAbs)};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'southeast');

% Results table
if isempty(specimenArea), areaTxt = 'not given'; else, areaTxt = fmt(specimenArea); end
if strcmpi(strainSource, 'displacement'), strainTxt = sprintf('Displacement / %g in', gaugeLength);
else, strainTxt = 'Axial Strain column'; end
[~, fName, fExt] = fileparts(datFile);
rows = {
    'Material',               matName,          ''
    'DAT file',               [fName fExt],     ''
    'Data points',            fmt(iR),          ''
    'Cross-section area',     areaTxt,          'in^2'
    'Strain source',          strainTxt,        ''
    'Elastic modulus E',      fmt(E),           [sUnit '/(' eUnit ')']
    '0.2% yield stress',      fmt(sY),          sUnit
    'Strain at yield',        fmt(eY),          eUnit
    'Ultimate stress',        fmt(sU),          sUnit
    'Strain at ultimate',     fmt(eU),          eUnit
    'Rupture stress',         fmt(sR),          sUnit
    'Strain at rupture',      fmt(eR),          eUnit
    'Stress error',           ['+/- ' fmt(stressErrPct) '%'], 'of reading'
    'Strain error',           ['+/- ' fmt(strainErrAbs)],     eUnit
    };
uitable(fig, 'Data', rows, 'ColumnName', {'Property', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.65 0.11 0.33 0.80], ...
    'ColumnWidth', {150, 140, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Property', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
```

```
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: number to short text for the table
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end
```

This is the same code that's already pushed to the `claude/brave-johnson-5v00jz` branch.

You 2026-10-07 21:52 UTC

why does the graph look ridiculous for all of them. it should be a stress strain curve what is happening

Claude 2026-10-07 21:53 UTC

Your test files don't contain any real strain. Look at the bottom-right of the x-axis: it says $\times 10^{-5}$. The Axial Strain column only moves between about −0.00002 and +0.00001 during the whole test, while the force goes from 0 to 1520 lbf. A real tensile test reaches strains around 0.01–0.2, which is 1,000–10,000 times more. So the extensometer wasn't attached or wasn't recording, and the script plotted the force against electrical noise. That also explains the long gray horizontal lines. The ±5e-5 strain error bars are wider than the whole noise range, so they stretch across the plot.

The force data is fine. So is the **Axial Displacement** column, which is the crosshead movement. I changed the script so that when the strain column is flat it calculates **strain = displacement ÷ gauge length** instead. This is now the default (`strainSource = 'auto'`), and

the Command Window tells you when it switches. I haven't run it in MATLAB, since I don't have it here.

Set `gaugeLength` to your specimen's real gauge length. The strain values depend directly on it. Strain from crosshead displacement also includes some slack and machine stretch, so the elastic modulus will come out lower than the book value. Yield, ultimate and rupture stress are not affected by this.

The updated script, ready to paste. It's also pushed to the branch.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line, error
% bars, and the yield, ultimate and rupture points, plus a table of the
% results in the same figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Stress ----
% [] = plot axial force (lbf) directly as the stress axis.
% Give the cross-sectional area [in^2] to get true engineering stress (psi),
% e.g. round bar: specimenArea = pi*0.505^2/4;  flat bar: specimenArea = 0.5*0.125;
specimenArea = [];

% ---- Strain ----
% 'auto'         = use the Axial Strain column if it actually changes; if it is
%                  flat (extensometer not attached/recording), use
%                  Axial Displacement / gaugeLength instead
% 'strain'       = always use the Axial Strain column (extensometer, in/in)
% 'displacement' = always use Axial Displacement / gaugeLength
strainSource = 'auto';
gaugeLength  = 2.0;            % [in] specimen gauge length, used for displacement-based strain
zeroStart    = true;           % shift strain so the curve starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [0.10 0.40];       % fit elastic slope between these fractions of ultimate stress
breakDrop = 0.10;              % after the ultimate, points below this fraction of
                               % ultimate are post-break and are removed

% ---- Error bars ----
stressErrPct = 1.0;            % +/- percent of reading (load cell accuracy)
strainErrAbs = 5e-5;           % +/- strain, same units as the strain axis
nErrorBars   = 15;             % number of points along the curve that get error bars

saveFigure = false;            % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
F  = data(:, iF);
fUnit = units{iF};

useDisp = strcmpi(strainSource, 'displacement');
if strcmpi(strainSource, 'auto')
    iE = findCol(names, 'Axial Strain');
    eCol = data(isfinite(data(:, iE)), iE);
    scale = 1 + 99*contains(units{iE}, '%');
    % a real test goes well past 0.1% strain; less than that is sensor noise
    useDisp = isempty(eCol) || max(eCol) - min(eCol) < 1e-3*scale;
    if useDisp
        fprintf(['Axial Strain column is flat (range %.3g) - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], ...
            max(eCol) - min(eCol), gaugeLength);
    end
end
if useDisp
    iD = findCol(names, 'Axial Displacement');
    e  = data(:, iD) / gaugeLength;
    eUnit = 'in/in';
else
    iE = findCol(names, 'Axial Strain');
    e  = data(:, iE);
    eUnit = units{iE};
end

good = isfinite(F) & isfinite(e);
F = F(good);
e = e(good);
if numel(F) < 5, error('Not enough data points in %s.', datFile); end
if zeroStart, e = e - e(1); end
```

```
% Strain recorded in percent instead of in/in
if contains(eUnit, '%')
    strainScale = 100;
else
    strainScale = 1;
end
offsetVal = offset * strainScale;

if max(e) - min(e) < 1e-4 * strainScale
    warning(['The strain barely changes (range %.3g %s). The extensometer may not have ' ...
        'been recording; set strainSource = ''auto'' or ''displacement''.'], max(e) - min(e), eUnit);
end

%% Stress
if isempty(specimenArea)
    s = F;
    sUnit  = fUnit;
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
else
    s = F / specimenArea;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';
        case 'kip', sUnit = 'ksi';
        case 'n',   sUnit = 'MPa';    % area in mm^2
        otherwise,  sUnit = [fUnit '/area'];
    end
    sLabel = sprintf('Stress (%s)', sUnit);
end

%% Ultimate and rupture
[sU, iU] = max(s);
iBreak = find(s(iU:end) < breakDrop * sU, 1);
if ~isempty(iBreak)                % drop post-break points
    n = iU + iBreak - 2;
    s = s(1:n);
    e = e(1:n);
end
eU = e(iU);
iR = numel(s);
sR = s(iR);
eR = e(iR);

%% Elastic modulus (linear fit) and 0.2% offset yield
idxFit = find(s(1:iU) >= fitRange(1)*sU & s(1:iU) <= fitRange(2)*sU);
if numel(idxFit) < 2
    error('Fewer than 2 points in the elastic fit range. Widen fitRange.');
end
p = polyfit(e(idxFit), s(idxFit), 1);
E = p(1);
b = p(2);

sY = NaN;
eY = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offsetVal) + b; yield is where the curve
    % first drops below it
    g  = s - (E*(e - offsetVal) + b);
    k0 = idxFit(1);
    k  = find(g(k0:end) <= 0, 1) + k0 - 1;
    if ~isempty(k) && k > 1
        t  = g(k-1) / (g(k-1) - g(k));
        eY = e(k-1) + t*(e(k) - e(k-1));
        sY = s(k-1) + t*(s(k) - s(k-1));
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    warning('The curve never crosses the 0.2% offset line; no yield point.');
end

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [80 80 1400 650]);
ax = axes(fig, 'Position', [0.06 0.11 0.56 0.80]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Error bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
sErr = stressErrPct/100 * abs(s(ib));
eErr = strainErrAbs * ones(size(ib(:)));
hErr = errorbar(ax, e(ib), s(ib), sErr, sErr, eErr, eErr, 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offsetVal;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
    'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
```

```
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
    'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', sprintf('Error (\\pm%g%% stress, \\pm%g strain)', stressErrPct, strainErrAbs)};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'southeast');

% Results table
if isempty(specimenArea), areaTxt = 'not given'; else, areaTxt = fmt(specimenArea); end
if useDisp, strainTxt = sprintf('Displacement / %g in', gaugeLength);
else, strainTxt = 'Axial Strain column'; end
[~, fName, fExt] = fileparts(datFile);
rows = {
    'Material',               matName,          ''
    'DAT file',               [fName fExt],     ''
    'Data points',            fmt(iR),          ''
    'Cross-section area',     areaTxt,          'in^2'
    'Strain source',          strainTxt,        ''
    'Elastic modulus E',      fmt(E),           [sUnit '/(' eUnit ')']
    '0.2% yield stress',      fmt(sY),          sUnit
    'Strain at yield',        fmt(eY),          eUnit
    'Ultimate stress',        fmt(sU),          sUnit
    'Strain at ultimate',     fmt(eU),          eUnit
    'Rupture stress',         fmt(sR),          sUnit
    'Strain at rupture',      fmt(eR),          eUnit
    'Stress error',           ['+/- ' fmt(stressErrPct) '%'], 'of reading'
    'Strain error',           ['+/- ' fmt(strainErrAbs)],     eUnit
    };
uitable(fig, 'Data', rows, 'ColumnName', {'Property', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.65 0.11 0.33 0.80], ...
    'ColumnWidth', {150, 140, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Property', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
```

```
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: number to short text for the table
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end
```

To get real stress in psi instead of force, set `specimenArea` (for example `pi*0.505^2/4` for a 0.505" round bar).

You 2026-10-07 21:56 UTC

📎 `specimen.dat` what would the stress strain curve look like for this data

Claude 2026-10-07 21:58 UTC

Rupture detection is fixed. Checking the updated preview now.

Claude 2026-10-07 21:58 UTC

Looks right. Committing the fix.

Claude 2026-10-07 21:58 UTC

This file gives a normal ductile-metal curve, which fits aluminum (it looks like the same test as your screenshot). I couldn't run MATLAB here, so I copied the script's calculations into Python and drew the preview above from that.

Point	Strain (in/in)	Force (lbf)
0.2% offset yield	0.0115	1128
Ultimate	0.145	1520
Rupture	0.163	1439

The elastic slope comes out to about 1.1 × 10^5 lbf per in/in. All strain values assume the default 2 in gauge length, so they will change when you set your real one.

The script had a rupture bug, now fixed. After the specimen broke, the force dropped from 1439 lbf to about 174 lbf, not to 0. The load cell kept reading about 174 lbf, probably a zero offset or the broken half still hanging in the grip. The old code only treated the test as over when the force fell below 10% of ultimate (152 lbf), so it kept that flat tail and put "rupture" at 173 lbf. That's the square near the bottom of your screenshot. Now any sudden drop of more than 10% of ultimate in one step counts as the break, and rupture is the last point before it.

Things to check in the curve:

- **The bottom of the curve bends before it straightens.** That's slack in the grips and fixture taking up, which you get when strain comes from displacement. It also makes the slope lower than aluminum's real modulus. The yield, ultimate and rupture loads aren't affected.
- **Set `gaugeLength`** to your specimen's real gauge length. Every strain value scales with it.
- **Set `specimenArea`** if you want the y-axis in psi instead of lbf.

Only the "Ultimate and rupture" section and the `breakDrop` comment changed. Here is the new block, to paste over the old one:

```
%% Ultimate and rupture
[sU, iU] = max(s);
% the load cell may not read 0 after the break, so look for the sudden drop
sAfter = s(iU:end);
iBreak = find(diff(sAfter) < -breakDrop*sU | sAfter(2:end) < breakDrop*sU, 1);
if ~isempty(iBreak)                % drop post-break points
    n = iU + iBreak - 1;           % last point before the drop = rupture
    s = s(1:n);
    e = e(1:n);
end
eU = e(iU);
iR = numel(s);
sR = s(iR);
eR = e(iR);
```

The full updated script is pushed to the `claude/brave-johnson-5v00jz` branch. I can paste the whole file again if that's easier.

You 2026-10-08 18:59 UTC

give me the full code to copy and paste

Claude 2026-10-08 19:00 UTC

Here's the full script, including the rupture fix. Paste it into a new MATLAB script, save it as `plotStressStrain.m`, and run it from the Editor. Then change `material`, `gaugeLength` and, if you want psi, `specimenArea` in the settings block at the top.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line, error
% bars, and the yield, ultimate and rupture points, plus a table of the
% results in the same figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Stress ----
% [] = plot axial force (lbf) directly as the stress axis.
% Give the cross-sectional area [in^2] to get true engineering stress (psi),
% e.g. round bar: specimenArea = pi*0.505^2/4;  flat bar: specimenArea = 0.5*0.125;
specimenArea = [];

% ---- Strain ----
% 'auto'         = use the Axial Strain column if it actually changes; if it is
%                  flat (extensometer not attached/recording), use
%                  Axial Displacement / gaugeLength instead
% 'strain'       = always use the Axial Strain column (extensometer, in/in)
% 'displacement' = always use Axial Displacement / gaugeLength
```

```
strainSource = 'auto';
gaugeLength  = 2.0;            % [in] specimen gauge length, used for displacement-based strain
zeroStart    = true;           % shift strain so the curve starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [0.10 0.40];       % fit elastic slope between these fractions of ultimate stress
breakDrop = 0.10;              % break = first point after the ultimate where the load drops by
                               % more than this fraction of ultimate in one step (or falls
                               % below it); everything after the break is removed

% ---- Error bars ----
stressErrPct = 1.0;            % +/- percent of reading (load cell accuracy)
strainErrAbs = 5e-5;           % +/- strain, same units as the strain axis
nErrorBars   = 15;             % number of points along the curve that get error bars

saveFigure = false;            % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
F  = data(:, iF);
fUnit = units{iF};

useDisp = strcmpi(strainSource, 'displacement');
if strcmpi(strainSource, 'auto')
    iE = findCol(names, 'Axial Strain');
    eCol = data(isfinite(data(:, iE)), iE);
    scale = 1 + 99*contains(units{iE}, '%');
    % a real test goes well past 0.1% strain; less than that is sensor noise
    useDisp = isempty(eCol) || max(eCol) - min(eCol) < 1e-3*scale;
    if useDisp
        fprintf(['Axial Strain column is flat (range %.3g) - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], ...
            max(eCol) - min(eCol), gaugeLength);
    end
end
if useDisp
    iD = findCol(names, 'Axial Displacement');
    e  = data(:, iD) / gaugeLength;
    eUnit = 'in/in';
else
    iE = findCol(names, 'Axial Strain');
    e  = data(:, iE);
    eUnit = units{iE};
end

good = isfinite(F) & isfinite(e);
F = F(good);
e = e(good);
if numel(F) < 5, error('Not enough data points in %s.', datFile); end
if zeroStart, e = e - e(1); end

% Strain recorded in percent instead of in/in
if contains(eUnit, '%')
    strainScale = 100;
else
    strainScale = 1;
end
offsetVal = offset * strainScale;

if max(e) - min(e) < 1e-4 * strainScale
    warning(['The strain barely changes (range %.3g %s). The extensometer may not have ' ...
        'been recording; set strainSource = ''auto'' or ''displacement''.'], max(e) - min(e), eUnit);
end

%% Stress
if isempty(specimenArea)
    s = F;
    sUnit  = fUnit;
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
else
    s = F / specimenArea;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';
        case 'kip', sUnit = 'ksi';
        case 'n',   sUnit = 'MPa';    % area in mm^2
        otherwise,  sUnit = [fUnit '/area'];
    end
    sLabel = sprintf('Stress (%s)', sUnit);
end

%% Ultimate and rupture
[sU, iU] = max(s);
% the load cell may not read 0 after the break, so look for the sudden drop
sAfter = s(iU:end);
iBreak = find(diff(sAfter) < -breakDrop*sU | sAfter(2:end) < breakDrop*sU, 1);
if ~isempty(iBreak)                % drop post-break points
    n = iU + iBreak - 1;           % last point before the drop = rupture
    s = s(1:n);
    e = e(1:n);
end
eU = e(iU);
iR = numel(s);
sR = s(iR);
eR = e(iR);

%% Elastic modulus (linear fit) and 0.2% offset yield
idxFit = find(s(1:iU) >= fitRange(1)*sU & s(1:iU) <= fitRange(2)*sU);
```

```
if numel(idxFit) < 2
    error('Fewer than 2 points in the elastic fit range. Widen fitRange.');
end
p = polyfit(e(idxFit), s(idxFit), 1);
E = p(1);
b = p(2);

sY = NaN;
eY = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offsetVal) + b; yield is where the curve
    % first drops below it
    g  = s - (E*(e - offsetVal) + b);
    k0 = idxFit(1);
    k  = find(g(k0:end) <= 0, 1) + k0 - 1;
    if ~isempty(k) && k > 1
        t  = g(k-1) / (g(k-1) - g(k));
        eY = e(k-1) + t*(e(k) - e(k-1));
        sY = s(k-1) + t*(s(k) - s(k-1));
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    warning('The curve never crosses the 0.2% offset line; no yield point.');
end

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [80 80 1400 650]);
ax = axes(fig, 'Position', [0.06 0.11 0.56 0.80]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Error bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
sErr = stressErrPct/100 * abs(s(ib));
eErr = strainErrAbs * ones(size(ib(:)));
hErr = errorbar(ax, e(ib), s(ib), sErr, sErr, eErr, eErr, 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offsetVal;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
    'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
    'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', sprintf('Error (\\pm%g%% stress, \\pm%g strain)', stressErrPct, strainErrAbs)};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'southeast');

% Results table
if isempty(specimenArea), areaTxt = 'not given'; else, areaTxt = fmt(specimenArea); end
if useDisp, strainTxt = sprintf('Displacement / %g in', gaugeLength);
else, strainTxt = 'Axial Strain column'; end
[~, fName, fExt] = fileparts(datFile);
rows = {
    'Material',               matName,          ''
    'DAT file',               [fName fExt],     ''
    'Data points',            fmt(iR),          ''
    'Cross-section area',     areaTxt,          'in^2'
    'Strain source',          strainTxt,        ''
    'Elastic modulus E',      fmt(E),           [sUnit '/(' eUnit ')']
    '0.2% yield stress',      fmt(sY),          sUnit
    'Strain at yield',        fmt(eY),          eUnit
    'Ultimate stress',        fmt(sU),          sUnit
    'Strain at ultimate',     fmt(eU),          eUnit
    'Rupture stress',         fmt(sR),          sUnit
    'Strain at rupture',      fmt(eR),          eUnit
    'Stress error',           ['+/- ' fmt(stressErrPct) '%'], 'of reading'
    'Strain error',           ['+/- ' fmt(strainErrAbs)],     eUnit
    };
uitable(fig, 'Data', rows, 'ColumnName', {'Property', 'Value', 'Units'}, ...
```

```
    'RowName', [], 'Units', 'normalized', 'Position', [0.65 0.11 0.33 0.80], ...
    'ColumnWidth', {150, 140, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Property', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
```

```
    end
end

%% Local function: number to short text for the table
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end
```

This is the same version that's pushed to the `claude/brave-johnson-5v00jz` branch. For your aluminum file it should give the figure from my last message, with rupture at 1439 lbf. That figure came from a Python copy of the script, so this exact code hasn't been run in MATLAB yet.

You 2026-10-08 19:02 UTC

i need the code to detect that point for all my files

Claude 2026-10-08 19:02 UTC

"That point" is the rupture point, I'll assume. The current rule (a drop of more than 10% of ultimate in one sample) misses breaks that happen over several samples, which is common when the sampling rate is high. I'll make it handle that case too and test it on several types of break.

Claude 2026-10-08 19:03 UTC

All cases pick the right point, including both of your files. Applying it to the MATLAB script.

— request interrupted —

You 2026-10-08 19:07 UTC

📎 `specimen.dat` 📎 `specimen.dat` 📎 `specimen.dat` 📎 `specimen.dat` 📎 `specimen.dat` allow me to give you examples of my files. i need all the graphs to look similar and detect the rupture point and not graph after that point. here there is carbon fiber 40, 90, aluminum, steel, and one plastic sample. i have more plastic samples but i need all of these files when graphed in the code to appear like a nice stress strain curve

Claude 2026-10-08 19:10 UTC

All five curves now look right and stop at the break. I'm porting this into the MATLAB script.

Claude 2026-10-08 19:11 UTC

That diff is my own unfinished edit from before you interrupted, so it's safe to replace.

Claude 2026-10-08 19:12 UTC

I'll try to install Octave so I can actually run the MATLAB code on your files rather than relying only on the Python copy.

Claude 2026-10-08 19:14 UTC

Octave needs the helper functions defined before the main code, unlike MATLAB, so I'll reorder the test copies only.

Claude 2026-10-08 19:14 UTC

Octave's `errorbar` doesn't take MATLAB's name/value options. That's an Octave limitation, not a bug in the script, so I'm patching only the test copies.

Claude 2026-10-08 19:14 UTC

The numbers match the Python prototype exactly. Now I'll look at the actual figures.

Claude 2026-10-08 19:16 UTC

All five files now give a clean stress-strain curve that stops at the rupture point (preview above). I ran the script on all five in Octave, a free MATLAB clone, with your files in an `M003\<material>` folder layout. The results matched my Python check. I haven't run it in real MATLAB, and Octave can't draw the results table, so the table hasn't been seen on screen yet.

I guessed which file is which material from the curve shapes, because they're all named `specimen.dat`: the straight one that ends at about 2530 lbf as carbon fiber 90, and the one that curves up to 988 lbf as carbon fiber 40.

Material	0.2% yield	Ultimate	Rupture
Carbon fiber 90	1764 lbf	2530 lbf	2530 lbf (breaks at its maximum)
Carbon fiber 40	351 lbf	988 lbf	984 lbf
Plastic	141 lbf	200 lbf	165 lbf
Steel	1986 lbf	2722 lbf	2635 lbf
Aluminum	1088 lbf	1520 lbf	1488 lbf

What changed:

- **Rupture detection:** the script finds the final drop in load and puts rupture at the start of that drop. Nothing after it is plotted.
- This works whether the break happens in one sample or over several.
- It also works when the load cell reads about 174 lbf (aluminum) or −42 lbf (plastic) after the break.
- Steel tore for a while before the last snap, which is why its rupture is at 2635 lbf rather than the low point before the snap.
- Carbon fiber 90's file ends right at the break, so its ultimate and rupture are the same point.
- **Steel's extensometer:** it was taken off at about 2.1% strain, and after that the strain column stays frozen. The script now uses the extensometer while it was on, then continues with crosshead displacement, scaled to match the extensometer.
- The other four files have no extensometer data, so they use displacement divided by gauge length.
- The small spike near 0.021 on the steel curve is where the extensometer was removed. It's in the raw data.
- **Starting slack ("toe"):** the slack at the beginning of the test is removed, so every curve starts at 0.
- **Elastic slope:** it's now found automatically from the steepest straight part of the curve, so it works for brittle, metal and plastic samples without changing any settings. You can still force a range with `fitRange = [0.10 0.40]`.
- **Yield on carbon fiber:** the script marks a 0.2% offset yield on both carbon fiber samples. Carbon fiber 90 is brittle, so that point probably comes from slight grip slip and may not mean much.

Before you use the numbers: set `gaugeLength`. Every displacement-based strain scales with it.

Here's the full code. Paste it into a new script, save it as `plotStressStrain.m`, and change `material` to run each sample.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line, error
% bars, and the yield, ultimate and rupture points, plus a table of the
% results in the same figure window.
%
% Folder layout:
```

```
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
clear; clc; close all;

%% ====================== USER SETTINGS ======================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Stress ----
% [] = plot axial force (lbf) directly as the stress axis.
% Give the cross-sectional area [in^2] to get true engineering stress (psi),
% e.g. round bar: specimenArea = pi*0.505^2/4;  flat bar: specimenArea = 0.5*0.125;
specimenArea = [];

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gaugeLength after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gaugeLength only
strainSource    = 'auto';
gaugeLength     = 2.0;         % [in] specimen gauge length, used for displacement-based strain
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

% ---- Error bars ----
stressErrPct = 1.0;            % +/- percent of reading (load cell accuracy)
strainErrAbs = 5e-5;           % +/- strain (in/in)
nErrorBars   = 15;             % number of points along the curve that get error bars

saveFigure = false;            % true = save .png and .fig into the material folder
%% ============================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};

good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', datFile); end

F  = data(:, iF);
eD = data(:, iD) / gaugeLength;
eD = eD - eD(1);

%% Strain
e = eD;
strainTxt = sprintf('Displacement / %g in', gaugeLength);
if ~strcmpi(strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(strainSource, 'strain')
        e = eX;
        strainTxt = 'Extensometer';
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        if iRem < numel(e) - 5
            strainTxt = sprintf('Extensometer to %.3g, then displacement', eX(iRem));
            fprintf('Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        else
            strainTxt = 'Extensometer';
        end
    else
        fprintf(['Axial Strain column is flat - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], gaugeLength);
    end
end
```

```
eUnit = 'in/in';

%% Stress
if isempty(specimenArea)
    s = F;
    sUnit  = fUnit;
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
else
    s = F / specimenArea;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';
        case 'kip', sUnit = 'ksi';
        case 'n',   sUnit = 'MPa';    % area in mm^2
        otherwise,  sUnit = [fUnit '/area'];
    end
    sLabel = sprintf('Stress (%s)', sUnit);
end

%% Break: biggest sudden load drop after the ultimate
% It can be one sample or spread over a few, and the load cell may not read
% 0 afterwards (it can even go negative).
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));               % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > breakDrop*sU
            iR = iU + a - 1;        % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('No break found in the data; rupture is the last data point.\n');
end

%% Elastic modulus (linear fit)
[E, b, idxFit] = fitElastic(e(1:iR), s(1:iR), fitRange);

%% Rupture = start of the final load drop
% Ductile samples can tear for a while before the last snap; walk back from
% the snap while the curve is still falling steeply (> 5% of E).
if E > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E
        j = j - 1;
    end
    if j < iR
        lo = max(iU, j - w);
        [~, k] = max(s(lo:j));
        iR = lo + k - 1;
    end
end
s = s(1:iR);
e = e(1:iR);

%% Toe compensation: shift so the elastic line passes through 0
kStart = idxFit(1);
if toeCompensation && E > 0
    e0 = -b/E;
    e  = [0; e(idxFit(1):end) - e0];
    s  = [0; s(idxFit(1):end)];
    b  = 0;
    kStart = 2;
end

%% Ultimate, rupture and 0.2% offset yield
[sU, iU] = max(s);
eU = e(iU);
iR = numel(s);
sR = s(iR);
eR = e(iR);

sY = NaN;
eY = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    g = s - (E*(e - offset) + b);
    k = find(g(kStart:end) <= 0, 1) + kStart - 1;
    if ~isempty(k) && k > 1
        t  = g(k-1) / (g(k-1) - g(k));
        eY = e(k-1) + t*(e(k) - e(k-1));
        sY = s(k-1) + t*(s(k) - s(k-1));
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    fprintf('The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
end

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [80 80 1400 650]);
ax = axes(fig, 'Position', [0.06 0.11 0.56 0.80]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Error bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
sErr = stressErrPct/100 * abs(s(ib));
```

```
eErr = strainErrAbs * ones(size(ib(:)));
hErr = errorbar(ax, e(ib), s(ib), sErr, sErr, eErr, eErr, 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offset;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                          % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.4g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', sprintf('Error (\\pm%g%% stress, \\pm%g strain)', stressErrPct, strainErrAbs)};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

% Results table
if isempty(specimenArea), areaTxt = 'not given'; else, areaTxt = fmt(specimenArea); end
[~, fName, fExt] = fileparts(datFile);
rows = {
    'Material',               matName,          ''
    'DAT file',               [fName fExt],     ''
    'Data points',            fmt(iR),          ''
    'Cross-section area',     areaTxt,          'in^2'
    'Strain source',          strainTxt,        ''
    'Elastic modulus E',      fmt(E),           sUnit
    '0.2% yield stress',      fmt(sY),          sUnit
    'Strain at yield',        fmt(eY),          eUnit
    'Ultimate stress',        fmt(sU),          sUnit
    'Strain at ultimate',     fmt(eU),          eUnit
    'Rupture stress',         fmt(sR),          sUnit
    'Strain at rupture',      fmt(eR),          eUnit
    'Stress error',           ['+/- ' fmt(stressErrPct) '%'], 'of reading'
    'Strain error',           ['+/- ' fmt(strainErrAbs)],     eUnit
    };
uitable(fig, 'Data', rows, 'ColumnName', {'Property', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.65 0.11 0.33 0.80], ...
    'ColumnWidth', {150, 160, 90}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Property', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
```

```
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: number to short text for the table
```

```
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end
```

This version is pushed to the `claude/brave-johnson-5v00jz` branch. If any of your other plastic samples comes out wrong, send me the file and I'll adjust the detection.

You 2026-10-08 19:24 UTC

PERFECT! okay now i just need the table to contain the following: All lab measurements w/ uncertainties
ii)
Ultimate Stress and Strain w/ Uncertainties
iii) Rupture Stress and Strain
iv) Yield Stress and Strain
v) Young’s Modulus w/ Uncertainties
vi) Modulus of Toughness w/ uncertainties
vii) Modulus of resilience w/ uncertainties . let me know if there is any missing information you need from me

Claude 2026-10-08 19:25 UTC

To fill this table I need numbers from you that aren't in the .dat files:

- **Cross-sectional area (width and thickness, or diameter) for each specimen.** Yield, ultimate and rupture stress, Young's modulus, toughness and resilience are all stress-based, and stress is force divided by area.
- **The real gauge length.**
- **How precise each instrument is.**

I'll build the script so you enter these in one table at the top, then tell you exactly which values to send me.

Claude 2026-10-08 19:26 UTC

Now I'll test it in Octave on all five files, once with no dimensions entered and once with made-up dimensions to exercise the stress path.

Claude 2026-10-08 19:27 UTC

The formatting reads well now. Committing.

Claude 2026-10-08 19:29 UTC

The table now has every item you listed. I ran it in Octave on all five sample files, once with no dimensions and once with made-up dimensions, and every value and uncertainty came out. Octave can't draw the table, so I checked the numbers in its text output; the table itself hasn't been seen on screen in MATLAB yet.

The table:

- **Lab measurements:** width and thickness (or diameter), cross-sectional area, gauge length, load cell accuracy, crosshead displacement accuracy, extensometer accuracy if one was used, and where the strain came from.
- **Results, each with ±:**
- ultimate stress and strain
- rupture stress and strain
- yield stress and strain (0.2% offset)
- Young's modulus
- modulus of toughness (area under the curve)

- modulus of resilience (yield stress² ÷ 2E)

How each uncertainty is combined is written out in the comments at the top of the script.

What I need from you

1. **Specimen dimensions** for each sample: width and thickness for flat specimens, or diameter for round ones. - Until you enter them, the script plots force in lbf instead of stress, and the moduli come out in lbf units.
2. **The real gauge length** for each sample. I used 2.0 in as a placeholder.
3. **Your material folder names.** Each row in the `specimens` table is matched by a word that appears in the folder name. Your aluminum folder is called `m003alum`, so the key `'alum'` already matches it. I guessed the other keys (`'steel'`, `'carbon fiber 40'`, `'carbon fiber 90'`, `'plastic'`). Change them to a piece of each real folder name. Give each plastic sample its own row if their dimensions differ.
4. **Instrument uncertainties**, if your lab handout gives them. My placeholders are: - calipers ±0.0005 in - gauge length ±0.01 in - load cell ±1% of reading - crosshead ±0.001 in - extensometer ±5e-5 in/in

Young's modulus will come out low for every sample except steel. Those samples had no extensometer, so strain comes from crosshead movement, which includes stretch in the machine and grips. With made-up dimensions, aluminum gave about 1.9 Mpsi; real aluminum is about 10 Mpsi. Steel used the extensometer for the elastic part, so its modulus is the most trustworthy. Resilience depends on the modulus, so it is affected the same way. You may want to mention this in your report.

The full code is below and is also pushed to the branch. Fill in the `specimens` table at the top once, then just change `material` for each run.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line,
% uncertainty bars, and the yield, ultimate and rupture points, plus a table
% of the lab measurements and results (with uncertainties) in the same
% figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
% Enter each specimen's measurements once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve     dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)            dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% key:   matched against the material folder name (not case sensitive;
%        the longest key found in the folder name wins, so 'plastic 2'
%        beats 'plastic')
% shape: 'rect' (width x thickness) or 'round' (diameter, thickness = NaN)
% NaN = not measured yet -> the stress axis falls back to axial force (lbf)
specimens = {
%   key              shape    width/diam  thickness  gauge length
    'alum',          'rect',  NaN,        NaN,       2.0
    'steel',         'rect',  NaN,        NaN,       2.0
    'carbon fiber 40','rect', NaN,        NaN,       2.0
    'carbon fiber 90','rect', NaN,        NaN,       2.0
    'plastic',       'rect',  NaN,        NaN,       2.0
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.0005;           % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.01;             % +/- in, gauge length
loadUncPct = 1.0;              % +/- percent of reading, load cell
```

```
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;         % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;              % offset strain (0.002 = 0.2%)
fitRange  = [];                 % [] = automatic (steepest straight part of the curve), or
                                % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;               % a sudden load drop bigger than this fraction of ultimate
                                % is the break; nothing after the rupture is plotted

nErrorBars = 15;                % number of points along the curve that get uncertainty bars
saveFigure = false;             % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Specimen geometry
[shape, dim1, dim2, L0] = lookupSpecimen(specimens, matName);
if strcmpi(shape, 'round')
    A    = pi*dim1^2/4;
    relA = 2*dimUnc/dim1;
else
    A    = dim1*dim2;
    relA = sqrt((dimUnc/dim1)^2 + (dimUnc/dim2)^2);
end
haveArea = isfinite(A) && A > 0;
if ~haveArea
    relA = 0;
    fprintf('No specimen dimensions for "%s" - plotting axial force instead of stress.\n', matName);
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', matName);
end

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};

good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', datFile); end

F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

%% Strain
e = eD;
isExt = false(size(e));             % which points come from the extensometer
strainTxt = sprintf('Displacement / %g in', L0);
if ~strcmpi(strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(strainSource, 'strain')
        e = eX;
        isExt(:) = true;
        strainTxt = 'Extensometer';
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            strainTxt = sprintf('Extensometer to %.3g, then displacement', eX(iRem));
            fprintf('Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        else
            strainTxt = 'Extensometer';
        end
    else
        fprintf(['Axial Strain column is flat - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
eUnit = 'in/in';
eUnc = sqrt((dispUnc/L0)^2 + (e*gaugeUnc/L0).^2);
eUnc(isExt) = extUnc;

%% Stress
if haveArea
    s = F / A;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';   uUnit = 'in-lbf/in^3';
        case 'kip', sUnit = 'ksi';   uUnit = 'in-kip/in^3';
```

```
        case 'n',   sUnit = 'MPa';   uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  sUnit = [fUnit '/area']; uUnit = sUnit;
    end
    sLabel = sprintf('Engineering Stress (%s)', sUnit);
else
    s = F;
    sUnit  = fUnit;
    uUnit  = [fUnit '*in/in'];
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
end
relS = sqrt((loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

%% Break: biggest sudden load drop after the ultimate
% It can be one sample or spread over a few, and the load cell may not read
% 0 afterwards (it can even go negative).
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));               % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > breakDrop*sU
            iR = iU + a - 1;        % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('No break found in the data; rupture is the last data point.\n');
end

%% Young's modulus (linear fit) and its uncertainty
[E, b, idxFit] = fitElastic(e(1:iR), s(1:iR), fitRange);
ef = e(idxFit);
sf = s(idxFit);
res = sf - (E*ef + b);
seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
spanFit = max(ef) - min(ef);
if isExt(idxFit(1))
    relEfit = sqrt(2)*extUnc/spanFit;
else
    relEfit = sqrt((sqrt(2)*dispUnc/L0/spanFit)^2 + (gaugeUnc/L0)^2);
end
EUnc = abs(E) * sqrt((seE/E)^2 + relS^2 + relEfit^2);

%% Rupture = start of the final load drop
% Ductile samples can tear for a while before the last snap; walk back from
% the snap while the curve is still falling steeply (> 5% of E).
if E > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E
        j = j - 1;
    end
    if j < iR
        lo = max(iU, j - w);
        [~, k] = max(s(lo:j));
        iR = lo + k - 1;
    end
end
s    = s(1:iR);
e    = e(1:iR);
sUnc = sUnc(1:iR);
eUnc = eUnc(1:iR);

%% Toe compensation: shift so the elastic line passes through 0
kStart = idxFit(1);
if toeCompensation && E > 0
    e0   = -b/E;
    e    = [0; e(idxFit(1):end) - e0];
    s    = [0; s(idxFit(1):end)];
    sUnc = [0; sUnc(idxFit(1):end)];
    eUnc = [0; eUnc(idxFit(1):end)];
    b    = 0;
    kStart = 2;
end

%% Ultimate, rupture and 0.2% offset yield
[sU, iU] = max(s);
eU  = e(iU);    sUu = sUnc(iU);    eUu = eUnc(iU);
iR  = numel(s);
sR  = s(iR);    sRu = sUnc(iR);    eR  = e(iR);    eRu = eUnc(iR);

sY = NaN;  eY = NaN;  sYu = NaN;  eYu = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    g = s - (E*(e - offset) + b);
    k = find(g(kStart:end) <= 0, 1) + kStart - 1;
    if ~isempty(k) && k > 1
        t   = g(k-1) / (g(k-1) - g(k));
        eY  = e(k-1) + t*(e(k) - e(k-1));
        sY  = s(k-1) + t*(s(k) - s(k-1));
        sYu = abs(sY) * relS;
        eYu = eUnc(k);
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    fprintf('The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
end

%% Modulus of toughness (area under the curve) and resilience (sY^2 / 2E)
```

```
UT  = trapz(e, s);
UTu = abs(UT) * sqrt(relS^2 + (eRu/eR)^2);
Ur  = sY^2 / (2*E);
Uru = abs(Ur) * sqrt((2*sYu/sY)^2 + (EUnc/E)^2);

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
hErr = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offset;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                          % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.4g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

%% Results table
if strcmpi(shape, 'round')
    dimRows = {'Diameter', pm(dim1, dimUnc), 'in'};
else
    dimRows = {'Width',     pm(dim1, dimUnc), 'in'
               'Thickness', pm(dim2, dimUnc), 'in'};
end
if haveArea, areaRow = {'Cross-section area', pm(A, A*relA), 'in^2'};
else,        areaRow = {'Cross-section area', 'not entered', 'in^2'}; end
if any(isExt), extRow = {'Extensometer', ['+/- ' fmt(extUnc)], 'in/in'};
else,          extRow = cell(0, 3); end

rows = [
    {'LAB MEASUREMENTS', '', ''}
    dimRows
    areaRow
    {'Gauge length', pm(L0, gaugeUnc), 'in'}
    {'Load cell', ['+/- ' fmt(loadUncPct) '% of reading'], ''}
    {'Crosshead displacement', ['+/- ' fmt(dispUnc)], 'in'}
    extRow
    {'Strain source', strainTxt, ''}
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(sU, sUu),   sUnit}
    {'Strain at ultimate',       pm(eU, eUu),   eUnit}
    {'Rupture stress',           pm(sR, sRu),   sUnit}
    {'Strain at rupture',        pm(eR, eRu),   eUnit}
    {'Yield stress (0.2%)',      pm(sY, sYu),   sUnit}
    {'Strain at yield',          pm(eY, eYu),   eUnit}
    {'Young''s modulus E',       pm(E, EUnc),   sUnit}
    {'Modulus of toughness',     pm(UT, UTu),   uUnit}
    {'Modulus of resilience',    pm(Ur, Uru),   uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Quantity', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
```

```
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: specimen row whose key appears in the folder name
function [shape, dim1, dim2, L0] = lookupSpecimen(specimens, matName)
    keys = specimens(:, 1);
    hit  = find(cellfun(@(k) contains(lower(matName), lower(k)), keys));
    if isempty(hit)
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    [~, i] = max(cellfun(@numel, keys(hit)));        % most specific key
    row   = specimens(hit(i), :);
    shape = row{2};
    dim1  = row{3};
    dim2  = row{4};
    L0    = row{5};
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
```

```
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: number to short text for the table
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

Send me your dimensions, gauge lengths and folder names and I'll fill in the table for you.

You 2026-10-08 19:43 UTC

Steel Aluminum CF 90 CF 45
L_i 2.7535 in 2.643 in 2.5110 in 2.8900 in
W_i 0.5100 in 0.516 in 0.5240 in 0.5145 in
t_i 0.0650 in 0.0615 in 0.0605 in 0.0565 in
L_f 3.0065 in 2.8455 in 2.5640 in 2.9610 in
W_f 0.4355 in 0.465 in 0.5205 in 0.4705 in
t_f 0.0575 in 0.055 in 0.0600 in 0.0615 in. the uncertainty for all is 0.00025 in

Claude 2026-10-08 19:44 UTC

You gave initial and final measurements, so I'll also add **% elongation** and **% reduction of area** (with uncertainties) to the lab-measurements section. Li becomes the gauge length for displacement-based strain. All measurements use your ±0.00025 in uncertainty.

Claude 2026-10-08 19:47 UTC

I put your measurements into the script and added two rows, **% elongation** and **% reduction of area**, worked out from your before and after measurements. Initial length is now the gauge length used to turn displacement into strain, and every measurement uses ±0.00025 in. I ran it on all five sample files in Octave and every row came out. Octave can't draw the table, so the figure itself hasn't been seen in MATLAB yet.

	Steel	Aluminum	CF 90	CF 45
Ultimate stress (psi)	82,110 ± 881	47,906 ± 518	79,796 ± 864	33,975 ± 372
Yield stress, 0.2% (psi)	59,919 ± 643	36,049 ± 390	56,995 ± 617	12,769 ± 140
Rupture stress (psi)	79,485 ± 853	46,896 ± 507	79,796 ± 864	33,844 ± 370
Young's modulus (psi)	(3.07 ± 0.46)e7	(4.9 ± 1.2)e6	(3.19 ± 0.30)e6	(9.87 ± 0.61)e5
Toughness (in-lbf/in³)	12,515 ± 137	5,213 ± 59	1,545 ± 25	5,263 ± 58
Resilience (in-lbf/in³)	58.5 ± 8.9	132 ± 33	509 ± 50	82.6 ± 5.4
Elongation	9.19 ± 0.01 %	7.66 ± 0.01 %	2.11 ± 0.01 %	2.46 ± 0.01 %
Reduction of area	24.5 ± 0.4 %	19.4 ± 0.5 %	1.5 ± 0.6 %	0.5 ± 0.6 %

The strains at ultimate, rupture and yield are in the figure's table too.

Things to know before you use these:

- **Steel's modulus (30.7 Mpsi) matches the textbook value** (about 29–30). Steel is the only sample where the extensometer measured the elastic part.
- **The other moduli are low.** Their strain comes from crosshead movement, which includes stretch in the machine and grips. Aluminum gives 4.9 Mpsi against about 10 Mpsi real. Resilience is affected the same way.
- **Strain read off the curve is larger than your measured elongation.**
- Steel's curve reaches 16% strain at rupture, but the specimen only grew 9.2%.
- CF 45 is the extreme case. The crosshead moved about 0.6 in, but the specimen only got 0.07 in longer, which points to grip slip.
- Use the measured % elongation for ductility.
- **CF 45's final thickness (0.0615 in) is larger than its initial thickness (0.0565 in).** That could be delamination or a measurement mix-up. It's why its reduction of area is about 0 ± 0.6%.
- **CF 90 reduction of area:** your table had "CF 90" and "CF 45", but earlier you called the second one CF 40. The script matches either name.
- **Plastic still needs its measurements.** It currently shows force in lbf and N/A for elongation and reduction of area. Send me those numbers, including for your other plastic samples, and I'll add them.
- **Folder names:** spaces, punctuation and capital letters are ignored, so a folder named `m003alum`, `CF 90` or `Carbon Fiber 90` will match.
- Keys used: `steel`, `alum`, `cf90` or `carbonfiber90`, `cf45`, `cf40` or `carbonfiber40`, and `plastic`.
- If a folder doesn't match, the error message tells you which folder name to add as a key.

The full code is below and is also pushed to the branch:

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line,
% uncertainty bars, and the yield, ultimate and rupture points, plus a table
% of the lab measurements and results (with uncertainties) in the same
% figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
% Enter each specimen's measurements once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Ductility (from the before/after measurements):
```

```
%   elongation      %EL = (Lf - Li)/Li * 100
%   reduction area  %RA = (Ai - Af)/Ai * 100
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> width/thickness: the stress axis falls back to
%       axial force (lbf); final values: %EL / %RA show N/A
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 2.0,    NaN,    NaN,    NaN,    NaN,    NaN      % <-- placeholder, enter measurements
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along the curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Specimen geometry
[shape, L0, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName);
[A,  relA ] = sectionArea(shape, Wi, ti, dimUnc);
[Af, relAf] = sectionArea(shape, Wf, tf, dimUnc);

% Ductility from the before/after measurements
EL   = (Lf - L0)/L0 * 100;
ELu  = 100 * sqrt((gaugeUnc/L0)^2 + (Lf*gaugeUnc/L0^2)^2);
RA   = (A - Af)/A * 100;
RAu  = 100 * (Af/A) * sqrt(relA^2 + relAf^2);

haveArea = isfinite(A) && A > 0;
if ~haveArea
    relA = 0;
    fprintf('No specimen dimensions for "%s" - plotting axial force instead of stress.\n', matName);
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', matName);
end

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};

good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', datFile); end

F  = data(:, iF);
```

```
eD = data(:, iD) / L0;
eD = eD - eD(1);

%% Strain
e = eD;
isExt = false(size(e));             % which points come from the extensometer
strainTxt = sprintf('Displacement / %g in', L0);
if ~strcmpi(strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(strainSource, 'strain')
        e = eX;
        isExt(:) = true;
        strainTxt = 'Extensometer';
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            strainTxt = sprintf('Extensometer to %.3g, then displacement', eX(iRem));
            fprintf('Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        else
            strainTxt = 'Extensometer';
        end
    else
        fprintf(['Axial Strain column is flat - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
eUnit = 'in/in';
eUnc = sqrt((dispUnc/L0)^2 + (e*gaugeUnc/L0).^2);
eUnc(isExt) = extUnc;

%% Stress
if haveArea
    s = F / A;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';   uUnit = 'in-lbf/in^3';
        case 'kip', sUnit = 'ksi';   uUnit = 'in-kip/in^3';
        case 'n',   sUnit = 'MPa';   uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  sUnit = [fUnit '/area']; uUnit = sUnit;
    end
    sLabel = sprintf('Engineering Stress (%s)', sUnit);
else
    s = F;
    sUnit  = fUnit;
    uUnit  = [fUnit '*in/in'];
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
end
relS = sqrt((loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

%% Break: biggest sudden load drop after the ultimate
% It can be one sample or spread over a few, and the load cell may not read
% 0 afterwards (it can even go negative).
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));               % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > breakDrop*sU
            iR = iU + a - 1;        % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('No break found in the data; rupture is the last data point.\n');
end

%% Young's modulus (linear fit) and its uncertainty
[E, b, idxFit] = fitElastic(e(1:iR), s(1:iR), fitRange);
ef = e(idxFit);
sf = s(idxFit);
res = sf - (E*ef + b);
seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
spanFit = max(ef) - min(ef);
if isExt(idxFit(1))
    relEfit = sqrt(2)*extUnc/spanFit;
else
    relEfit = sqrt((sqrt(2)*dispUnc/L0/spanFit)^2 + (gaugeUnc/L0)^2);
end
EUnc = abs(E) * sqrt((seE/E)^2 + relS^2 + relEfit^2);

%% Rupture = start of the final load drop
% Ductile samples can tear for a while before the last snap; walk back from
% the snap while the curve is still falling steeply (> 5% of E).
if E > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E
        j = j - 1;
    end
    if j < iR
        lo = max(iU, j - w);
        [~, k] = max(s(lo:j));
        iR = lo + k - 1;
```

```
    end
end
s    = s(1:iR);
e    = e(1:iR);
sUnc = sUnc(1:iR);
eUnc = eUnc(1:iR);

%% Toe compensation: shift so the elastic line passes through 0
kStart = idxFit(1);
if toeCompensation && E > 0
    e0   = -b/E;
    e    = [0; e(idxFit(1):end) - e0];
    s    = [0; s(idxFit(1):end)];
    sUnc = [0; sUnc(idxFit(1):end)];
    eUnc = [0; eUnc(idxFit(1):end)];
    b    = 0;
    kStart = 2;
end

%% Ultimate, rupture and 0.2% offset yield
[sU, iU] = max(s);
eU  = e(iU);    sUu = sUnc(iU);    eUu = eUnc(iU);
iR  = numel(s);
sR  = s(iR);    sRu = sUnc(iR);    eR  = e(iR);    eRu = eUnc(iR);

sY = NaN;  eY = NaN;  sYu = NaN;  eYu = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    g = s - (E*(e - offset) + b);
    k = find(g(kStart:end) <= 0, 1) + kStart - 1;
    if ~isempty(k) && k > 1
        t   = g(k-1) / (g(k-1) - g(k));
        eY  = e(k-1) + t*(e(k) - e(k-1));
        sY  = s(k-1) + t*(s(k) - s(k-1));
        sYu = abs(sY) * relS;
        eYu = eUnc(k);
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    fprintf('The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
end

%% Modulus of toughness (area under the curve) and resilience (sY^2 / 2E)
UT  = trapz(e, s);
UTu = abs(UT) * sqrt(relS^2 + (eRu/eR)^2);
Ur  = sY^2 / (2*E);
Uru = abs(Ur) * sqrt((2*sYu/sY)^2 + (EUnc/E)^2);

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
hErr = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offset;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                          % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.4g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
```

```
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

%% Results table
if strcmpi(shape, 'round')
    dimRows = {'Initial diameter', pm(Wi, dimUnc), 'in'
               'Final diameter',   pm(Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(Wi, dimUnc), 'in'
               'Initial thickness', pm(ti, dimUnc), 'in'
               'Final width',       pm(Wf, dimUnc), 'in'
               'Final thickness',   pm(tf, dimUnc), 'in'};
end
areaRows = {'Initial area', pm(A,  A*relA),   'in^2'
            'Final area',   pm(Af, Af*relAf), 'in^2'};
if any(isExt), extRow = {'Extensometer', ['+/- ' fmt(extUnc)], 'in/in'};
else,          extRow = cell(0, 3); end

rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(Lf, gaugeUnc), 'in'}
    dimRows
    areaRows
    {'Load cell', ['+/- ' fmt(loadUncPct) '% of reading'], ''}
    {'Crosshead displacement', ['+/- ' fmt(dispUnc)], 'in'}
    extRow
    {'Strain source', strainTxt, ''}
    {'RESULTS', '', ''}
    {'Elongation',              pm(EL, ELu),   '%'}
    {'Reduction of area',       pm(RA, RAu),   '%'}
    {'Ultimate stress',         pm(sU, sUu),   sUnit}
    {'Strain at ultimate',      pm(eU, eUu),   eUnit}
    {'Rupture stress',          pm(sR, sRu),   sUnit}
    {'Strain at rupture',       pm(eR, eRu),   eUnit}
    {'Yield stress (0.2%)',     pm(sY, sYu),   sUnit}
    {'Strain at yield',         pm(eY, eYu),   eUnit}
    {'Young''s modulus E',      pm(E, EUnc),   sUnit}
    {'Modulus of toughness',    pm(UT, UTu),   uUnit}
    {'Modulus of resilience',   pm(Ur, Uru),   uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Quantity', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: specimen row whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(matName);
    best = 0;
    bestLen = 0;
    for r = 1:size(specimens, 1)
        for key = strsplit(norm(specimens{r, 1}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
```

```
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
```

```
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: number to short text for the table
function str = fmt(x)
    if isnan(x)
        str = 'N/A';
    else
        str = sprintf('%.5g', x);
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-08 19:54 UTC

amazing and perfect. okay get rid of any other variables in the chart that are not something i have named already

Claude 2026-10-08 19:54 UTC

By "chart" I'm taking you to mean the results table, since the plot only shows things you asked for. I'll keep your six measurements (L_i, W_i, t_i, L_f, W_f, t_f) and the seven results, and remove the rows you never named: the two areas, load cell, crosshead, extensometer, strain source, elongation and reduction of area. The load cell and other uncertainties still go into the calculations; they just won't be listed.

Claude 2026-10-08 19:55 UTC

`strainTxt` and `fmt` are no longer used anywhere, so I'll remove those too.

Claude 2026-10-08 19:56 UTC

The table now lists only the measurements and results you asked for. I re-ran steel and plastic in Octave and the remaining values are exactly the same as before. As before, I checked the table's text output, not the drawn table in MATLAB.

Lab measurements (each ± 0.00025 in): initial and final gauge length (L_i, L_f), initial and final width (W_i, W_f), initial and final thickness (t_i, t_f).

Results, each with ±: ultimate stress and strain, rupture stress and strain, 0.2% yield stress and strain, Young's modulus, modulus of toughness, modulus of resilience.

Removed: initial and final area, load cell, crosshead displacement, extensometer, strain source, elongation and reduction of area. The load cell, crosshead and extensometer uncertainties still go into every ± value; they're just not listed in the table. The Command Window still prints a line when the extensometer was missing or removed.

Here is the updated full code. It's also pushed to the branch.

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line,
% uncertainty bars, and the yield, ultimate and rupture points, plus a table
% of the lab measurements and results (with uncertainties) in the same
% figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
% Enter each specimen's measurements once in the "specimens" table.
```

```
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 2.0,    NaN,    NaN,    NaN,    NaN,    NaN      % <-- placeholder, enter measurements
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along the curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig into the material folder
%% ========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:       %s\n', matPath, datFile);

%% Specimen geometry
[shape, L0, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName);
[A, relA] = sectionArea(shape, Wi, ti, dimUnc);

haveArea = isfinite(A) && A > 0;
if ~haveArea
    relA = 0;
    fprintf('No specimen dimensions for "%s" - plotting axial force instead of stress.\n', matName);
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', matName);
end

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};

good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', datFile); end
```

```
F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

%% Strain
e = eD;
isExt = false(size(e));             % which points come from the extensometer
if ~strcmpi(strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(strainSource, 'strain')
        e = eX;
        isExt(:) = true;
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            fprintf('Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        end
    else
        fprintf(['Axial Strain column is flat - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
eUnit = 'in/in';
eUnc = sqrt((dispUnc/L0)^2 + (e*gaugeUnc/L0).^2);
eUnc(isExt) = extUnc;

%% Stress
if haveArea
    s = F / A;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';   uUnit = 'in-lbf/in^3';
        case 'kip', sUnit = 'ksi';   uUnit = 'in-kip/in^3';
        case 'n',   sUnit = 'MPa';   uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  sUnit = [fUnit '/area']; uUnit = sUnit;
    end
    sLabel = sprintf('Engineering Stress (%s)', sUnit);
else
    s = F;
    sUnit  = fUnit;
    uUnit  = [fUnit '*in/in'];
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
end
relS = sqrt((loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

%% Break: biggest sudden load drop after the ultimate
% It can be one sample or spread over a few, and the load cell may not read
% 0 afterwards (it can even go negative).
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));               % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > breakDrop*sU
            iR = iU + a - 1;        % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('No break found in the data; rupture is the last data point.\n');
end

%% Young's modulus (linear fit) and its uncertainty
[E, b, idxFit] = fitElastic(e(1:iR), s(1:iR), fitRange);
ef = e(idxFit);
sf = s(idxFit);
res = sf - (E*ef + b);
seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
spanFit = max(ef) - min(ef);
if isExt(idxFit(1))
    relEfit = sqrt(2)*extUnc/spanFit;
else
    relEfit = sqrt((sqrt(2)*dispUnc/L0/spanFit)^2 + (gaugeUnc/L0)^2);
end
EUnc = abs(E) * sqrt((seE/E)^2 + relS^2 + relEfit^2);

%% Rupture = start of the final load drop
% Ductile samples can tear for a while before the last snap; walk back from
% the snap while the curve is still falling steeply (> 5% of E).
if E > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E
        j = j - 1;
    end
    if j < iR
        lo = max(iU, j - w);
        [~, k] = max(s(lo:j));
        iR = lo + k - 1;
    end
end
s    = s(1:iR);
```

```
e    = e(1:iR);
sUnc = sUnc(1:iR);
eUnc = eUnc(1:iR);

%% Toe compensation: shift so the elastic line passes through 0
kStart = idxFit(1);
if toeCompensation && E > 0
    e0   = -b/E;
    e    = [0; e(idxFit(1):end) - e0];
    s    = [0; s(idxFit(1):end)];
    sUnc = [0; sUnc(idxFit(1):end)];
    eUnc = [0; eUnc(idxFit(1):end)];
    b    = 0;
    kStart = 2;
end

%% Ultimate, rupture and 0.2% offset yield
[sU, iU] = max(s);
eU  = e(iU);    sUu = sUnc(iU);    eUu = eUnc(iU);
iR  = numel(s);
sR  = s(iR);    sRu = sUnc(iR);    eR  = e(iR);    eRu = eUnc(iR);

sY = NaN;  eY = NaN;  sYu = NaN;  eYu = NaN;
if isfinite(E) && E > 0
    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    g = s - (E*(e - offset) + b);
    k = find(g(kStart:end) <= 0, 1) + kStart - 1;
    if ~isempty(k) && k > 1
        t   = g(k-1) / (g(k-1) - g(k));
        eY  = e(k-1) + t*(e(k) - e(k-1));
        sY  = s(k-1) + t*(s(k) - s(k-1));
        sYu = abs(sY) * relS;
        eYu = eUnc(k);
    end
else
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
end
if isnan(sY)
    fprintf('The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
end

%% Modulus of toughness (area under the curve) and resilience (sY^2 / 2E)
UT  = trapz(e, s);
UTu = abs(UT) * sqrt(relS^2 + (eRu/eR)^2);
Ur  = sY^2 / (2*E);
Uru = abs(Ur) * sqrt((2*sYu/sY)^2 + (EUnc/E)^2);

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
ib   = unique(round(linspace(1, iR, min(nErrorBars, iR))));
hErr = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
    'Color', [0.5 0.5 0.5], 'CapSize', 4);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    if isfinite(sY), sTop = 1.15*sY; else, sTop = sU; end
    sLine = [0 sTop];
    eLine = (sLine - b)/E + offset;
    hOff  = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.4g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                              % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.4g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.4g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.4g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
```

```
legend(ax, hLeg, legTxt, 'Location', 'best');

%% Results table
if strcmpi(shape, 'round')
    dimRows = {'Initial diameter', pm(Wi, dimUnc), 'in'
               'Final diameter',   pm(Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(Wi, dimUnc), 'in'
               'Initial thickness', pm(ti, dimUnc), 'in'
               'Final width',       pm(Wf, dimUnc), 'in'
               'Final thickness',   pm(tf, dimUnc), 'in'};
end

rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(sU, sUu),   sUnit}
    {'Strain at ultimate',       pm(eU, eUu),   eUnit}
    {'Rupture stress',           pm(sR, sRu),   sUnit}
    {'Strain at rupture',        pm(eR, eRu),   eUnit}
    {'Yield stress (0.2%)',      pm(sY, sYu),   sUnit}
    {'Strain at yield',          pm(eY, eYu),   eUnit}
    {'Young''s modulus E',       pm(E, EUnc),   sUnit}
    {'Modulus of toughness',     pm(UT, UTu),   uUnit}
    {'Modulus of resilience',    pm(Ur, Uru),   uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Quantity', 'Value', 'Units'}));

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: specimen row whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(matName);
    best = 0;
    bestLen = 0;
    for r = 1:size(specimens, 1)
        for key = strsplit(norm(specimens{r, 1}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
```

```
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
```

```
    end
    p = floor(log10(dx)) - 1;              % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

If by "chart" you meant the plot rather than the table, tell me which plot items to drop. Right now it shows the curve, uncertainty bars, the 0.2% offset line, and the yield, ultimate and rupture points.

You 2026-10-08 20:11 UTC

okay now i need to edit the code so that way for steel i have : (both MTS displacement measurements and extensometer readings. One plot containing
i)
ii)
Engineering Stress/Strain curve for both readings
0.2% Offset lines for both readings
iii) Error bars for both readings
iv) Yield Stress and Strain for both readings
b) Organized table(s) containing
i)
Young’s Modulus w/ uncertainties for both readings
ii)
Yield Stress and Strain for both readings
iii) Modulus of Resilience w/ uncertainties for both readings
iv) Percent Difference between the two (treat extensometer as more accurate)
(1) Yield Stress and Strain
(2) Young’s Modulus

Claude 2026-10-08 20:12 UTC

I'll add a second figure that appears automatically when a file has extensometer data, which among your samples is only steel. It shows the extensometer and MTS displacement readings side by side, each analyzed separately. The main figure stays as it is. First I'll move the per-curve calculations into one function so both readings go through exactly the same steps.

— request interrupted —

You 2026-10-08 20:13 UTC

i need one plot containing engineering stress/strain for BOTH readings

Claude 2026-10-08 20:13 UTC

Understood: one plot with both curves on it, no inset or second axes. Each reading gets its own 0.2% offset line, uncertainty bars and yield point, and the comparison table goes beside it. Your existing figure stays the same.

Claude 2026-10-08 20:14 UTC

My Octave harness splits the file at the wrong spot now that the new helper functions come first. That's a test-harness problem, not a script bug, so I'm fixing the harness.

Claude 2026-10-08 20:14 UTC

The refactor didn't change any main-figure values, and only steel gets the comparison figure. Here's the plot.

Claude 2026-10-08 20:14 UTC

Two things to fix: the two yield labels overlap, and the stress values in the labels print as `6.035e+04`. That second issue also affects the labels on the main plot.

Claude 2026-10-08 20:15 UTC

The labels are readable now. The thin line across the orange curve and the missing table are Octave-only quirks. Committing.

Claude 2026-10-08 20:17 UTC

When you run steel, a second figure window now opens with **one plot showing both readings**, the extensometer in blue and MTS displacement in orange. Each reading has its own:

- engineering stress-strain curve
- 0.2% offset line
- uncertainty bars
- yield point, with its value written next to it

A comparison table sits beside the plot. I ran it in Octave on your steel file and the plot is above. Octave can't draw the table, so I checked the table values in its text output; the table hasn't been seen on screen in MATLAB yet. Aluminum and carbon fiber files have no extensometer data, so they don't get this figure, and their main figure is unchanged.

Your steel results:

Quantity	Extensometer	MTS displacement	% difference
Young's modulus E (psi)	(3.07 ± 0.46)e+07	(1.25 ± 0.47)e+07	59.1%
Yield stress, 0.2% (psi)	59919 ± 643	60353 ± 647	0.7%
Yield strain (in/in)	0.003954 ± 0.000050	0.00682 ± 0.00036	72.4%
Modulus of resilience (in-lbf/in³)	58.5 ± 8.9	145 ± 54	(not requested)

Percent difference is |MTS − extensometer| ÷ extensometer × 100, taking the extensometer as the accurate value, as you asked.

The yield stress agrees within 0.7% because both readings share the same load data. The modulus and yield strain differ a lot because crosshead displacement also picks up stretch in the machine and grips, so the MTS strain reads high and its slope comes out low. That's the expected result and worth saying in your report.

On the plot:

- The blue extensometer curve stops at about 2.1% strain, where the extensometer was removed.
- The orange MTS curve runs to the end of the test. It goes further right (about 0.19) than the stitched curve in the main figure, because it is raw crosshead strain.
- When `saveFigure = true`, this figure is also saved as `<folder>_stress_strain_ext_vs_MTS.png` and `.fig`.
- To turn the comparison figure off, set `compareStrain = false`.

Your original steel figure, with ultimate, rupture and toughness, still appears as before and its numbers didn't change.

The full code is below and is pushed to the branch:

```
% Stress-strain curve from an MTS .DAT file, with a 0.2% offset line,
% uncertainty bars, and the yield, ultimate and rupture points, plus a table
% of the lab measurements and results (with uncertainties) in the same
% figure window.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
```

```
% Set "material" below to the material folder name; the script finds the
% folder, finds the .dat file inside it, reads it, and makes the figure.
% Enter each specimen's measurements once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve     dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)            dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
clear; clc; close all;

%% ====================== USER SETTINGS =====================
material = 'Aluminum';         % <-- material folder inside M003 (not case sensitive; a partial name works)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 2.0,    NaN,    NaN,    NaN,    NaN,    NaN      % <-- placeholder, enter measurements
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along the curve that get uncertainty bars
compareStrain = true;          % files with extensometer data (steel) also get a figure comparing
                               % the extensometer and MTS displacement readings
saveFigure = false;            % true = save .png and .fig into the material folder
%% ==========================================================

%% Locate the folder and the .dat file
baseFolder = resolveBaseFolder(baseFolder);
testPath   = findSubfolder(baseFolder, testFolder);
matPath    = findSubfolder(testPath, material);
[~, matName] = fileparts(matPath);
datFile    = findDatFile(matPath, datFileName);
fprintf('Material folder: %s\nDAT file:        %s\n', matPath, datFile);

%% Specimen geometry
[shape, L0, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName);
[A, relA] = sectionArea(shape, Wi, ti, dimUnc);

haveArea = isfinite(A) && A > 0;
if ~haveArea
    relA = 0;
    fprintf('No specimen dimensions for "%s" - plotting axial force instead of stress.\n', matName);
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', matName);
end

%% Read the data
[data, names, units] = readMtsDat(datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};
```

```
good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', datFile); end

F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

%% Strain
e = eD;
eExt = [];                          % extensometer strain, in/in (empty if none)
isExt = false(size(e));             % which points come from the extensometer
if ~strcmpi(strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    eExt = eX;
    [~, iRem] = max(eX);                            % last reading before the extensometer came off
    if strcmpi(strainSource, 'strain')
        e = eX;
        isExt(:) = true;
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3       % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            fprintf('Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        end
    else
        fprintf(['Axial Strain column is flat - extensometer was not recording.\n' ...
            'Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
eUnit = 'in/in';
eUnc = sqrt((dispUnc/L0)^2 + (e*gaugeUnc/L0).^2);
eUnc(isExt) = extUnc;

%% Stress
if haveArea
    s = F / A;
    switch lower(fUnit)
        case 'lbf', sUnit = 'psi';   uUnit = 'in-lbf/in^3';
        case 'kip', sUnit = 'ksi';   uUnit = 'in-kip/in^3';
        case 'n',   sUnit = 'MPa';   uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  sUnit = [fUnit '/area']; uUnit = sUnit;
    end
    sLabel = sprintf('Engineering Stress (%s)', sUnit);
else
    s = F;
    sUnit  = fUnit;
    uUnit  = [fUnit '*in/in'];
    sLabel = sprintf('Stress - Axial Force (%s)', sUnit);
end
relS = sqrt((loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;
sRaw = s;
sUncRaw = sUnc;

%% Break: biggest sudden load drop after the ultimate
% It can be one sample or spread over a few, and the load cell may not read
% 0 afterwards (it can even go negative).
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));               % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > breakDrop*sU
            iR = iU + a - 1;        % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('No break found in the data; rupture is the last data point.\n');
end

%% Rupture = start of the final load drop
% Ductile samples can tear for a while before the last snap; walk back from
% the snap while the curve is still falling steeply (> 5% of E).
E0 = fitElastic(e(1:iR), s(1:iR), fitRange);
if E0 > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
        j = j - 1;
    end
    if j < iR
        lo = max(iU, j - w);
        [~, k] = max(s(lo:j));
        iR = lo + k - 1;
    end
end
iR0 = iR;                           % rupture index in the raw data

%% Young's modulus, toe compensation and 0.2% offset yield
P = struct('fitRange', fitRange, 'offset', offset, 'toe', toeCompensation, 'relS', relS, ...
    'extUnc', extUnc, 'dispUnc', dispUnc, 'gaugeUnc', gaugeUnc, 'L0', L0);
```

```
r = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
e    = r.e;     s    = r.s;
eUnc = r.eUnc;  sUnc = r.sUnc;
E  = r.E;   EUnc = r.EUnc;  b = r.b;
sY = r.sY;  eY = r.eY;  sYu = r.sYu;  eYu = r.eYu;
if ~(E > 0)
    warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', E);
elseif isnan(sY)
    fprintf('The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
end

%% Ultimate and rupture
[sU, iU] = max(s);
eU  = e(iU);    sUu = sUnc(iU);    eUu = eUnc(iU);
iR  = numel(s);
sR  = s(iR);    sRu = sUnc(iR);    eR  = e(iR);    eRu = eUnc(iR);

%% Modulus of toughness (area under the curve) and resilience (sY^2 / 2E)
UT  = trapz(e, s);
UTu = abs(UT) * sqrt(relS^2 + (eRu/eR)^2);
Ur  = r.Ur;
Uru = r.Uru;

%% Figure: plot on the left, table on the right
fig = figure('Name', sprintf('%s stress-strain', matName), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
hErr = drawUncertainty(ax, e, s, eUnc, sUnc, nErrorBars, [0.5 0.5 0.5]);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    [eLine, sLine] = offsetLine(r, offset, sU);
    hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                             % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', eUnit));
ylabel(ax, sLabel);
title(ax, sprintf('%s - Stress vs. Strain', matName), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

%% Results table
if strcmpi(shape, 'round')
    dimRows = {'Initial diameter', pm(Wi, dimUnc), 'in'
               'Final diameter',   pm(Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(Wi, dimUnc), 'in'
               'Initial thickness', pm(ti, dimUnc), 'in'
               'Final width',       pm(Wf, dimUnc), 'in'
               'Final thickness',   pm(tf, dimUnc), 'in'};
end

rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(sU, sUu),   sUnit}
    {'Strain at ultimate',       pm(eU, eUu),   eUnit}
    {'Rupture stress',           pm(sR, sRu),   sUnit}
    {'Strain at rupture',        pm(eR, eRu),   eUnit}
    {'Yield stress (0.2%)',      pm(sY, sYu),   sUnit}
    {'Strain at yield',          pm(eY, eYu),   eUnit}
    {'Young''s modulus E',       pm(E, EUnc),   sUnit}
    {'Modulus of toughness',     pm(UT, UTu),   uUnit}
    {'Modulus of resilience',    pm(Ur, Uru),   uUnit}
    ];
```

```
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);

disp(cell2table(rows, 'VariableNames', {'Quantity', 'Value', 'Units'}));

%% Extensometer vs. MTS displacement (files with extensometer data, e.g. steel)
% Both readings analyzed separately on one plot. The extensometer curve
% stops where the extensometer was removed.
fig2 = [];
if compareStrain && ~isempty(eExt) && any(isExt)
    nX = min(find(isExt, 1, 'last'), iR0);
    rX = analyzeCurve(eExt(1:nX), sRaw(1:nX), extUnc*ones(nX, 1), sUncRaw(1:nX), true(nX, 1), P);
    eDu = sqrt((dispUnc/L0)^2 + (eD*gaugeUnc/L0).^2);
    rD = analyzeCurve(eD(1:iR0), sRaw(1:iR0), eDu(1:iR0), sUncRaw(1:iR0), false(iR0, 1), P);

    fig2 = figure('Name', sprintf('%s extensometer vs MTS', matName), 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [90 90 1500 700]);
    ax2 = axes(fig2, 'Position', [0.05 0.10 0.50 0.82]);
    hold(ax2, 'on');
    cX = [0 0.3 0.6];               % extensometer
    cD = [0.85 0.33 0.1];           % MTS displacement
    readings = {rX, cX, 'Extensometer', 'ext.'; rD, cD, 'MTS displacement', 'MTS'};
    h2 = gobjects(0);
    leg2 = {};
    for q = 1:2
        [rq, cq, nameq, shortq] = readings{q, :};
        h2(end+1) = plot(ax2, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg2{end+1} = nameq;
        h2(end+1) = drawUncertainty(ax2, rq.e, rq.s, rq.eUnc, rq.sUnc, nErrorBars, 0.5*cq + 0.5);
        leg2{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, offset, max(rq.s));
            h2(end+1) = plot(ax2, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg2{end+1} = sprintf('%g%% offset line (%s)', offset*100, shortq);
        end
        if isfinite(rq.sY)
            h2(end+1) = plot(ax2, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg2{end+1} = sprintf('Yield (%s)', shortq);
            % labels stacked below the yield points so the two don't overlap
            yLab = rq.sY - (0.04 + 0.08*(q - 1))*max([rX.s; rD.s]);
            text(ax2, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq);
        end
    end
    hold(ax2, 'off');
    grid(ax2, 'on');
    box(ax2, 'on');
    xlabel(ax2, sprintf('Axial Strain (%s)', eUnit));
    ylabel(ax2, sLabel);
    title(ax2, sprintf('%s - Extensometer vs. MTS Displacement', matName), 'Interpreter', 'none');
    xlim(ax2, [0, 1.05*max([rX.e; rD.e])]);
    ylim(ax2, [0, 1.12*max([rX.s; rD.s])]);
    legend(ax2, h2, leg2, 'Location', 'southeast');

    % percent difference, extensometer taken as the accurate value
    pd = @(mts, ext) sprintf('%.1f%%', abs(mts - ext)/abs(ext)*100);
    rows2 = {
        ['Young''s modulus E (' sUnit ')'],         pm(rX.E,  rX.EUnc), pm(rD.E,  rD.EUnc), pd(rD.E,  rX.E)
        ['Yield stress, 0.2% (' sUnit ')'],         pm(rX.sY, rX.sYu),  pm(rD.sY, rD.sYu),  pd(rD.sY, rX.sY)
        ['Yield strain (' eUnit ')'],               pm(rX.eY, rX.eYu),  pm(rD.eY, rD.eYu),  pd(rD.eY, rX.eY)
        ['Modulus of resilience (' uUnit ')'],      pm(rX.Ur, rX.Uru),  pm(rD.Ur, rD.Uru),  ''
        };
    uitable(fig2, 'Data', rows2, ...
        'ColumnName', {'Quantity', 'Extensometer', 'MTS displacement', '% difference'}, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.57 0.40 0.42 0.30], ...
        'ColumnWidth', {215, 165, 165, 95}, 'FontSize', 11);
    disp(cell2table(rows2, 'VariableNames', {'Quantity', 'Extensometer', 'MTS', 'PercentDiff'}));
end

if saveFigure
    outBase = fullfile(matPath, [matName '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
    if ~isempty(fig2)
        saveas(fig2, [outBase '_ext_vs_MTS.png']);
        savefig(fig2, [outBase '_ext_vs_MTS.fig']);
        fprintf('Saved %s_ext_vs_MTS.png and .fig\n', outBase);
    end
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
```

```
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: specimen row whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(matName);
    best = 0;
    bestLen = 0;
    for r = 1:size(specimens, 1)
        for key = strsplit(norm(specimens{r, 1}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
```

```
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;              % place of the 2nd significant figure
```

```
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-08 20:31 UTC

okay now alter this code so it plots the same thing but also plots aluminum. and now the table should ALSO include ultimate stress and strain of both materials w/ uncertainties. rupture stress and strain of both materials, true rupture stress and strain of both materials, modulus of toughness of toughness of both materials w/uncertainties, and percent difference from published values for ultimate stress, rupture stress, yield stress, and youngs modulus

Claude 2026-10-08 20:32 UTC

I'll rebuild the comparison figure so one plot shows steel (extensometer and MTS) and aluminum (MTS). To do that, the per-material processing (find folder, read file, strain, stress, rupture, analysis) needs to become one function the script can call for each material. Here's the table layout I'm going with:

- **Table 1, by reading:** Young's modulus, yield stress and strain, and resilience for steel extensometer, steel MTS, and aluminum MTS, plus the steel extensometer-vs-MTS % difference.
- **Table 2, steel vs aluminum:** ultimate, rupture and true rupture stress and strain, toughness, yield and E, with published values and % difference.

For published values I need the exact alloys. I'll put standard placeholder values in (6061-T6 aluminum and AISI 1018 steel) and flag them for you to confirm.

— request interrupted —

You 2026-10-08 20:33 UTC

continue

Claude 2026-10-08 20:37 UTC

The numbers work out. Main-figure steel values are unchanged, and both new tables fill in. One fix: published values print as `6.38e+04`; I'll change them to plain numbers.

Claude 2026-10-08 20:37 UTC

The Aluminum yield label sits far below its point, because each label is stepped down by its position in the list. I'll only step a label down when it would actually overlap one already placed.

Claude 2026-10-08 20:37 UTC

The labels are placed correctly now. The Steel MTS label crosses the aluminum curve, so I'll give the labels a white background to keep them readable.

Claude 2026-10-08 20:40 UTC

Figure 2 now shows steel (extensometer and MTS) and aluminum (MTS) on one plot. Each reading has its own curve, 0.2% offset line, uncertainty bars and yield point, and two tables sit beside it (plot preview above). Figure 1 still shows the single material you set in `material`, and its numbers are the same as before.

I ran it in Octave with steel, aluminum and carbon fiber 90 as the figure-1 material, with no errors. I checked the table values in the Command Window output. Octave can't draw tables on

the figure, so the on-screen table layout in MATLAB hasn't been seen yet.

Table 1 – each reading

Quantity	Steel – extensometer	Steel – MTS	% diff (MTS vs ext.)	Aluminum – MTS
Young's modulus E (psi)	(3.07 ± 0.46)e+07	(1.25 ± 0.47)e+07	59.1%	(4.9 ± 1.2)e+06
Yield stress, 0.2% (psi)	59919 ± 643	60353 ± 647	0.7%	36049 ± 390
Yield strain (in/in)	0.003954 ± 0.000050	0.00682 ± 0.00036	72.4%	0.00935 ± 0.00038
Modulus of resilience (in-lbf/in³)	58.5 ± 8.9	145 ± 54		132 ± 33

Table 2 – materials and published values

Quantity	Steel	Published	% diff	Aluminum	Published	% diff
Ultimate stress (psi)	82110 ± 881	63800	28.7%	47906 ± 518	45000	6.5%
Strain at ultimate	0.15732 ± 0.00036			0.10976 ± 0.00038		
Rupture stress (psi)	79485 ± 853	N/A		46896 ± 507	N/A	
Strain at rupture	0.18823 ± 0.00036			0.12113 ± 0.00038		
True rupture stress (psi)	(1.052 ± 0.011)e+05			58189 ± 640		
True rupture strain	0.2805 ± 0.0059			0.2158 ± 0.0061		
Yield stress (psi)	59919 ± 643	53700	11.6%	36049 ± 390	40000	9.9%
Young's modulus (psi)	(3.07 ± 0.46)e+07	2.97e+07	3.3%	(4.9 ± 1.2)e+06	1e+07	50.9%
Modulus of toughness (in-lbf/in³)	14416 ± 157			5213 ± 59		

How the values are worked out:

- **True rupture:** stress = load at rupture ÷ your measured final area; strain = ln(initial area ÷ final area).
- **Which strain is used:**
- Ultimate, rupture and toughness come from the MTS curve, because the extensometer comes off at 2.1% strain, before the ultimate.
- Steel's yield and E come from the extensometer, since you said to treat it as more accurate.
- **Steel's numbers differ from figure 1.** Figure 1 uses the extensometer-then-displacement curve, so its steel ultimate strain is 0.136 and toughness is 12,515. Table 2 uses MTS-only strain, giving 0.157 and 14,416. A note under the tables in the figure explains this.

I need from you:

1. **The alloys / published values from your handout.** The `published` table at the top currently has placeholders: AISI 1018 cold-drawn steel and 6061-T6 aluminum. Your steel (60 ksi yield, 82 ksi ultimate) doesn't look like 1018, which is why it shows 28.7% off.
2. **Published rupture stress.** Handbooks usually don't list one, so it shows N/A. Enter the value if your handout gives it.

Edit the `published` table and every % diff updates. With `saveFigure = true`, figure 2 is saved as `comparison_stress_strain.png` and `.fig` in the M003 folder.

Full code (pushed to the branch):

```
% Stress-strain curves from MTS .DAT files.
%
% Figure 1: one material (set "material" below) - stress-strain curve with a
%   0.2% offset line, uncertainty bars, yield, ultimate and rupture points,
```

```
%   and a table of the lab measurements and results with uncertainties.
% Figure 2: the materials in "compareMaterials" on one plot - every reading
%   (extensometer and MTS displacement where both exist) with its own 0.2%
%   offset line, uncertainty bars and yield point, plus tables comparing the
%   readings, the materials, and published values.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each material folder by name, finds the .dat file inside
% it, reads it, and makes the figures. Enter each specimen's measurements
% once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af     dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   percent difference   |measured - reference| / reference * 100
clear; clc; close all;

%% ===================== USER SETTINGS =====================
material = 'Steel';            % <-- material for figure 1 (folder inside M003; not case sensitive, partial name works)
compareMaterials = {'Steel', 'Aluminum'};   % <-- materials for figure 2 ({} = no figure 2)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 2.0,    NaN,    NaN,    NaN,    NaN,    NaN      % <-- placeholder, enter measurements
    };

% ---- Published values for the percent differences [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
% <-- replace with the alloys / values from your lab handout
published = {
%   keys       name                        yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    NaN,     29.7e6
    'alum',    '6061-T6',                  40000,   45000,    NaN,     10.0e6
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig (figure 1 in the material folder,
                               % figure 2 in the M003 folder)
%% ============================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% ================= FIGURE 1: one material =================
```

```
M = processMaterial(material, S);
r = M.r;
p = M.p;
e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
E = r.E;  sY = r.sY;  eY = r.eY;
iU = p.iU;  iR = numel(s);
sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
hErr = drawUncertainty(ax, e, s, eUnc, sUnc, nErrorBars, [0.5 0.5 0.5]);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    [eLine, sLine] = offsetLine(r, offset, sU);
    hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                          % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
ylabel(ax, M.sLabel);
title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

% Results table
if strcmpi(M.shape, 'round')
    dimRows = {'Initial diameter', pm(M.Wi, dimUnc), 'in'
               'Final diameter',   pm(M.Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(M.Wi, dimUnc), 'in'
               'Initial thickness', pm(M.ti, dimUnc), 'in'
               'Final width',       pm(M.Wf, dimUnc), 'in'
               'Final thickness',   pm(M.tf, dimUnc), 'in'};
end
rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(M.L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(M.Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
    {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
    {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
    {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
    {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
    {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
    {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
    {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
    {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);

%% ============ FIGURE 2: materials compared on one plot ============
fig2 = [];
if ~isempty(compareMaterials)
    Ms = cellfun(@(m) processMaterial(m, S), compareMaterials, 'UniformOutput', false);
    sUnit2 = Ms{1}.sUnit;
    uUnit2 = Ms{1}.uUnit;

    % every reading: extensometer (if the file has it) and MTS displacement
    R = cell(0, 3);                  % {analyzed curve, legend name, short label}
    for m = 1:numel(Ms)
        if ~isempty(Ms{m}.rX)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.']};
```

```
    end
    R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS']};
end
colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

fig2 = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [40 40 1650 820]);
ax2 = axes(fig2, 'Position', [0.04 0.08 0.43 0.85]);
hold(ax2, 'on');
sMax = max(cellfun(@(c) max(c.s), R(:, 1)));
eMax = max(cellfun(@(c) max(c.e), R(:, 1)));
h2 = gobjects(0);
leg2 = {};
labY = [];                      % heights of the yield labels placed so far
for q = 1:size(R, 1)
    [rq, nameq, shortq] = R{q, :};
    cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
    h2(end+1) = plot(ax2, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
    leg2{end+1} = nameq;
    h2(end+1) = drawUncertainty(ax2, rq.e, rq.s, rq.eUnc, rq.sUnc, nErrorBars, 0.5*cq + 0.5);
    leg2{end+1} = sprintf('Uncertainty (%s)', shortq);
    if rq.E > 0
        [eLine, sLine] = offsetLine(rq, offset, max(rq.s));
        h2(end+1) = plot(ax2, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
        leg2{end+1} = sprintf('%g%% offset line (%s)', offset*100, shortq);
    end
    if isfinite(rq.sY)
        h2(end+1) = plot(ax2, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
            'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
        leg2{end+1} = sprintf('Yield (%s)', shortq);
        % label just below the yield point, stepped further down only
        % if it would overlap a label already placed
        yLab = rq.sY - 0.04*sMax;
        while any(abs(labY - yLab) < 0.06*sMax)
            yLab = yLab - 0.06*sMax;
        end
        labY(end+1) = yLab;
        text(ax2, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
            'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
            'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
    end
end
hold(ax2, 'off');
grid(ax2, 'on');
box(ax2, 'on');
xlabel(ax2, 'Axial Strain (in/in)');
ylabel(ax2, Ms{1}.sLabel);
title(ax2, ['Stress vs. Strain - ' strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
    'Interpreter', 'none');
xlim(ax2, [0, 1.05*eMax]);
ylim(ax2, [0, 1.12*sMax]);
legend(ax2, h2, leg2, 'Location', 'southeast', 'Interpreter', 'none');

pdTxt = @(x, ref) pctDiff(x, ref);

% ---- Table 1: each reading ----
cols1 = {'Quantity'};
T1 = {['Young''s modulus E (' sUnit2 ')']; ['Yield stress, 0.2% (' sUnit2 ')']; ...
      'Yield strain (in/in)'; ['Modulus of resilience (' uUnit2 ')']};
for m = 1:numel(Ms)
    rD = Ms{m}.rD;
    colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
    if ~isempty(Ms{m}.rX)
        rX = Ms{m}.rX;
        colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
        colP = {pdTxt(rD.E, rX.E); pdTxt(rD.sY, rX.sY); pdTxt(rD.eY, rX.eY); ''};
        cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
        T1 = [T1, colX, colD, colP];
    else
        cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
        T1 = [T1, colD];
    end
end

% ---- Table 2: materials and published values ----
% ultimate, rupture and toughness from the MTS displacement curve (the
% extensometer comes off before the ultimate); yield and E from the
% extensometer when the file has it, MTS displacement otherwise
labels2 = {['Ultimate stress (' sUnit2 ')']; 'Strain at ultimate (in/in)'; ...
           ['Rupture stress (' sUnit2 ')']; 'Strain at rupture (in/in)'; ...
           ['True rupture stress (' sUnit2 ')']; 'True rupture strain (in/in)'; ...
           ['Yield stress, 0.2% (' sUnit2 ')']; ['Young''s modulus E (' sUnit2 ')']; ...
           ['Modulus of toughness (' uUnit2 ')']};
pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
cols2 = {'Quantity'};
T2 = labels2;
pubNames = {};
for m = 1:numel(Ms)
    Mm = Ms{m};
    pD = Mm.pD;
    if ~isempty(Mm.rX), best = Mm.rX; else, best = Mm.rD; end
    vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
            Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
    [pub, pubName] = lookupPublished(published, Mm.name);
    if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
    colV = cell(numel(labels2), 1);
    colPub = repmat({''}, numel(labels2), 1);
    colPd  = repmat({''}, numel(labels2), 1);
    for k = 1:numel(labels2)
        colV{k} = pm(vals(k, 1), vals(k, 2));
        if pubCol(k) > 0
            ref = pub(pubCol(k));
            if isfinite(ref)
                colPub{k} = sprintf('%g', ref);
                colPd{k}  = pdTxt(vals(k, 1), ref);
            else
                colPub{k} = 'N/A';
            end
```

```
            end
        end
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    note = {['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
             'the extensometer where the file has it.'], ...
            'True rupture: stress = rupture load / final area, strain = ln(initial area / final area).', ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

if saveFigure
    outBase = fullfile(M.path, [M.name '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
    if ~isempty(fig2)
        outBase2 = fullfile(fileparts(Ms{1}.path), 'comparison_stress_strain');
        saveas(fig2, [outBase2 '.png']);
        savefig(fig2, [outBase2 '.fig']);
        fprintf('Saved %s.png and .fig\n', outBase2);
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
```

```
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
```

```
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
```

```
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
```

```
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 22:15 UTC

okay the plastic dimensions were 3.0070 long, 0.5060 wide, and 0.1365 thick. now i need you to write me a code similar to the onle that plotted the MTS displacement measurements for steel but i need the plot to plot both steel and aluminum.

Claude 2026-10-14 22:15 UTC

I'll make two changes:

1. **Plastic:** add the measurements you gave. You sent only the initial ones, so the final length, width and thickness stay blank (N/A).
2. **Comparison plot:** add a setting, `includeExtensometer`. Set to `false`, it plots only the MTS displacement curves for steel and aluminum, each with its own offset line, uncertainty bars and yield point. Set to `true`, it brings back the version with steel's extensometer curve.

Claude 2026-10-14 22:15 UTC

Testing in Octave, with plastic as the figure-1 material so the new dimensions get checked too.

Claude 2026-10-14 22:18 UTC

Both changes are in, and I ran them in Octave. Octave can't draw tables, so I checked the table values in its text output; the table layout in MATLAB hasn't been seen yet.

- **Plastic:** the specimens table now has 3.0070 in, 0.5060 in and 0.1365 in. The plastic results are now in psi:
- ultimate 2900 ± 30 psi
- yield 2304 ± 23 psi
- rupture 2394 ± 24 psi
- E (1.99 ± 0.29)e+05 psi

Final length, width and thickness show N/A until you send them.

- **Steel and aluminum on one MTS displacement plot:** that is what figure 2 shows now, with the new setting `includeExtensometer = false` (plot above). Each material has its own curve, 0.2% offset line, uncertainty bars and yield point.
- **Table 1** lists E, yield stress and strain, and resilience for both.
- **Table 2** keeps ultimate, rupture, true rupture and toughness, plus the published-value % differences.
- With MTS-only strain, steel's modulus is 1.25e+07 psi, about 58% below the published 29.7e+06, because crosshead strain includes machine and grip stretch.
- Set `includeExtensometer = true` to get the previous version back, with steel's extensometer curve and the extensometer-vs-MTS columns.

The published values are still placeholders (AISI 1018 steel, 6061-T6 aluminum). Send the alloys or values from your handout and I'll put them in.

Full code (also pushed to the branch):

```
% Stress-strain curves from MTS .DAT files.
%
% Figure 1: one material (set "material" below) - stress-strain curve with a
%   0.2% offset line, uncertainty bars, yield, ultimate and rupture points,
%   and a table of the lab measurements and results with uncertainties.
% Figure 2: the materials in "compareMaterials" on one plot - the MTS
%   displacement reading of each (plus the extensometer reading when
%   includeExtensometer = true) with its own 0.2% offset line, uncertainty
%   bars and yield point, plus tables comparing the readings, the
%   materials, and published values.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each material folder by name, finds the .dat file inside
% it, reads it, and makes the figures. Enter each specimen's measurements
% once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af     dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   percent difference   |measured - reference| / reference * 100
clear; clc; close all;

%% ==================== USER SETTINGS ====================
material = 'Steel';            % <-- material for figure 1 (folder inside M003; not case sensitive, partial name works)
compareMaterials = {'Steel', 'Aluminum'};   % <-- materials for figure 2 ({} = no figure 2)
includeExtensometer = false;   % figure 2: false = MTS displacement curves only;
                               % true = also the extensometer curve where the file has one

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
```

```
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN      % <-- final measurements not entered
    };

% ---- Published values for the percent differences [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
% <-- replace with the alloys / values from your lab handout
published = {
%   keys       name                        yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    NaN,     29.7e6
    'alum',    '6061-T6',                  40000,   45000,    NaN,     10.0e6
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig (figure 1 in the material folder,
                               % figure 2 in the M003 folder)
%% ============================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% ================= FIGURE 1: one material =================
M = processMaterial(material, S);
r = M.r;
p = M.p;
e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
E = r.E;  sY = r.sY;  eY = r.eY;
iU = p.iU;  iR = numel(s);
sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
hErr = drawUncertainty(ax, e, s, eUnc, sUnc, nErrorBars, [0.5 0.5 0.5]);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    [eLine, sLine] = offsetLine(r, offset, sU);
    hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                        % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
ylabel(ax, M.sLabel);
title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
```

```
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

% Results table
if strcmpi(M.shape, 'round')
    dimRows = {'Initial diameter', pm(M.Wi, dimUnc), 'in'
               'Final diameter',   pm(M.Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(M.Wi, dimUnc), 'in'
               'Initial thickness', pm(M.ti, dimUnc), 'in'
               'Final width',       pm(M.Wf, dimUnc), 'in'
               'Final thickness',   pm(M.tf, dimUnc), 'in'};
end
rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(M.L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(M.Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
    {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
    {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
    {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
    {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
    {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
    {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
    {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
    {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);

%% ============ FIGURE 2: materials compared on one plot ============
fig2 = [];
if ~isempty(compareMaterials)
    Ms = cellfun(@(m) processMaterial(m, S), compareMaterials, 'UniformOutput', false);
    sUnit2 = Ms{1}.sUnit;
    uUnit2 = Ms{1}.uUnit;

    % readings to plot: MTS displacement, plus the extensometer if asked for
    % and the file has it
    useExt = cellfun(@(c) includeExtensometer && ~isempty(c.rX), Ms);
    R = cell(0, 3);                 % {analyzed curve, legend name, short label}
    for m = 1:numel(Ms)
        if useExt(m)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.']};
        end
        R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS']};
    end
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

    fig2 = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [40 40 1650 820]);
    ax2 = axes(fig2, 'Position', [0.04 0.08 0.43 0.85]);
    hold(ax2, 'on');
    sMax = max(cellfun(@(c) max(c.s), R(:, 1)));
    eMax = max(cellfun(@(c) max(c.e), R(:, 1)));
    h2 = gobjects(0);
    leg2 = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:size(R, 1)
        [rq, nameq, shortq] = R{q, :};
        cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
        h2(end+1) = plot(ax2, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg2{end+1} = nameq;
        h2(end+1) = drawUncertainty(ax2, rq.e, rq.s, rq.eUnc, rq.sUnc, nErrorBars, 0.5*cq + 0.5);
        leg2{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, offset, max(rq.s));
            h2(end+1) = plot(ax2, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg2{end+1} = sprintf('%g%% offset line (%s)', offset*100, shortq);
        end
        if isfinite(rq.sY)
            h2(end+1) = plot(ax2, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg2{end+1} = sprintf('Yield (%s)', shortq);
            % label just below the yield point, stepped further down only
            % if it would overlap a label already placed
            yLab = rq.sY - 0.04*sMax;
            while any(abs(labY - yLab) < 0.06*sMax)
                yLab = yLab - 0.06*sMax;
            end
            labY(end+1) = yLab;
            text(ax2, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
        end
    end
    hold(ax2, 'off');
    grid(ax2, 'on');
    box(ax2, 'on');
    xlabel(ax2, 'Axial Strain (in/in)');
    ylabel(ax2, Ms{1}.sLabel);
    if any(useExt), ttl = 'Stress vs. Strain - '; else, ttl = 'Stress vs. Strain (MTS displacement) - '; end
    title(ax2, [ttl strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax2, [0, 1.05*eMax]);
    ylim(ax2, [0, 1.12*sMax]);
```

```
    legend(ax2, h2, leg2, 'Location', 'southeast', 'Interpreter', 'none');

    pdTxt = @(x, ref) pctDiff(x, ref);

    % ---- Table 1: each reading ----
    cols1 = {'Quantity'};
    T1 = {['Young''s modulus E (' sUnit2 ')']; ['Yield stress, 0.2% (' sUnit2 ')']; ...
          'Yield strain (in/in)'; ['Modulus of resilience (' uUnit2 ')']};
    for m = 1:numel(Ms)
        rD = Ms{m}.rD;
        colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
        if useExt(m)
            rX = Ms{m}.rX;
            colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
            colP = {pdTxt(rD.E, rX.E); pdTxt(rD.sY, rX.sY); pdTxt(rD.eY, rX.eY); ''};
            cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
            T1 = [T1, colX, colD, colP];
        else
            cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
            T1 = [T1, colD];
        end
    end

    % ---- Table 2: materials and published values ----
    % ultimate, rupture and toughness from the MTS displacement curve (the
    % extensometer comes off before the ultimate); yield and E from the
    % extensometer when it is plotted, MTS displacement otherwise
    labels2 = {['Ultimate stress (' sUnit2 ')']; 'Strain at ultimate (in/in)'; ...
               ['Rupture stress (' sUnit2 ')']; 'Strain at rupture (in/in)'; ...
               ['True rupture stress (' sUnit2 ')']; 'True rupture strain (in/in)'; ...
               ['Yield stress, 0.2% (' sUnit2 ')']; ['Young''s modulus E (' sUnit2 ')']; ...
               ['Modulus of toughness (' uUnit2 ')']};
    pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
    cols2 = {'Quantity'};
    T2 = labels2;
    pubNames = {};
    for m = 1:numel(Ms)
        Mm = Ms{m};
        pD = Mm.pD;
        if useExt(m), best = Mm.rX; else, best = Mm.rD; end
        vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
                Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
        [pub, pubName] = lookupPublished(published, Mm.name);
        if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
        colV = cell(numel(labels2), 1);
        colPub = repmat({''}, numel(labels2), 1);
        colPd  = repmat({''}, numel(labels2), 1);
        for k = 1:numel(labels2)
            colV{k} = pm(vals(k, 1), vals(k, 2));
            if pubCol(k) > 0
                ref = pub(pubCol(k));
                if isfinite(ref)
                    colPub{k} = sprintf('%g', ref);
                    colPd{k}  = pdTxt(vals(k, 1), ref);
                else
                    colPub{k} = 'N/A';
                end
            end
        end
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    if any(useExt)
        strainNote = ['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
                      'the extensometer where it is plotted.'];
    else
        strainNote = 'All strains are from the MTS crosshead displacement.';
    end
    note = {strainNote, ...
            'True rupture: stress = rupture load / final area, strain = ln(initial area / final area).', ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

if saveFigure
    outBase = fullfile(M.path, [M.name '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
    if ~isempty(fig2)
        outBase2 = fullfile(fileparts(Ms{1}.path), 'comparison_stress_strain');
        saveas(fig2, [outBase2 '.png']);
        savefig(fig2, [outBase2 '.fig']);
        fprintf('Saved %s.png and .fig\n', outBase2);
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
```

```
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
```

```
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
```

```
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
```

```
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
```

```
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 22:22 UTC

okay perfect i just need the graph to also plot the trust stress/strain curves of both materials (error bars not needed) and i need the table to show the percent difference from published values for rupture stress

Claude 2026-10-14 22:22 UTC

By "trust" I'm taking you to mean **true** stress-strain. I'll add a true curve for each material using σ_T = σ(1+ε) and ε_T = ln(1+ε), drawn only up to the ultimate point. Past the ultimate the specimen necks and those formulas no longer hold. Each material also gets its true rupture point, calculated from your measured final area, so the end of the true curve is shown.

Claude 2026-10-14 22:23 UTC

Two fixes: the white label box covers the true-rupture diamond, and 105220 prints as `1.0522e+05`. I'll also move the axes slightly so the y-axis label isn't clipped.

Claude 2026-10-14 22:23 UTC

The plot looks right. Since the axes now carry both engineering and true values, I'll relabel them to say so.

Claude 2026-10-14 22:26 UTC

Figure 2 now draws a true stress-strain curve for steel and aluminum, without error bars. Each is a dash-dot line in the same color as that material's engineering curve, and it runs only to the ultimate point, because the conversion stops being valid once the specimen necks. Each material also gets a diamond at its true rupture point, worked out from the load at rupture and your measured final area:

- Steel: 0.2805 in/in, 105,223 psi
- Aluminum: 0.2158 in/in, 58,189 psi

The x-axis now reaches past 0.28 so the steel true rupture point fits, which squeezes the curves a little. I ran it in Octave on your steel and aluminum files and checked the plot (above). Octave can't draw the tables, so I checked their values in the printed output; the table layout in MATLAB hasn't been seen yet.

Rupture % difference: the table already computes this, but it shows N/A because there's no published rupture stress to compare against. I left those values blank rather than guess, since handbooks usually don't list one. Put the values from your handout in the `rupture` column of the `published` table (both rows are marked `% <-- enter published rupture stress`) and the % difference will fill in. If you send me the numbers, I'll add them. The yield, ultimate and modulus values in

that table are still the placeholder alloys (AISI 1018 and 6061-T6), so send those alloys too if they're different.

Full code (also pushed to the branch):

```
% Stress-strain curves from MTS .DAT files.
%
% Figure 1: one material (set "material" below) - stress-strain curve with a
%   0.2% offset line, uncertainty bars, yield, ultimate and rupture points,
%   and a table of the lab measurements and results with uncertainties.
% Figure 2: the materials in "compareMaterials" on one plot - the MTS
%   displacement reading of each (plus the extensometer reading when
%   includeExtensometer = true) with its own 0.2% offset line, uncertainty
%   bars and yield point, the true stress-strain curve of each material,
%   plus tables comparing the readings, the materials, and published values.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each material folder by name, finds the .dat file inside
% it, reads it, and makes the figures. Enter each specimen's measurements
% once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af     dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   true curve (to the ultimate, before necking)  sT = s*(1 + e),  eT = ln(1 + e)
%   percent difference   |measured - reference| / reference * 100
clear; clc; close all;

%% ===================== USER SETTINGS =====================
material = 'Steel';           % <-- material for figure 1 (folder inside M003; not case sensitive, partial name works)
compareMaterials = {'Steel', 'Aluminum'};   % <-- materials for figure 2 ({} = no figure 2)
includeExtensometer = false;   % figure 2: false = MTS displacement curves only;
                               % true = also the extensometer curve where the file has one

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN      % <-- final measurements not entered
    };

% ---- Published values for the percent differences [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
% <-- replace with the alloys / values from your lab handout
published = {
%   keys       name                        yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    NaN,     29.7e6     % <-- enter published rupture stress
    'alum',    '6061-T6',                  40000,   45000,    NaN,     10.0e6     % <-- enter published rupture stress
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
```

```
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig (figure 1 in the material folder,
                               % figure 2 in the M003 folder)
%% ============================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% ================ FIGURE 1: one material ================
M = processMaterial(material, S);
r = M.r;
p = M.p;
e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
E = r.E;  sY = r.sY;  eY = r.eY;
iU = p.iU;  iR = numel(s);
sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
hErr = drawUncertainty(ax, e, s, eUnc, sUnc, nErrorBars, [0.5 0.5 0.5]);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    [eLine, sLine] = offsetLine(r, offset, sU);
    hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                        % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
ylabel(ax, M.sLabel);
title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

% Results table
if strcmpi(M.shape, 'round')
    dimRows = {'Initial diameter', pm(M.Wi, dimUnc), 'in'
               'Final diameter',   pm(M.Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(M.Wi, dimUnc), 'in'
               'Initial thickness', pm(M.ti, dimUnc), 'in'
               'Final width',       pm(M.Wf, dimUnc), 'in'
               'Final thickness',   pm(M.tf, dimUnc), 'in'};
end
rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(M.L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(M.Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
    {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
    {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
    {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
    {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
    {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
    {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
    {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
    {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
```

```
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);

%% ============ FIGURE 2: materials compared on one plot ============
fig2 = [];
if ~isempty(compareMaterials)
    Ms = cellfun(@(m) processMaterial(m, S), compareMaterials, 'UniformOutput', false);
    sUnit2 = Ms{1}.sUnit;
    uUnit2 = Ms{1}.uUnit;

    % readings to plot: MTS displacement, plus the extensometer if asked for
    % and the file has it
    useExt = cellfun(@(c) includeExtensometer && ~isempty(c.rX), Ms);
    R = cell(0, 4);                 % {analyzed curve, legend name, short label, material (0 = extensometer)}
    for m = 1:numel(Ms)
        if useExt(m)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.'], 0};
        end
        R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS'], m};
    end

    % true stress-strain up to the ultimate (valid until necking starts)
    for m = 1:numel(Ms)
        iUm = Ms{m}.pD.iU;
        eEng = Ms{m}.rD.e(1:iUm);
        Ms{m}.eT = log(1 + eEng);
        Ms{m}.sT = Ms{m}.rD.s(1:iUm) .* (1 + eEng);
    end
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

    fig2 = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [40 40 1650 820]);
    ax2 = axes(fig2, 'Position', [0.055 0.08 0.415 0.85]);
    hold(ax2, 'on');
    sMax = max([cellfun(@(c) max(c.s), R(:, 1)); cellfun(@(c) max([c.sT; c.sTR]), Ms(:))]);
    eMax = max([cellfun(@(c) max(c.e), R(:, 1)); cellfun(@(c) max([c.eT; c.eTR]), Ms(:))]);
    h2 = gobjects(0);
    leg2 = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:size(R, 1)
        [rq, nameq, shortq, mq] = R{q, :};
        cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
        h2(end+1) = plot(ax2, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg2{end+1} = nameq;
        h2(end+1) = drawUncertainty(ax2, rq.e, rq.s, rq.eUnc, rq.sUnc, nErrorBars, 0.5*cq + 0.5);
        leg2{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, offset, max(rq.s));
            h2(end+1) = plot(ax2, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg2{end+1} = sprintf('%g%% offset line (%s)', offset*100, shortq);
        end
        if isfinite(rq.sY)
            h2(end+1) = plot(ax2, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg2{end+1} = sprintf('Yield (%s)', shortq);
            % label just below the yield point, stepped further down only
            % if it would overlap a label already placed
            yLab = rq.sY - 0.04*sMax;
            while any(abs(labY - yLab) < 0.06*sMax)
                yLab = yLab - 0.06*sMax;
            end
            labY(end+1) = yLab;
            text(ax2, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
        end
        if mq > 0                   % true curve and true rupture point, same color as the MTS curve
            Mm = Ms{mq};
            h2(end+1) = plot(ax2, Mm.eT, Mm.sT, '-.', 'LineWidth', 1.5, 'Color', cq);
            leg2{end+1} = sprintf('True stress-strain (%s)', Mm.name);
            if isfinite(Mm.sTR) && isfinite(Mm.eTR)
                h2(end+1) = plot(ax2, Mm.eTR, Mm.sTR, 'd', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                    'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
                leg2{end+1} = sprintf('True rupture (%s)', Mm.name);
                text(ax2, Mm.eTR - 0.012*eMax, Mm.sTR, sprintf('True rupture, %s (%.4g, %.0f)', Mm.name, Mm.eTR, Mm.sTR), ...
                    'VerticalAlignment', 'middle', 'HorizontalAlignment', 'right', 'Color', cq, ...
                    'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
            end
        end
    end
    hold(ax2, 'off');
    grid(ax2, 'on');
    box(ax2, 'on');
    xlabel(ax2, 'Engineering / True Strain (in/in)');
    ylabel(ax2, strrep(Ms{1}.sLabel, 'Engineering Stress', 'Engineering / True Stress'));
    if any(useExt), ttl = 'Stress vs. Strain - '; else, ttl = 'Stress vs. Strain (MTS displacement) - '; end
    title(ax2, [ttl strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax2, [0, 1.05*eMax]);
    ylim(ax2, [0, 1.12*sMax]);
    legend(ax2, h2, leg2, 'Location', 'southeast', 'Interpreter', 'none');

    pdTxt = @(x, ref) pctDiff(x, ref);

    % ---- Table 1: each reading ----
    cols1 = {'Quantity'};
    T1 = {['Young''s modulus E (' sUnit2 ')']; ['Yield stress, 0.2% (' sUnit2 ')']; ...
          'Yield strain (in/in)'; ['Modulus of resilience (' uUnit2 ')']};
    for m = 1:numel(Ms)
        rD = Ms{m}.rD;
        colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
        if useExt(m)
            rX = Ms{m}.rX;
            colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
            colP = {pdTxt(rD.E, rX.E); pdTxt(rD.sY, rX.sY); pdTxt(rD.eY, rX.eY); ''};
            cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
            T1 = [T1, colX, colD, colP];
        else
```

```
            cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
            T1 = [T1, colD];
        end
    end

    % ---- Table 2: materials and published values ----
    % ultimate, rupture and toughness from the MTS displacement curve (the
    % extensometer comes off before the ultimate); yield and E from the
    % extensometer when it is plotted, MTS displacement otherwise
    labels2 = {['Ultimate stress (' sUnit2 ')']; 'Strain at ultimate (in/in)'; ...
               ['Rupture stress (' sUnit2 ')']; 'Strain at rupture (in/in)'; ...
               ['True rupture stress (' sUnit2 ')']; 'True rupture strain (in/in)'; ...
               ['Yield stress, 0.2% (' sUnit2 ')']; ['Young''s modulus E (' sUnit2 ')']; ...
               ['Modulus of toughness (' uUnit2 ')']};
    pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
    cols2 = {'Quantity'};
    T2 = labels2;
    pubNames = {};
    for m = 1:numel(Ms)
        Mm = Ms{m};
        pD = Mm.pD;
        if useExt(m), best = Mm.rX; else, best = Mm.rD; end
        vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
                Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
        [pub, pubName] = lookupPublished(published, Mm.name);
        if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
        colV = cell(numel(labels2), 1);
        colPub = repmat({''}, numel(labels2), 1);
        colPd  = repmat({''}, numel(labels2), 1);
        for k = 1:numel(labels2)
            colV{k} = pm(vals(k, 1), vals(k, 2));
            if pubCol(k) > 0
                ref = pub(pubCol(k));
                if isfinite(ref)
                    colPub{k} = sprintf('%g', ref);
                    colPd{k}  = pdTxt(vals(k, 1), ref);
                else
                    colPub{k} = 'N/A';
                end
            end
        end
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    if any(useExt)
        strainNote = ['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
                      'the extensometer where it is plotted.'];
    else
        strainNote = 'All strains are from the MTS crosshead displacement.';
    end
    note = {strainNote, ...
            ['True curve (dash-dot): stress x (1 + strain) and ln(1 + strain), up to the ultimate (valid until necking). ' ...
             'True rupture: rupture load / final area, ln(initial area / final area).'], ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

if saveFigure
    outBase = fullfile(M.path, [M.name '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
    if ~isempty(fig2)
        outBase2 = fullfile(fileparts(Ms{1}.path), 'comparison_stress_strain');
        saveas(fig2, [outBase2 '.png']);
        savefig(fig2, [outBase2 '.fig']);
        fprintf('Saved %s.png and .fig\n', outBase2);
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
```

```
if ~haveArea
    relA = 0;
    fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', M.name);
end

% read the data
[data, names, units] = readMtsDat(M.datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};
good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

% strain
e = eD;
eExt = [];                      % extensometer strain, in/in (empty if none)
isExt = false(size(e));         % which points come from the extensometer
if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(S.strainSource, 'strain')
        e = eX;
        eExt = eX;
        isExt(:) = true;
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        eExt = eX;
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        end
    else
        fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
            '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
M.eUnit = 'in/in';
eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
eUnc = eDu;
eUnc(isExt) = S.extUnc;

% stress
if haveArea
    s = F / M.A;
    switch lower(fUnit)
        case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
        case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
        case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
    end
    M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
else
    s = F;
    M.sUnit  = fUnit;
    M.uUnit  = [fUnit '*in/in'];
    M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
end
relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

% break: biggest sudden load drop after the ultimate (one sample or a
% few; the load cell may not read 0 afterwards, it can even go negative)
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > S.breakDrop*sU
            iR = iU + a - 1;    % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('  No break found in the data; rupture is the last data point.\n');
end

% rupture = start of the final load drop: ductile samples can tear for
% a while before the last snap, so walk back from the snap while the
% curve is still falling steeply (> 5% of E)
E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
if E0 > 0
    w = max(2, round(0.01*numel(s)));
    j = iR;
    while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
        j = j - 1;
    end
```

```
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
```

```
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
```

```
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
```

```
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 22:39 UTC

use published rupture stress for steel as 79770 psi and aluminum as 42060 psi

Claude 2026-10-14 22:39 UTC

I added both published rupture stresses and re-ran steel and aluminum in Octave. Table 2 now fills in the rupture stress row:

	Measured	Published	% diff
Steel rupture stress	79485 ± 853 psi	79770 psi	0.4%
Aluminum rupture stress	46896 ± 507 psi	42060 psi	11.5%

Only the two `published` lines changed. Paste these over the old ones at the top of your script:

```
published = {
%   keys       name                    yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    79770,   29.7e6
    'alum',    '6061-T6',                  40000,   45000,    42060,   10.0e6
    };
```

The rest of the code is the same as my last message, and the full updated file is pushed to the branch.

The yield, ultimate and Young's modulus values in that table are still the placeholder alloys (AISI 1018 cold drawn and 6061-T6). If your handout lists different values for those, send them and I'll update them the same way.

You 2026-10-14 22:41 UTC

just put in in the whole code and give me the code to copy and paste

Claude 2026-10-14 22:43 UTC

Here's the full script with the published rupture stresses filled in (79770 psi for steel, 42060 psi for aluminum). Copy all of it into `plotStressStrain.m`. It matches the version pushed to the branch.

```
% Stress-strain curves from MTS .DAT files.
%
% Figure 1: one material (set "material" below) - stress-strain curve with a
%   0.2% offset line, uncertainty bars, yield, ultimate and rupture points,
%   and a table of the lab measurements and results with uncertainties.
% Figure 2: the materials in "compareMaterials" on one plot - the MTS
%   displacement reading of each (plus the extensometer reading when
%   includeExtensometer = true) with its own 0.2% offset line, uncertainty
%   bars and yield point, the true stress-strain curve of each material,
%   plus tables comparing the readings, the materials, and published values.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each material folder by name, finds the .dat file inside
% it, reads it, and makes the figures. Enter each specimen's measurements
% once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
```

```
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af     dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   true curve (to the ultimate, before necking)  sT = s*(1 + e),  eT = ln(1 + e)
%   percent difference   |measured - reference| / reference * 100
clear; clc; close all;

%% ===================== USER SETTINGS =====================
material = 'Steel';            % <-- material for figure 1 (folder inside M003; not case sensitive, partial name works)
compareMaterials = {'Steel', 'Aluminum'};   % <-- materials for figure 2 ({} = no figure 2)
includeExtensometer = false;   % figure 2: false = MTS displacement curves only;
                               % true = also the extensometer curve where the file has one

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN      % <-- final measurements not entered
    };

% ---- Published values for the percent differences [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
% <-- replace with the alloys / values from your lab handout
published = {
%   keys       name                        yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    79770,   29.7e6
    'alum',    '6061-T6',                  40000,   45000,    42060,   10.0e6
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save .png and .fig (figure 1 in the material folder,
                               % figure 2 in the M003 folder)
%% ============================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% ================ FIGURE 1: one material =================
M = processMaterial(material, S);
r = M.r;
p = M.p;
e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
E = r.E;  sY = r.sY;  eY = r.eY;
iU = p.iU;  iR = numel(s);
sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [60 60 1500 700]);
ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
hold(ax, 'on');

hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

% Uncertainty bars on evenly spaced points along the curve
hErr = drawUncertainty(ax, e, s, eUnc, sUnc, nErrorBars, [0.5 0.5 0.5]);

% 0.2% offset line, from zero stress up to a bit past yield
hOff = gobjects(0);
if isfinite(E) && E > 0
    [eLine, sLine] = offsetLine(r, offset, sU);
    hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
end

% Key points
```

```
hY = gobjects(0);
if isfinite(sY)
    hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
    text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
end
hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
    'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
if iU == iR                             % brittle: breaks at the ultimate
    text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
else
    text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
    text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
end
hold(ax, 'off');

grid(ax, 'on');
box(ax, 'on');
xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
ylabel(ax, M.sLabel);
title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
eSpan = max(e) - min(0, min(e));
sSpan = sU - min(0, min(s));
if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

hLeg = [hCurve hErr hOff hY hU hR];
legTxt = {'Stress-strain curve', 'Uncertainty'};
if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', offset*100); end
if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
legTxt = [legTxt {'Ultimate', 'Rupture'}];
legend(ax, hLeg, legTxt, 'Location', 'best');

% Results table
if strcmpi(M.shape, 'round')
    dimRows = {'Initial diameter', pm(M.Wi, dimUnc), 'in'
               'Final diameter',   pm(M.Wf, dimUnc), 'in'};
else
    dimRows = {'Initial width',     pm(M.Wi, dimUnc), 'in'
               'Initial thickness', pm(M.ti, dimUnc), 'in'
               'Final width',       pm(M.Wf, dimUnc), 'in'
               'Final thickness',   pm(M.tf, dimUnc), 'in'};
end
rows = [
    {'LAB MEASUREMENTS', '', ''}
    {'Initial gauge length', pm(M.L0, gaugeUnc), 'in'}
    {'Final gauge length',   pm(M.Lf, gaugeUnc), 'in'}
    dimRows
    {'RESULTS', '', ''}
    {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
    {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
    {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
    {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
    {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
    {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
    {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
    {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
    {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
    ];
uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
    'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);

%% ============ FIGURE 2: materials compared on one plot ============
fig2 = [];
if ~isempty(compareMaterials)
    Ms = cellfun(@(m) processMaterial(m, S), compareMaterials, 'UniformOutput', false);
    sUnit2 = Ms{1}.sUnit;
    uUnit2 = Ms{1}.uUnit;

    % readings to plot: MTS displacement, plus the extensometer if asked for
    % and the file has it
    useExt = cellfun(@(c) includeExtensometer && ~isempty(c.rX), Ms);
    R = cell(0, 4);                 % {analyzed curve, legend name, short label, material (0 = extensometer)}
    for m = 1:numel(Ms)
        if useExt(m)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.'], 0};
        end
        R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS'], m};
    end

    % true stress-strain up to the ultimate (valid until necking starts)
    for m = 1:numel(Ms)
        iUm = Ms{m}.pD.iU;
        eEng = Ms{m}.rD.e(1:iUm);
        Ms{m}.eT = log(1 + eEng);
        Ms{m}.sT = Ms{m}.rD.s(1:iUm) .* (1 + eEng);
    end
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

    fig2 = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [40 40 1650 820]);
    ax2 = axes(fig2, 'Position', [0.055 0.08 0.415 0.85]);
    hold(ax2, 'on');
    sMax = max([cellfun(@(c) max(c.s), R(:, 1)); cellfun(@(c) max([c.sT; c.sTR]), Ms(:))]);
    eMax = max([cellfun(@(c) max(c.e), R(:, 1)); cellfun(@(c) max([c.eT; c.eTR]), Ms(:))]);
    h2 = gobjects(0);
    leg2 = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:size(R, 1)
        [rq, nameq, shortq, mq] = R{q, :};
```

```
        cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
        h2(end+1) = plot(ax2, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg2{end+1} = nameq;
        h2(end+1) = drawUncertainty(ax2, rq.e, rq.s, rq.eUnc, rq.sUnc, nErrorBars, 0.5*cq + 0.5);
        leg2{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, offset, max(rq.s));
            h2(end+1) = plot(ax2, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg2{end+1} = sprintf('%g%% offset line (%s)', offset*100, shortq);
        end
        if isfinite(rq.sY)
            h2(end+1) = plot(ax2, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg2{end+1} = sprintf('Yield (%s)', shortq);
            % label just below the yield point, stepped further down only
            % if it would overlap a label already placed
            yLab = rq.sY - 0.04*sMax;
            while any(abs(labY - yLab) < 0.06*sMax)
                yLab = yLab - 0.06*sMax;
            end
            labY(end+1) = yLab;
            text(ax2, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
        end
        if mq > 0                   % true curve and true rupture point, same color as the MTS curve
            Mm = Ms{mq};
            h2(end+1) = plot(ax2, Mm.eT, Mm.sT, '-.', 'LineWidth', 1.5, 'Color', cq);
            leg2{end+1} = sprintf('True stress-strain (%s)', Mm.name);
            if isfinite(Mm.sTR) && isfinite(Mm.eTR)
                h2(end+1) = plot(ax2, Mm.eTR, Mm.sTR, 'd', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                    'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
                leg2{end+1} = sprintf('True rupture (%s)', Mm.name);
                text(ax2, Mm.eTR - 0.012*eMax, Mm.sTR, sprintf('True rupture, %s (%.4g, %.0f)', Mm.name, Mm.eTR, Mm.sTR), ...
                    'VerticalAlignment', 'middle', 'HorizontalAlignment', 'right', 'Color', cq, ...
                    'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
            end
        end
    end
    hold(ax2, 'off');
    grid(ax2, 'on');
    box(ax2, 'on');
    xlabel(ax2, 'Engineering / True Strain (in/in)');
    ylabel(ax2, strrep(Ms{1}.sLabel, 'Engineering Stress', 'Engineering / True Stress'));
    if any(useExt), ttl = 'Stress vs. Strain - '; else, ttl = 'Stress vs. Strain (MTS displacement) - '; end
    title(ax2, [ttl strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax2, [0, 1.05*eMax]);
    ylim(ax2, [0, 1.12*sMax]);
    legend(ax2, h2, leg2, 'Location', 'southeast', 'Interpreter', 'none');

    pdTxt = @(x, ref) pctDiff(x, ref);

    % ---- Table 1: each reading ----
    cols1 = {'Quantity'};
    T1 = {['Young''s modulus E (' sUnit2 ')']; ['Yield stress, 0.2% (' sUnit2 ')']; ...
          'Yield strain (in/in)'; ['Modulus of resilience (' uUnit2 ')']};
    for m = 1:numel(Ms)
        rD = Ms{m}.rD;
        colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
        if useExt(m)
            rX = Ms{m}.rX;
            colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
            colP = {pdTxt(rD.E, rX.E); pdTxt(rD.sY, rX.sY); pdTxt(rD.eY, rX.eY); ''};
            cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
            T1 = [T1, colX, colD, colP];
        else
            cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
            T1 = [T1, colD];
        end
    end

    % ---- Table 2: materials and published values ----
    % ultimate, rupture and toughness from the MTS displacement curve (the
    % extensometer comes off before the ultimate); yield and E from the
    % extensometer when it is plotted, MTS displacement otherwise
    labels2 = {['Ultimate stress (' sUnit2 ')']; 'Strain at ultimate (in/in)'; ...
               ['Rupture stress (' sUnit2 ')']; 'Strain at rupture (in/in)'; ...
               ['True rupture stress (' sUnit2 ')']; 'True rupture strain (in/in)'; ...
               ['Yield stress, 0.2% (' sUnit2 ')']; ['Young''s modulus E (' sUnit2 ')']; ...
               ['Modulus of toughness (' uUnit2 ')']};
    pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
    cols2 = {'Quantity'};
    T2 = labels2;
    pubNames = {};
    for m = 1:numel(Ms)
        Mm = Ms{m};
        pD = Mm.pD;
        if useExt(m), best = Mm.rX; else, best = Mm.rD; end
        vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
                Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
        [pub, pubName] = lookupPublished(published, Mm.name);
        if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
        colV = cell(numel(labels2), 1);
        colPub = repmat({''}, numel(labels2), 1);
        colPd  = repmat({''}, numel(labels2), 1);
        for k = 1:numel(labels2)
            colV{k} = pm(vals(k, 1), vals(k, 2));
            if pubCol(k) > 0
                ref = pub(pubCol(k));
                if isfinite(ref)
                    colPub{k} = sprintf('%g', ref);
                    colPd{k}  = pdTxt(vals(k, 1), ref);
                else
                    colPub{k} = 'N/A';
                end
            end
        end
```

```
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig2, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    if any(useExt)
        strainNote = ['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
                      'the extensometer where it is plotted.'];
    else
        strainNote = 'All strains are from the MTS crosshead displacement.';
    end
    note = {strainNote, ...
            ['True curve (dash-dot): stress x (1 + strain) and ln(1 + strain), up to the ultimate (valid until necking). ' ...
             'True rupture: rupture load / final area, ln(initial area / final area).'], ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig2, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

if saveFigure
    outBase = fullfile(M.path, [M.name '_stress_strain']);
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
    if ~isempty(fig2)
        outBase2 = fullfile(fileparts(Ms{1}.path), 'comparison_stress_strain');
        saveas(fig2, [outBase2 '.png']);
        savefig(fig2, [outBase2 '.fig']);
        fprintf('Saved %s.png and .fig\n', outBase2);
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
```

```
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end
```

```
%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
```

```
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name (exact match first, then partial match)
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    hit = strcmpi(names, name);
    if ~any(hit)
        hit = contains(names, name, 'IgnoreCase', true);
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
```

```
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 22:47 UTC

now i need code for For both carbon fiber samples
a) One plot containing
i)
Engineering Stress/strain curve of both materials
ii)
Error bars of both materials
b) Organized table(s) containing
i)
Ultimate Stress and Strain of both materials w/ uncertainties
ii)
Rupture Stress and Strain of both materials
iii) Modulus of Toughness of both materials w/ uncertainties

Claude 2026-10-14 22:47 UTC

I'll add a **Figure 3** for the two carbon fiber samples. It plots both engineering stress-strain curves with uncertainty bars, and a table beside it lists ultimate stress and strain, rupture stress and strain, and modulus of toughness for each. You'll set the two folder names in a new `carbonMaterials` line at the top.

Each folder name there can list alternatives separated by `|`, and matching will ignore spaces and punctuation. That way names like `CF 90`, `cf90` or `Carbon Fiber 90` all work without you having to type the exact folder name.

— request interrupted —

You 2026-10-14 22:48 UTC

i want a seperate code for this plot. do not put it with the other one

Claude 2026-10-14 22:48 UTC

Now I'll write the standalone carbon fiber script, reusing the extracted helper functions.

— request interrupted —

You 2026-10-14 22:49 UTC

continue

Claude 2026-10-14 22:50 UTC

Now I'll test the new script in Octave on both carbon fiber files.

Claude 2026-10-14 22:51 UTC

The carbon fiber plot is now its own script, **`plotCarbonFiber.m`**. `plotStressStrain.m` is exactly as it was before; the changes I had started on it are undone. The new script carries its own copies of the helper functions, so it runs without the other file. I ran it in Octave on both carbon fiber files and the plot came out (preview above). Octave can't draw the table, so I checked its values in the printed output; the table layout in MATLAB hasn't been seen yet.

Quantity	CF 90	CF 45
Ultimate stress (psi)	79796 ± 864	33975 ± 372
Strain at ultimate (in/in)	0.03352 ± 0.00040	0.20529 ± 0.00035
Rupture stress (psi)	79796 ± 864	33844 ± 370
Strain at rupture (in/in)	0.03352 ± 0.00040	0.21280 ± 0.00035
Modulus of toughness (in-lbf/in³)	1545 ± 25	5263 ± 58

CF 90's rupture equals its ultimate because its data file ends right at the break. Strain for both samples comes from the MTS crosshead displacement.

Folder names: set them in `carbonMaterials` at the top. Each entry can list alternatives separated by `|`, and matching ignores case, spaces and punctuation, so `CF 90`, `cf90` or `Carbon Fiber 90` all match. If a name doesn't match, the error message lists the folders it found.

Save this as `plotCarbonFiber.m` (it's also pushed to the branch):

```
% Carbon fiber samples on one plot: engineering stress-strain curves with
% uncertainty bars, plus a table of ultimate stress and strain, rupture
% stress and strain, and modulus of toughness for each sample.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each folder named in "carbonMaterials" (case, spaces and
% punctuation are ignored, and '|' separates alternative names), finds the
% .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
%  - strain is crosshead displacement / gauge length (the extensometer
```

```
%    channel is used instead if it was recording)
%  - the slack "toe" at the start is removed so each curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
clear; clc; close all;

%% ===================== USER SETTINGS =====================
carbonMaterials = {'cf90|carbon fiber 90', 'cf45|carbon fiber 45|cf40|carbon fiber 40'};
                            % <-- the two carbon fiber folders inside M003

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';           % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'
% i = initial (before the test), f = final (after fracture)
specimens = {
%   keys                                  shape   Li      Wi      ti      Lf      Wf      tf
    'cf90|carbonfiber90',                 'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40',  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;       % +/- in, width/thickness (calipers)
gaugeUnc   = 0.00025;       % +/- in, gauge length
loadUncPct = 1.0;           % +/- percent of reading, load cell
dispUnc    = 0.001;         % +/- in, crosshead displacement
extUnc     = 5e-5;          % +/- in/in, extensometer

toeCompensation = true;     % remove the slack at the start so each curve starts at 0
fitRange  = [];             % [] = automatic elastic fit (used for the toe removal), or
                            % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;           % a sudden load drop bigger than this fraction of ultimate
                            % is the break; nothing after the rupture is plotted

nErrorBars = 15;            % number of points along each curve that get uncertainty bars
saveFigure = false;         % true = save .png and .fig in the M003 folder
%% ========================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', 'auto', 'toe', toeCompensation, ...
    'offset', 0.002, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

Cs = cellfun(@(m) processMaterial(m, S), carbonMaterials, 'UniformOutput', false);
colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6];

%% Plot: engineering stress-strain curves with uncertainty bars
fig = figure('Name', 'Carbon fiber stress-strain', 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [70 70 1500 700]);
ax = axes(fig, 'Position', [0.06 0.10 0.50 0.82]);
hold(ax, 'on');
h = gobjects(0);
leg = {};
for m = 1:numel(Cs)
    r  = Cs{m}.r;
    cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
    h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
    leg{end+1} = Cs{m}.name;
    h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, nErrorBars, 0.5*cm + 0.5);
    leg{end+1} = sprintf('Uncertainty (%s)', Cs{m}.name);
end
hold(ax, 'off');
grid(ax, 'on');
box(ax, 'on');
xlabel(ax, 'Axial Strain (in/in)');
ylabel(ax, Cs{1}.sLabel);
title(ax, ['Stress vs. Strain - ' strjoin(cellfun(@(c) c.name, Cs, 'UniformOutput', false), ' vs. ')], ...
    'Interpreter', 'none');
xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.r.e), Cs))]);
ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.r.s), Cs))]);
legend(ax, h, leg, 'Location', 'southeast', 'Interpreter', 'none');

%% Table: ultimate, rupture and modulus of toughness of each sample
sUnit = Cs{1}.sUnit;
T = {['Ultimate stress (' sUnit ')']; 'Strain at ultimate (in/in)'; ...
     ['Rupture stress (' sUnit ')']; 'Strain at rupture (in/in)'; ...
     ['Modulus of toughness (' Cs{1}.uUnit ')']};
cols = {'Quantity'};
for m = 1:numel(Cs)
    p = Cs{m}.p;
    T = [T, {pm(p.sU, p.sUu); pm(p.eU, p.eUu); pm(p.sR, p.sRu); pm(p.eR, p.eRu); pm(p.UT, p.UTu)}];
    cols = [cols, {Cs{m}.name}];
end
uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.80 0.38 0.04], ...
    'String', 'Carbon fiber results', 'FontWeight', 'bold', 'FontSize', 11, ...
    'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
uitable(fig, 'Data', T, 'ColumnName', cols, 'RowName', [], 'Units', 'normalized', ...
    'Position', [0.60 0.52 0.38 0.28], 'FontSize', 11, ...
    'ColumnWidth', [{220}, repmat({170}, 1, numel(Cs))]);
uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.38 0.38 0.12], ...
    'String', {'Strain is from the MTS crosshead displacement.', ...
               'A sample that breaks at its highest load has rupture = ultimate.'}, ...
    'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
printTable('Carbon fiber results', cols, T);

if saveFigure
    outBase = fullfile(fileparts(Cs{1}.path), 'carbon_fiber_stress_strain');
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
```

```
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
```

```
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
```

```
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name
% Tries each '|'-separated alternative: exact match, then partial match,
% then partial match ignoring case, spaces and punctuation.
function folder = findSubfolder(parent, name)
    d = dir(parent);
```

```
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
    normNames = cellfun(norm, names, 'UniformOutput', false);
    hit = false(size(names));
    for alt = strtrim(strsplit(name, '|'))
        a = alt{1};
        if isempty(a), continue; end
        hit = strcmpi(names, a);
        if ~any(hit), hit = contains(names, a, 'IgnoreCase', true); end
        if ~any(hit), hit = contains(normNames, norm(a)); end
        if any(hit), break; end
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
```

```
        return;
    end
    p = floor(log10(dx)) - 1;              % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 22:57 UTC

okay now i need one more code. i need to plot all 5 materials. it needs to contain: For all the materials together
a) One plot containing
i)
Engineering Stress/strain curves
ii)
Error bars

Claude 2026-10-14 22:57 UTC

I'll make a third standalone script, `plotAllMaterials.m`, built the same way as the carbon fiber one. It plots all five materials on one plot: each engineering stress-strain curve in its own color, with uncertainty bars and nothing else. All five curves use MTS displacement strain so they're compared the same way. A setting lets you switch steel to its extensometer-based curve instead.

Claude 2026-10-14 22:58 UTC

Testing it in Octave on all five sample files.

Claude 2026-10-14 22:58 UTC

All five curves plot. The thin straight lines cutting across some curves are an Octave drawing quirk with error bars and won't appear in MATLAB. Committing.

Claude 2026-10-14 22:59 UTC

The new script is **`plotAllMaterials.m`**. It plots steel, aluminum, both carbon fiber samples and plastic on one figure, each as an engineering stress-strain curve with uncertainty bars and a legend. I ran it in Octave on all five sample files and the plot came out (above). It runs on its own; your other two scripts are unchanged.

- **Strain:** every curve uses MTS crosshead displacement, so all five are on the same footing. Set `useExtensometer = true` if you want steel's extensometer-based curve instead.
- **Plastic:** its curve sits near the bottom because it only reaches about 2,900 psi, against 80,000 psi for steel. That's the real data, not a bug.
- **Several plastic folders:** if your M003 folder has more than one, `'plastic'` matches all of them and the script stops with an error listing them. Put the full folder name of the one you want in `materials`.

Save this as `plotAllMaterials.m` (it's also pushed to the branch):

```
% All materials on one plot: engineering stress-strain curves with
% uncertainty bars.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% The script finds each folder named in "materials" (case, spaces and
% punctuation are ignored, and '|' separates alternative names), finds the
% .dat file inside it, reads it, and makes the figure.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
%  - strain is crosshead displacement / gauge length for every material (or
```

```
%    the extensometer where it was recording, if useExtensometer = true)
%  - the slack "toe" at the start is removed so each curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
clear; clc; close all;

%% ===================== USER SETTINGS =====================
materials = {'steel', 'alum', 'cf90|carbon fiber 90', 'cf45|carbon fiber 45|cf40|carbon fiber 40', 'plastic'};
                              % <-- the material folders inside M003 (if you have several
                              %     plastic folders, use the full name of the one to plot)
useExtensometer = false;      % false = MTS displacement strain for every material (same basis);
                              % true  = extensometer while it was on, where a file has it (steel)

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';             % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% i = initial (before the test), f = final (after fracture); NaN = not measured
specimens = {
%   keys                                  shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                              'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                               'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',                 'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40',  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                            'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;         % +/- in, width/thickness (calipers)
gaugeUnc   = 0.00025;         % +/- in, gauge length
loadUncPct = 1.0;             % +/- percent of reading, load cell
dispUnc    = 0.001;           % +/- in, crosshead displacement
extUnc     = 5e-5;            % +/- in/in, extensometer

toeCompensation = true;       % remove the slack at the start so each curve starts at 0
fitRange  = [];               % [] = automatic elastic fit (used for the toe removal), or
                              % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;             % a sudden load drop bigger than this fraction of ultimate
                              % is the break; nothing after the rupture is plotted

nErrorBars = 15;              % number of points along each curve that get uncertainty bars
saveFigure = false;           % true = save .png and .fig in the M003 folder
%% ========================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', 'auto', 'toe', toeCompensation, ...
    'offset', 0.002, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

Ms = cellfun(@(m) processMaterial(m, S), materials, 'UniformOutput', false);
colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

%% Plot: engineering stress-strain curves with uncertainty bars
fig = figure('Name', 'All materials stress-strain', 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [70 70 1200 750]);
ax = axes(fig);
hold(ax, 'on');
h = gobjects(0);
leg = {};
curves = cell(size(Ms));
for m = 1:numel(Ms)
    if useExtensometer, r = Ms{m}.r; else, r = Ms{m}.rD; end
    curves{m} = r;
    cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
    h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
    leg{end+1} = Ms{m}.name;
    h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, nErrorBars, 0.5*cm + 0.5);
    leg{end+1} = sprintf('Uncertainty (%s)', Ms{m}.name);
end
hold(ax, 'off');
grid(ax, 'on');
box(ax, 'on');
xlabel(ax, 'Axial Strain (in/in)');
ylabel(ax, Ms{1}.sLabel);
if useExtensometer, ttl = 'Stress vs. Strain - All Materials';
else, ttl = 'Stress vs. Strain - All Materials (MTS displacement)'; end
title(ax, ttl);
xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.e), curves))]);
ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.s), curves))]);
legend(ax, h, leg, 'Location', 'eastoutside', 'Interpreter', 'none');

if saveFigure
    outBase = fullfile(fileparts(Ms{1}.path), 'all_materials_stress_strain');
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
```

```
M.datFile = findDatFile(M.path, S.datFileName);
fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

% specimen geometry
[M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
L0 = M.L0;
[M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
[M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
haveArea = isfinite(M.A) && M.A > 0;
relA = M.relA;
if ~haveArea
    relA = 0;
    fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', M.name);
end

% read the data
[data, names, units] = readMtsDat(M.datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};
good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

% strain
e = eD;
eExt = [];                      % extensometer strain, in/in (empty if none)
isExt = false(size(e));         % which points come from the extensometer
if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(S.strainSource, 'strain')
        e = eX;
        eExt = eX;
        isExt(:) = true;
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        eExt = eX;
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        end
    else
        fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
            '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
M.eUnit = 'in/in';
eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
eUnc = eDu;
eUnc(isExt) = S.extUnc;

% stress
if haveArea
    s = F / M.A;
    switch lower(fUnit)
        case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
        case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
        case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
    end
    M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
else
    s = F;
    M.sUnit  = fUnit;
    M.uUnit  = [fUnit '*in/in'];
    M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
end
relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

% break: biggest sudden load drop after the ultimate (one sample or a
% few; the load cell may not read 0 afterwards, it can even go negative)
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > S.breakDrop*sU
            iR = iU + a - 1;    % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('  No break found in the data; rupture is the last data point.\n');
end
```

```
    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
```

```
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name
% Tries each '|'-separated alternative: exact match, then partial match,
% then partial match ignoring case, spaces and punctuation.
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
    normNames = cellfun(norm, names, 'UniformOutput', false);
    hit = false(size(names));
    for alt = strtrim(strsplit(name, '|'))
        a = alt{1};
        if isempty(a), continue; end
        hit = strcmpi(names, a);
        if ~any(hit), hit = contains(names, a, 'IgnoreCase', true); end
        if ~any(hit), hit = contains(normNames, norm(a)); end
        if any(hit), break; end
    end
```

```
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end
```

You 2026-10-14 23:06 UTC

can you make a seperate code for a plot that plots all the plastic values, there are 12

Claude 2026-10-14 23:06 UTC

I'll make one more standalone script, **`plotPlastics.m`**. It finds every plastic sample by itself. It looks for every folder in M003 whose name contains "plastic", and every .dat file inside those folders is treated as one sample. That works whether you have 12 separate folders or several files in one folder. All the samples go on one plot as engineering stress-strain curves with uncertainty bars, each in its own color.

Claude 2026-10-14 23:07 UTC

Synthetic samples (Plastic 2–11, scaled copies of your one plastic file) are in place, giving 11 total. That's enough to check ordering, colors and the legend. Running it in Octave now.

Claude 2026-10-14 23:09 UTC

The new script is **`plotPlastics.m`**. It finds the plastic samples itself and puts them all on one plot, each as an engineering stress-strain curve with uncertainty bars, in its own color and with its own legend entry. It runs on its own and doesn't change your other scripts.

I only have one real plastic file. To test it, I made 10 scaled copies of that file, so the preview above is not your data. The run in Octave found 11 samples and put them in the right order (Plastic 2 before Plastic 10).

How it finds the samples:

- **Folders:** any folder in M003 whose name contains "plastic" (capitals, spaces and punctuation don't matter). You set the word in `plasticKey`.
- **Files:** every `.dat` file inside those folders is one sample. So it works whether each of your 12 samples has its own folder (Plastic 1–12) or several files share one folder. In the second case each curve is labeled with the folder name plus the file name.
- **When it runs:** the Command Window prints "Found N plastic samples." Check that it says 12.

Dimensions: right now all 12 samples use the one set you gave me (3.0070 × 0.5060 × 0.1365 in). If the samples had different measurements, add a row for each one to the `specimens` table. There's a commented `'plastic 2'` example to copy. A sample with its own row uses it, and the rest fall back to the `'plastic'` row; a row for sample 1 won't be applied to samples 10–12. Send me the 12 sets of measurements and I'll fill in the table.

With 12 curves on one plot, the legend shows one "Uncertainty" entry rather than twelve. Each sample's bars are a lighter shade of its curve color, and there are 10 bars per curve so the plot isn't crowded; change `nErrorBars` for more or fewer.

Save this as `plotPlastics.m` (it's also pushed to the branch):

```
% All plastic samples on one plot: engineering stress-strain curves with
% uncertainty bars.
%
% The script finds every folder inside M003 whose name contains
% "plasticKey" (case, spaces and punctuation are ignored), and every .dat
% file inside those folders (subfolders included) is one sample. So it works
% whether each sample has its own folder (Plastic 1, Plastic 2, ...) or
% several .dat files share one folder.
%   MAE Solids datasets\M003\<plastic folder>\...\<file>.dat
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
%  - strain is crosshead displacement / gauge length (the extensometer
%    channel is used instead if it was recording)
%  - the slack "toe" at the start is removed so each curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
clear; clc; close all;

%% ==================== USER SETTINGS ====================
plasticKey = 'plastic';        % <-- every folder in M003 whose name contains this is plotted

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';

% ---- Specimen measurements [in] ----
% keys:  matched against each sample's label (folder name, plus the file
%        name when a folder holds several .dat files), ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found wins, so add a 'plastic 7' row to give sample 7 its own
%        measurements; samples without their own row use the 'plastic' row.
% i = initial (before the test), f = final (after fracture); NaN = not measured
specimens = {
%   keys        shape   Li      Wi      ti      Lf      Wf      tf
    'plastic',  'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN
%   'plastic 2', 'rect', 3.0000, 0.5000, 0.1350, NaN,   NaN,    NaN   % <-- example of a per-sample row
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

toeCompensation = true;        % remove the slack at the start so each curve starts at 0
fitRange  = [];                % [] = automatic elastic fit (used for the toe removal), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 10;               % number of points along each curve that get uncertainty bars
```

```
saveFigure = false;            % true = save .png and .fig in the M003 folder
%% ========================================================

S = struct('dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', 'auto', 'toe', toeCompensation, ...
    'offset', 0.002, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% Find every plastic sample
testPath = fullfile(resolveBaseFolder(baseFolder), testFolder);
if ~isfolder(testPath), error('Folder not found: %s', testPath); end
norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
d = dir(testPath);
d = d([d.isdir] & ~startsWith({d.name}, '.'));
d = d(contains(cellfun(norm, {d.name}, 'UniformOutput', false), norm(plasticKey)));
if isempty(d), error('No folder in %s has "%s" in its name.', testPath, plasticKey); end
[~, order] = sort(naturalKey({d.name}));
d = d(order);

labels = {};
files  = {};
for k = 1:numel(d)
    folder = fullfile(testPath, d(k).name);
    f = dir(fullfile(folder, '**', '*'));
    f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    if isempty(f)
        warning('No .dat file in %s - skipped.', folder);
        continue;
    end
    [~, order] = sort(naturalKey(fullfile({f.folder}, {f.name})));
    f = f(order);
    for j = 1:numel(f)
        if numel(f) == 1
            labels{end+1} = d(k).name;
        else
            [~, fn] = fileparts(f(j).name);
            labels{end+1} = sprintf('%s - %s', d(k).name, fn);
        end
        files{end+1} = fullfile(f(j).folder, f(j).name);
    end
end
fprintf('Found %d plastic samples.\n', numel(files));

Ms = cellfun(@(n, f) processSample(n, f, S), labels, files, 'UniformOutput', false);

%% Plot: engineering stress-strain curves with uncertainty bars
colors = [0.12 0.47 0.71; 1.00 0.50 0.05; 0.17 0.63 0.17; 0.84 0.15 0.16; ...
          0.58 0.40 0.74; 0.55 0.34 0.29; 0.89 0.47 0.76; 0.50 0.50 0.50; ...
          0.74 0.74 0.13; 0.09 0.75 0.81; 0.00 0.20 0.45; 0.60 0.10 0.30];
fig = figure('Name', 'Plastic samples stress-strain', 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [70 70 1250 750]);
ax = axes(fig);
hold(ax, 'on');
h = gobjects(0);
leg = {};
hErr = gobjects(0);
for m = 1:numel(Ms)
    r  = Ms{m}.r;
    cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
    h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
    leg{end+1} = Ms{m}.name;
    hE = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, nErrorBars, 0.5*cm + 0.5);
    if isempty(hErr), hErr = hE; end
end
hold(ax, 'off');
grid(ax, 'on');
box(ax, 'on');
xlabel(ax, 'Axial Strain (in/in)');
ylabel(ax, Ms{1}.sLabel);
title(ax, sprintf('Stress vs. Strain - Plastic Samples (%d)', numel(Ms)));
xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.r.e), Ms))]);
ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.r.s), Ms))]);
legend(ax, [h hErr], [leg {'Uncertainty (each sample, its color)'}], ...
    'Location', 'eastoutside', 'Interpreter', 'none');

if saveFigure
    outBase = fullfile(testPath, 'plastic_samples_stress_strain');
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: sort key that puts "Plastic 2" before "Plastic 10"
function keys = naturalKey(names)
    keys = cell(size(names));
    for i = 1:numel(names)
        [num, txt] = regexp(lower(names{i}), '\d+', 'match', 'split');
        k = txt{1};
        for j = 1:numel(num)
            k = [k sprintf('%010d', str2double(num{j})) txt{j+1}];
        end
        keys{i} = k;
    end
end

%% Local function: read and analyze one sample (.dat file)
% name is the label for the sample (also used to find its row in the
% specimens table). Returns a struct with the specimen measurements, units,
% and the analyzed curves: r (best strain: extensometer, then displacement)
% and rD (MTS displacement only), plus ultimate/rupture/toughness.
function M = processSample(name, datFile, S)
    M.name = name;
    M.datFile = datFile;
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
```

```
[M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
[M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
haveArea = isfinite(M.A) && M.A > 0;
relA = M.relA;
if ~haveArea
    relA = 0;
    fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
end
if ~isfinite(L0) || L0 <= 0
    error('Enter the gauge length for "%s" in the specimens table.', M.name);
end

% read the data
[data, names, units] = readMtsDat(M.datFile);
iF = findCol(names, 'Axial Force');
iD = findCol(names, 'Axial Displacement');
iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
fUnit = units{iF};
good = isfinite(data(:, iF)) & isfinite(data(:, iD));
if ~isempty(iE), good = good & isfinite(data(:, iE)); end
data = data(good, :);
if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
F  = data(:, iF);
eD = data(:, iD) / L0;
eD = eD - eD(1);

% strain
e = eD;
eExt = [];                      % extensometer strain, in/in (empty if none)
isExt = false(size(e));         % which points come from the extensometer
if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
    eX = data(:, iE);
    if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
    eX = eX - eX(1);
    [~, iRem] = max(eX);                             % last reading before the extensometer came off
    if strcmpi(S.strainSource, 'strain')
        e = eX;
        eExt = eX;
        isExt(:) = true;
    elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
        % after removal, continue with displacement, scaled to match the
        % extensometer over the second half of the time it was on
        iA = max(1, round(iRem/2));
        k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
        if ~isfinite(k) || k <= 0, k = 1; end
        e = eX;
        e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
        eExt = eX;
        isExt(1:iRem) = true;
        if iRem < numel(e) - 5
            fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
        end
    else
        fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
            '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
    end
end
M.eUnit = 'in/in';
eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
eUnc = eDu;
eUnc(isExt) = S.extUnc;

% stress
if haveArea
    s = F / M.A;
    switch lower(fUnit)
        case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
        case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
        case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
        otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
    end
    M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
else
    s = F;
    M.sUnit  = fUnit;
    M.uUnit  = [fUnit '*in/in'];
    M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
end
relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
sUnc = abs(s) * relS;

% break: biggest sudden load drop after the ultimate (one sample or a
% few; the load cell may not read 0 afterwards, it can even go negative)
[sU, iU] = max(s);
iR = numel(s);
d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
if ~isempty(d)
    [dMax, j] = min(d);
    if dMax < 0
        a = j;
        z = j;
        while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
        while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
        if -sum(d(a:z)) > S.breakDrop*sU
            iR = iU + a - 1;    % last point before the drop
        end
    end
end
if iR == numel(s)
    fprintf('  No break found in the data; rupture is the last data point.\n');
end

% rupture = start of the final load drop: ductile samples can tear for
% a while before the last snap, so walk back from the snap while the
% curve is still falling steeply (> 5% of E)
E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
if E0 > 0
    w = max(2, round(0.01*numel(s)));
```

```
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% A key ending in a number only matches that whole number ('plastic1' does
% not match 'plastic12'). Returns 0 if no key matches.
```

```
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && keyFound(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: key appears in text, not followed by a digit if it ends in one
function tf = keyFound(text, key)
    tf = false;
    for i = strfind(text, key)
        after = i + numel(key);
        if ~isstrprop(key(end), 'digit') || after > numel(text) || ~isstrprop(text(after), 'digit')
            tf = true;
            return;
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
```

```
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end
```

You 2026-10-14 23:10 UTC

sorry, i do not need a plot i need For all plastic samples
a) Organized table(s) containing
i)
Average of the ultimate stresses for all the plastic samples
ii)
Minimum ultimate stress recorded
iii) Maximum ultimate stress recorded

Claude 2026-10-14 23:11 UTC

No plot, then: the script will make a figure that holds only tables. Here's what goes in it:

- **Summary table:**
- average ultimate stress
- standard deviation across the samples
- minimum ultimate stress, with the sample it came from
- maximum ultimate stress, with the sample it came from
- how many samples were found
- **Per-sample table:** each sample's ultimate stress ± uncertainty.

The plot code comes out. Since the file no longer plots anything, I'll rename it from `plotPlastics.m` to `plasticUltimateStress.m`.

Claude 2026-10-14 23:11 UTC

One of my text replacements didn't match because that sentence wraps across lines, and it wasn't needed anyway. Nothing was written to the file, so I'll drop that replacement and run it again.

Claude 2026-10-14 23:12 UTC

Testing it in Octave on the 11-sample test set.

Claude 2026-10-14 23:12 UTC

Both tables come out right. The minimum and maximum are matched to the correct samples, and the average is the mean of the 11 values. Committing.

Claude 2026-10-14 23:13 UTC

The plot is gone. The script is now **`plasticUltimateStress.m`** (renamed from `plotPlastics.m`), and its figure has only tables. I ran it in Octave on 11 test samples and both tables filled in correctly. Those test files are scaled copies of your one plastic file, so the numbers below only show the layout. Octave can't draw the tables in the figure, so I checked them in the Command Window output, where the script also prints them.

Table 1 – Ultimate stress summary

Quantity	Value	Units	Sample
Number of samples	11		
Average ultimate stress	3018.5 ± 9.3	psi	
Standard deviation	252.96	psi	sample-to-sample scatter
Minimum ultimate stress	2639 ± 27	psi	Plastic 2
Maximum ultimate stress	3422 ± 35	psi	Plastic 11

Table 2 – each sample's ultimate stress ± uncertainty, sorted Plastic 1, 2, ... 12.

How the summary is worked out:

- **Average ± uncertainty:** the mean of all the ultimate stresses. The ± is the measurement uncertainty of that mean: the square root of the sum of each sample's uncertainty squared, divided by the number of samples.
- **Standard deviation:** how much the samples differ from each other. It's usually much larger than the measurement ±, and both are worth reporting.
- **Minimum and maximum:** each shows its own ± and the name of the sample it came from.

Every plastic sample still uses the one set of dimensions you gave me (3.0070 × 0.5060 × 0.1365 in). If the 12 samples were measured separately, add a row for each to the `specimens` table (the commented `'plastic 2'` line is the template), or send me the measurements and I'll fill it in.

Save this as `plasticUltimateStress.m` (it's also pushed to the branch):

```
% Ultimate stress of every plastic sample: a table of each sample's ultimate
% stress, and a summary table with the average, minimum and maximum.
%
% The script finds every folder inside M003 whose name contains
% "plasticKey" (case, spaces and punctuation are ignored), and every .dat
% file inside those folders (subfolders included) is one sample. So it works
% whether each sample has its own folder (Plastic 1, Plastic 2, ...) or
% several .dat files share one folder.
%   MAE Solids datasets\M003\<plastic folder>\...\<file>.dat
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Ultimate stress = highest engineering stress before rupture (load / initial area).
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   average    sAvg = sum(sU)/n   dsAvg = sqrt(sum(dsU.^2)) / n   (measurement uncertainty)
%   spread     standard deviation of the n ultimate stresses (sample-to-sample scatter)
clear; clc; close all;

%% ==================== USER SETTINGS ====================
plasticKey = 'plastic';        % <-- every folder in M003 whose name contains this is used

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';

% ---- Specimen measurements [in] ----
% keys:  matched against each sample's label (folder name, plus the file
%        name when a folder holds several .dat files), ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found wins, so add a 'plastic 7' row to give sample 7 its own
%        measurements; samples without their own row use the 'plastic' row.
% i = initial (before the test), f = final (after fracture); NaN = not measured
specimens = {
%   keys        shape   Li      Wi      ti      Lf      Wf      tf
    'plastic',  'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN
%   'plastic 2', 'rect', 3.0000, 0.5000, 0.1350, NaN,   NaN,    NaN   % <-- example of a per-sample row
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

toeCompensation = true;        % remove the slack at the start of each curve
fitRange  = [];                % [] = automatic elastic fit (used for the toe removal), or
```

```
                            % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;           % a sudden load drop bigger than this fraction of ultimate
                            % is the break; data after the rupture is ignored

saveFigure = false;         % true = save the table figure (.png and .fig) in the M003 folder
%% ============================================================

S = struct('dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', 'auto', 'toe', toeCompensation, ...
    'offset', 0.002, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;

%% Find every plastic sample
testPath = fullfile(resolveBaseFolder(baseFolder), testFolder);
if ~isfolder(testPath), error('Folder not found: %s', testPath); end
norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
d = dir(testPath);
d = d([d.isdir] & ~startsWith({d.name}, '.'));
d = d(contains(cellfun(norm, {d.name}, 'UniformOutput', false), norm(plasticKey)));
if isempty(d), error('No folder in %s has "%s" in its name.', testPath, plasticKey); end
[~, order] = sort(naturalKey({d.name}));
d = d(order);

labels = {};
files  = {};
for k = 1:numel(d)
    folder = fullfile(testPath, d(k).name);
    f = dir(fullfile(folder, '**', '*'));
    f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    if isempty(f)
        warning('No .dat file in %s - skipped.', folder);
        continue;
    end
    [~, order] = sort(naturalKey(fullfile({f.folder}, {f.name})));
    f = f(order);
    for j = 1:numel(f)
        if numel(f) == 1
            labels{end+1} = d(k).name;
        else
            [~, fn] = fileparts(f(j).name);
            labels{end+1} = sprintf('%s - %s', d(k).name, fn);
        end
        files{end+1} = fullfile(f(j).folder, f(j).name);
    end
end
fprintf('Found %d plastic samples.\n', numel(files));

Ms = cellfun(@(n, f) processSample(n, f, S), labels, files, 'UniformOutput', false);

%% Ultimate stress of each sample
n   = numel(Ms);
sU  = cellfun(@(c) c.p.sU,  Ms);
sUu = cellfun(@(c) c.p.sUu, Ms);
sUnit = Ms{1}.sUnit;

[sMin, iMin] = min(sU);
[sMax, iMax] = max(sU);
sAvg  = mean(sU);
sAvgU = sqrt(sum(sUu.^2)) / n;              % measurement uncertainty of the average
if n > 1, sStd = std(sU); else, sStd = NaN; end

summary = {
    'Number of samples',            sprintf('%d', n),        '',     ''
    'Average ultimate stress',      pm(sAvg, sAvgU),         sUnit,  ''
    'Standard deviation',           pm(sStd, NaN),           sUnit,  'sample-to-sample scatter'
    'Minimum ultimate stress',      pm(sMin, sUu(iMin)),     sUnit,  Ms{iMin}.name
    'Maximum ultimate stress',      pm(sMax, sUu(iMax)),     sUnit,  Ms{iMax}.name
    };
perSample = [cellfun(@(c) c.name, Ms(:), 'UniformOutput', false), ...
             arrayfun(@(k) pm(sU(k), sUu(k)), (1:n)', 'UniformOutput', false)];

%% Tables
fig = figure('Name', 'Plastic ultimate stress', 'NumberTitle', 'off', ...
    'Color', 'w', 'Position', [70 70 1100 650]);
uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.03 0.90 0.50 0.05], ...
    'String', 'Plastic samples - ultimate stress summary', 'FontWeight', 'bold', 'FontSize', 12, ...
    'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
uitable(fig, 'Data', summary, 'ColumnName', {'Quantity', 'Value', 'Units', 'Sample'}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.03 0.55 0.55 0.35], 'FontSize', 11, ...
    'ColumnWidth', {190, 150, 60, 190});
uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.62 0.90 0.35 0.05], ...
    'String', 'Ultimate stress of each sample', 'FontWeight', 'bold', 'FontSize', 12, ...
    'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
uitable(fig, 'Data', perSample, 'ColumnName', {'Sample', ['Ultimate stress (' sUnit ')']}, ...
    'RowName', [], 'Units', 'normalized', 'Position', [0.62 0.05 0.35 0.85], 'FontSize', 11, ...
    'ColumnWidth', {170, 170});
uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.03 0.30 0.55 0.20], ...
    'String', {'Ultimate stress = highest engineering stress (load / initial area) before rupture.', ...
               'Average uncertainty = sqrt(sum of squared sample uncertainties) / n.', ...
               'Standard deviation = scatter between the samples.'}, ...
    'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

printTable('Plastic samples - ultimate stress summary', {'Quantity', 'Value', 'Units', 'Sample'}, summary);
printTable('Ultimate stress of each sample', {'Sample', ['Ultimate stress (' sUnit ')']}, perSample);

if saveFigure
    outBase = fullfile(testPath, 'plastic_ultimate_stress');
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: sort key that puts "Plastic 2" before "Plastic 10"
function keys = naturalKey(names)
    keys = cell(size(names));
    for i = 1:numel(names)
        [num, txt] = regexp(lower(names{i}), '\d+', 'match', 'split');
        k = txt{1};
```

```
        for j = 1:numel(num)
            k = [k sprintf('%010d', str2double(num{j})) txt{j+1}];
        end
        keys{i} = k;
    end
end

%% Local function: read and analyze one sample (.dat file)
% name is the label for the sample (also used to find its row in the
% specimens table). Returns a struct with the specimen measurements, units,
% and the analyzed curves: r (best strain: extensometer, then displacement)
% and rD (MTS displacement only), plus ultimate/rupture/toughness.
function M = processSample(name, datFile, S)
    M.name = name;
    M.datFile = datFile;
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
```

```
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
```

```
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% A key ending in a number only matches that whole number ('plastic1' does
% not match 'plastic12'). Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && keyFound(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: key appears in text, not followed by a digit if it ends in one
function tf = keyFound(text, key)
    tf = false;
    for i = strfind(text, key)
        after = i + numel(key);
        if ~isstrprop(key(end), 'digit') || after > numel(text) || ~isstrprop(text(after), 'digit')
            tf = true;
            return;
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
```

```
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

You 2026-10-14 23:33 UTC

📎 NewMaterialsCode.m 📎 SteelCode.m 📎 SteelandAlumcode.m 📎 bothcarbonfiber.m 📎 Alltogether.m put these matlab codes together to be 1 large matlab code. ensure it does not lose its functions

Claude 2026-10-14 23:34 UTC

I'll merge the five scripts into one file, `materialsAnalysis.m`, organized as five sections. Each section makes the same figures and tables its original script made, and each can be switched on or off. A few decisions:

- **Shared settings and helpers:** the settings (folders, specimen measurements, uncertainties, published values) and the helper functions appear once. Each section keeps its own material list exactly as in your files.
- **Steel's single-material figure only once:** both `SteelCode.m` and `SteelandAlumcode.m` made it. The single-material section takes a list (`{'m003alum', 'm003_steel'}`), which covers

`NewMaterialsCode.m` as well.
- **Each file read once:** a material used by several sections isn't reprocessed.
- **Sections run independently:** each runs inside its own error handler, so a problem in one (for example `'plastic'` matching several folders) prints a warning without stopping the others.

Claude 2026-10-14 23:38 UTC

Now build an Octave test copy.

Claude 2026-10-14 23:39 UTC

Sections 1–4 work; section 5 skipped as expected because the test data has 11 plastic folders. Re-running with a single plastic folder to exercise section 5:

Claude 2026-10-14 23:39 UTC

All five sections run. The figures match the separate scripts, and the uitables don't render in Octave's PNG export, which is expected. Committing and pushing now.

Claude 2026-10-14 23:42 UTC

I combined your five scripts into one file, `materialsAnalysis.m`, and kept every figure, table and setting from them. It's pushed to `claude/brave-johnson-5v00jz`. I ran all five sections in Octave on your sample files and they made the same plots and tables as the separate scripts.

Where each of your scripts went:

Your file	Section in the merged code
`NewMaterialsCode.m` (aluminum)	1. Single material, `singleMaterials = {'m003alum', 'm003_steel'}`
`SteelCode.m`, figure 1	1. (the steel figure)
`SteelCode.m`, extensometer vs MTS	2. `extVsMtsMaterial = 'm003_steel'`
`SteelandAlumcode.m`, figure 1	1. (same steel figure, so it appears once)
`SteelandAlumcode.m`, figure 2	3. `compareMaterials`, with true curves and published values (79770 / 42060 psi rupture)
`bothcarbonfiber.m`	4. `carbonMaterials`
`Alltogether.m`	5. `allMaterials`, `useExtensometer = false`

How it runs:

- **Switches:** at the top there's an on/off switch for each section (`runSingle`, `runExtVsMTS`, `runComparison`, `runCarbon`, `runAll`).
- **Each file is read once:** a material that shows up in several sections, like steel, is read and analyzed one time and reused.
- **Sections are independent:** if one section hits a problem, it prints a warning and the rest still run.
- **Measurements in one place:** specimen sizes, published values and uncertainties are entered once and shared by every section.

If the all-materials plot is skipped: `'plastic'` in `allMaterials` will match more than one folder if you have several plastic folders. Section 5 then skips with a warning listing the matches. Replace `'plastic'` with the full name of the one you want, for example `'Plastic 1'`.

```
% Stress-strain analysis of the M003 tensile tests - all figures in one script.
%
% Sections (each can be switched on/off below, and each runs on its own, so a
% problem in one section does not stop the others):
%   1. Single material   - for every folder in "singleMaterials": stress-strain
%                          curve with a 0.2% offset line, uncertainty bars,
%                          yield, ultimate and rupture points, and a table of the
%                          lab measurements and results with uncertainties.
```

```
%   2. Extensometer vs   - for "extVsMtsMaterial" (steel): the extensometer and
%      MTS displacement    MTS displacement readings on one plot, each with its
%                          own offset line, uncertainty bars and yield point,
%                          and a table of E, yield and resilience with the
%                          percent difference (extensometer = reference).
%   3. Material          - "compareMaterials" (steel and aluminum) on one plot:
%      comparison          engineering and true stress-strain curves, offset
%                          lines, uncertainty bars, yield and true rupture
%                          points, plus tables of each reading and of the
%                          materials vs. published values.
%   4. Carbon fiber      - "carbonMaterials" on one plot with uncertainty bars,
%                          and a table of ultimate, rupture and toughness.
%   5. All materials     - "allMaterials" on one plot with uncertainty bars.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Material folders are found by name (case, spaces and punctuation are
% ignored, and '|' separates alternative names). Enter each specimen's
% measurements once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve     dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)            dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af    dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   true curve (to the ultimate, before necking)  sT = s*(1 + e),  eT = ln(1 + e)
%   percent difference   |measured - reference| / reference * 100
clear; clc; close all;

%% ==================== USER SETTINGS ====================
% ---- Which sections to run ----
runSingle     = true;          % 1. single-material figure(s)
runExtVsMTS   = true;          % 2. extensometer vs. MTS displacement (steel)
runComparison = true;          % 3. steel vs. aluminum comparison
runCarbon     = true;          % 4. carbon fiber samples
runAll        = true;          % 5. all materials on one plot

% ---- Materials for each section (folders inside M003) ----
singleMaterials   = {'m003alum', 'm003_steel'};          % 1. one figure each
extVsMtsMaterial  = 'm003_steel';                        % 2. needs extensometer data
compareMaterials  = {'m003_steel', 'm003alum'};          % 3.
includeExtensometer = false;   % 3. false = MTS displacement curves only;
                               %    true = also the extensometer curve where the file has one
carbonMaterials   = {'M003 Carbon Fiber 90', 'm003 cf45'};   % 4.
allMaterials      = {'steel', 'alum', 'cf90|carbon fiber 90', 'cf45|carbon fiber 45|cf40|carbon fiber 40', 'plastic'};
                               % 5. (if you have several plastic folders, use the full
                               %    name of the one to plot)
useExtensometer   = false;     % 5. false = MTS displacement strain for every material;
                               %    true  = extensometer while it was on, where a file has it

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN      % <-- final measurements not entered
    };

% ---- Published values for the percent differences (section 3) [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
published = {
%   keys       name                       yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',    53700,   63800,    79770,   29.7e6
    'alum',    '6061-T6',                 40000,   45000,    42060,   10.0e6
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
```

```
%                 displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save every figure as .png and .fig (single-material
                               % figures in their material folder, the rest in M003)
%% ========================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;
opt = struct('offset', offset, 'nErrorBars', nErrorBars, 'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc);
cache = containers.Map();      % each material is read and analyzed only once
testPath = findSubfolder(resolveBaseFolder(baseFolder), testFolder);

%% ================ 1. Single material ================
if runSingle
    for m = 1:numel(singleMaterials)
        try
            M = loadMaterial(singleMaterials{m}, S, cache);
            fig = plotSingle(M, opt);
            if saveFigure, saveFig(fig, fullfile(M.path, [M.name '_stress_strain'])); end
        catch err
            warning('Single-material figure for "%s" skipped: %s', singleMaterials{m}, err.message);
        end
    end
end

%% ============ 2. Extensometer vs. MTS displacement ============
if runExtVsMTS && ~isempty(extVsMtsMaterial)
    try
        M = loadMaterial(extVsMtsMaterial, S, cache);
        fig = plotExtVsMTS(M, opt);
        if saveFigure && ~isempty(fig)
            saveFig(fig, fullfile(M.path, [M.name '_stress_strain_ext_vs_MTS']));
        end
    catch err
        warning('Extensometer vs. MTS figure skipped: %s', err.message);
    end
end

%% ============ 3. Material comparison (steel vs. aluminum) ============
if runComparison && ~isempty(compareMaterials)
    try
        Ms = cellfun(@(m) loadMaterial(m, S, cache), compareMaterials, 'UniformOutput', false);
        fig = plotComparison(Ms, published, includeExtensometer, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'comparison_stress_strain')); end
    catch err
        warning('Material comparison figure skipped: %s', err.message);
    end
end

%% ============ 4. Carbon fiber samples ============
if runCarbon && ~isempty(carbonMaterials)
    try
        Cs = cellfun(@(m) loadMaterial(m, S, cache), carbonMaterials, 'UniformOutput', false);
        fig = plotCarbon(Cs, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'carbon_fiber_stress_strain')); end
    catch err
        warning('Carbon fiber figure skipped: %s', err.message);
    end
end

%% ============ 5. All materials ============
if runAll && ~isempty(allMaterials)
    try
        Ms = cellfun(@(m) loadMaterial(m, S, cache), allMaterials, 'UniformOutput', false);
        fig = plotAll(Ms, useExtensometer, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'all_materials_stress_strain')); end
    catch err
        warning('All-materials figure skipped: %s', err.message);
    end
end

%% Local function: section 1 - one material with its results table
function fig = plotSingle(M, opt)
    r = M.r;
    p = M.p;
    e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
    E = r.E;  sY = r.sY;  eY = r.eY;
    iU = p.iU;  iR = numel(s);
    sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

    fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [60 60 1500 700]);
    ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
    hold(ax, 'on');

    hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

    % uncertainty bars on evenly spaced points along the curve
    hErr = drawUncertainty(ax, e, s, eUnc, sUnc, opt.nErrorBars, [0.5 0.5 0.5]);

    % 0.2% offset line, from zero stress up to a bit past yield
    hOff = gobjects(0);
    if isfinite(E) && E > 0
```

```
        [eLine, sLine] = offsetLine(r, opt.offset, sU);
        hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
    end

    % key points
    hY = gobjects(0);
    if isfinite(sY)
        hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
            'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
        text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
            'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
    end
    hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
    hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
    if iU == iR                     % brittle: breaks at the ultimate
        text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
    else
        text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
        text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
            'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
    end
    hold(ax, 'off');

    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
    ylabel(ax, M.sLabel);
    title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
    eSpan = max(e) - min(0, min(e));
    sSpan = sU - min(0, min(s));
    if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
    if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

    hLeg = [hCurve hErr hOff hY hU hR];
    legTxt = {'Stress-strain curve', 'Uncertainty'};
    if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', opt.offset*100); end
    if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
    legTxt = [legTxt {'Ultimate', 'Rupture'}];
    legend(ax, hLeg, legTxt, 'Location', 'best');

    % results table
    if strcmpi(M.shape, 'round')
        dimRows = {'Initial diameter', pm(M.Wi, opt.dimUnc), 'in'
                   'Final diameter',   pm(M.Wf, opt.dimUnc), 'in'};
    else
        dimRows = {'Initial width',     pm(M.Wi, opt.dimUnc), 'in'
                   'Initial thickness', pm(M.ti, opt.dimUnc), 'in'
                   'Final width',       pm(M.Wf, opt.dimUnc), 'in'
                   'Final thickness',   pm(M.tf, opt.dimUnc), 'in'};
    end
    rows = [
        {'LAB MEASUREMENTS', '', ''}
        {'Initial gauge length', pm(M.L0, opt.gaugeUnc), 'in'}
        {'Final gauge length',   pm(M.Lf, opt.gaugeUnc), 'in'}
        dimRows
        {'RESULTS', '', ''}
        {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
        {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
        {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
        {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
        {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
        {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
        {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
        {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
        {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
        ];
    uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
        'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
    printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);
end

%% Local function: section 2 - extensometer vs. MTS displacement of one material
% Both readings analyzed separately on one plot. The extensometer curve stops
% where the extensometer was removed.
function fig = plotExtVsMTS(M, opt)
    fig = [];
    if isempty(M.rX)
        warning('"%s" has no extensometer data - extensometer vs. MTS figure skipped.', M.name);
        return;
    end
    rX = M.rX;
    rD = M.rD;

    fig = figure('Name', sprintf('%s extensometer vs MTS', M.name), 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [90 90 1500 700]);
    ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
    hold(ax, 'on');
    cX = [0 0.3 0.6];               % extensometer
    cD = [0.85 0.33 0.1];           % MTS displacement
    readings = {rX, cX, 'Extensometer', 'ext.'; rD, cD, 'MTS displacement', 'MTS'};
    sMax = max([rX.s; rD.s]);
    h = gobjects(0);
    leg = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:2
        [rq, cq, nameq, shortq] = readings{q, :};
        h(end+1) = plot(ax, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg{end+1} = nameq;
        h(end+1) = drawUncertainty(ax, rq.e, rq.s, rq.eUnc, rq.sUnc, opt.nErrorBars, 0.5*cq + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, opt.offset, max(rq.s));
            h(end+1) = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
```

```
            leg{end+1} = sprintf('%g%% offset line (%s)', opt.offset*100, shortq);
        end
        if isfinite(rq.sY)
            h(end+1) = plot(ax, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg{end+1} = sprintf('Yield (%s)', shortq);
            yLab = placeLabel(rq.sY - 0.04*sMax, labY, sMax);
            labY(end+1) = yLab;
            text(ax, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'BackgroundColor', 'w', 'Margin', 1);
        end
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
    ylabel(ax, M.sLabel);
    title(ax, sprintf('%s - Extensometer vs. MTS Displacement', M.name), 'Interpreter', 'none');
    xlim(ax, [0, 1.05*max([rX.e; rD.e])]);
    ylim(ax, [0, 1.12*sMax]);
    legend(ax, h, leg, 'Location', 'southeast');

    % percent difference, extensometer taken as the accurate value
    rows = {
        ['Young''s modulus E (' M.sUnit ')'],      pm(rX.E,  rX.EUnc), pm(rD.E,  rD.EUnc), pctDiff(rD.E,  rX.E)
        ['Yield stress, 0.2% (' M.sUnit ')'],      pm(rX.sY, rX.sYu),  pm(rD.sY, rD.sYu),  pctDiff(rD.sY, rX.sY)
        ['Yield strain (' M.eUnit ')'],            pm(rX.eY, rX.eYu),  pm(rD.eY, rD.eYu),  pctDiff(rD.eY, rX.eY)
        ['Modulus of resilience (' M.uUnit ')'],   pm(rX.Ur, rX.Uru),  pm(rD.Ur, rD.Uru),  ''
        };
    cols = {'Quantity', 'Extensometer', 'MTS displacement', '% difference'};
    uitable(fig, 'Data', rows, 'ColumnName', cols, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.57 0.40 0.42 0.30], ...
        'ColumnWidth', {215, 165, 165, 95}, 'FontSize', 11);
    printTable(sprintf('%s - extensometer vs. MTS displacement', M.name), cols, rows);
end

%% Local function: section 3 - materials compared on one plot
function fig = plotComparison(Ms, published, includeExtensometer, opt)
    sUnit = Ms{1}.sUnit;
    uUnit = Ms{1}.uUnit;

    % readings to plot: MTS displacement, plus the extensometer if asked for
    % and the file has it
    useExt = cellfun(@(c) includeExtensometer && ~isempty(c.rX), Ms);
    R = cell(0, 4);                 % {analyzed curve, legend name, short label, material (0 = extensometer)}
    for m = 1:numel(Ms)
        if useExt(m)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.'], 0};
        end
        R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS'], m};
    end

    % true stress-strain up to the ultimate (valid until necking starts)
    for m = 1:numel(Ms)
        iUm = Ms{m}.pD.iU;
        eEng = Ms{m}.rD.e(1:iUm);
        Ms{m}.eT = log(1 + eEng);
        Ms{m}.sT = Ms{m}.rD.s(1:iUm) .* (1 + eEng);
    end
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

    fig = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [40 40 1650 820]);
    ax = axes(fig, 'Position', [0.055 0.08 0.415 0.85]);
    hold(ax, 'on');
    sMax = max([cellfun(@(c) max(c.s), R(:, 1)); cellfun(@(c) max([c.sT; c.sTR]), Ms(:))]);
    eMax = max([cellfun(@(c) max(c.e), R(:, 1)); cellfun(@(c) max([c.eT; c.eTR]), Ms(:))]);
    h = gobjects(0);
    leg = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:size(R, 1)
        [rq, nameq, shortq, mq] = R{q, :};
        cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg{end+1} = nameq;
        h(end+1) = drawUncertainty(ax, rq.e, rq.s, rq.eUnc, rq.sUnc, opt.nErrorBars, 0.5*cq + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, opt.offset, max(rq.s));
            h(end+1) = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg{end+1} = sprintf('%g%% offset line (%s)', opt.offset*100, shortq);
        end
        if isfinite(rq.sY)
            h(end+1) = plot(ax, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg{end+1} = sprintf('Yield (%s)', shortq);
            yLab = placeLabel(rq.sY - 0.04*sMax, labY, sMax);
            labY(end+1) = yLab;
            text(ax, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
        end
        if mq > 0                   % true curve and true rupture point, same color as the MTS curve
            Mm = Ms{mq};
            h(end+1) = plot(ax, Mm.eT, Mm.sT, '-.', 'LineWidth', 1.5, 'Color', cq);
            leg{end+1} = sprintf('True stress-strain (%s)', Mm.name);
            if isfinite(Mm.sTR) && isfinite(Mm.eTR)
                h(end+1) = plot(ax, Mm.eTR, Mm.sTR, 'd', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                    'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
                leg{end+1} = sprintf('True rupture (%s)', Mm.name);
                text(ax, Mm.eTR - 0.012*eMax, Mm.sTR, sprintf('True rupture, %s (%.4g, %.0f)', Mm.name, Mm.eTR, Mm.sTR), ...
                    'VerticalAlignment', 'middle', 'HorizontalAlignment', 'right', 'Color', cq, ...
                    'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
            end
        end
    end
    hold(ax, 'off');
```

```
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Engineering / True Strain (in/in)');
    ylabel(ax, strrep(Ms{1}.sLabel, 'Engineering Stress', 'Engineering / True Stress'));
    if any(useExt), ttl = 'Stress vs. Strain - '; else, ttl = 'Stress vs. Strain (MTS displacement) - '; end
    title(ax, [ttl strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax, [0, 1.05*eMax]);
    ylim(ax, [0, 1.12*sMax]);
    legend(ax, h, leg, 'Location', 'southeast', 'Interpreter', 'none');

    % ---- Table 1: each reading ----
    cols1 = {'Quantity'};
    T1 = {['Young''s modulus E (' sUnit ')']; ['Yield stress, 0.2% (' sUnit ')']; ...
          'Yield strain (in/in)'; ['Modulus of resilience (' uUnit ')']};
    for m = 1:numel(Ms)
        rD = Ms{m}.rD;
        colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
        if useExt(m)
            rX = Ms{m}.rX;
            colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
            colP = {pctDiff(rD.E, rX.E); pctDiff(rD.sY, rX.sY); pctDiff(rD.eY, rX.eY); ''};
            cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
            T1 = [T1, colX, colD, colP];
        else
            cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
            T1 = [T1, colD];
        end
    end

    % ---- Table 2: materials and published values ----
    % ultimate, rupture and toughness from the MTS displacement curve (the
    % extensometer comes off before the ultimate); yield and E from the
    % extensometer when it is plotted, MTS displacement otherwise
    labels2 = {['Ultimate stress (' sUnit ')']; 'Strain at ultimate (in/in)'; ...
               ['Rupture stress (' sUnit ')']; 'Strain at rupture (in/in)'; ...
               ['True rupture stress (' sUnit ')']; 'True rupture strain (in/in)'; ...
               ['Yield stress, 0.2% (' sUnit ')']; ['Young''s modulus E (' sUnit ')']; ...
               ['Modulus of toughness (' uUnit ')']};
    pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
    cols2 = {'Quantity'};
    T2 = labels2;
    pubNames = {};
    for m = 1:numel(Ms)
        Mm = Ms{m};
        pD = Mm.pD;
        if useExt(m), best = Mm.rX; else, best = Mm.rD; end
        vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
                Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
        [pub, pubName] = lookupPublished(published, Mm.name);
        if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
        colV = cell(numel(labels2), 1);
        colPub = repmat({''}, numel(labels2), 1);
        colPd  = repmat({''}, numel(labels2), 1);
        for k = 1:numel(labels2)
            colV{k} = pm(vals(k, 1), vals(k, 2));
            if pubCol(k) > 0
                ref = pub(pubCol(k));
                if isfinite(ref)
                    colPub{k} = sprintf('%g', ref);
                    colPd{k}  = pctDiff(vals(k, 1), ref);
                else
                    colPub{k} = 'N/A';
                end
            end
        end
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    if any(useExt)
        strainNote = ['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
                      'the extensometer where it is plotted.'];
    else
        strainNote = 'All strains are from the MTS crosshead displacement.';
    end
    note = {strainNote, ...
            ['True curve (dash-dot): stress x (1 + strain) and ln(1 + strain), up to the ultimate (valid until necking). ' ...
             'True rupture: rupture load / final area, ln(initial area / final area).'], ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

%% Local function: section 4 - carbon fiber samples with their results table
function fig = plotCarbon(Cs, opt)
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6];
    fig = figure('Name', 'Carbon fiber stress-strain', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [70 70 1500 700]);
    ax = axes(fig, 'Position', [0.06 0.10 0.50 0.82]);
    hold(ax, 'on');
    h = gobjects(0);
```

```
    leg = {};
    for m = 1:numel(Cs)
        r  = Cs{m}.r;
        cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
        leg{end+1} = Cs{m}.name;
        h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, opt.nErrorBars, 0.5*cm + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', Cs{m}.name);
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Axial Strain (in/in)');
    ylabel(ax, Cs{1}.sLabel);
    title(ax, ['Stress vs. Strain - ' strjoin(cellfun(@(c) c.name, Cs, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.r.e), Cs))]);
    ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.r.s), Cs))]);
    legend(ax, h, leg, 'Location', 'southeast', 'Interpreter', 'none');

    % table: ultimate, rupture and modulus of toughness of each sample
    sUnit = Cs{1}.sUnit;
    T = {['Ultimate stress (' sUnit ')']; 'Strain at ultimate (in/in)'; ...
         ['Rupture stress (' sUnit ')']; 'Strain at rupture (in/in)'; ...
         ['Modulus of toughness (' Cs{1}.uUnit ')']};
    cols = {'Quantity'};
    for m = 1:numel(Cs)
        p = Cs{m}.p;
        T = [T, {pm(p.sU, p.sUu); pm(p.eU, p.eUu); pm(p.sR, p.sRu); pm(p.eR, p.eRu); pm(p.UT, p.UTu)}];
        cols = [cols, {Cs{m}.name}];
    end
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.80 0.38 0.04], ...
        'String', 'Carbon fiber results', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T, 'ColumnName', cols, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.60 0.52 0.38 0.28], 'FontSize', 11, ...
        'ColumnWidth', [{220}, repmat({170}, 1, numel(Cs))]);
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.38 0.38 0.12], ...
        'String', {'Strain is from the MTS crosshead displacement.', ...
                   'A sample that breaks at its highest load has rupture = ultimate.'}, ...
        'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    printTable('Carbon fiber results', cols, T);
end

%% Local function: section 5 - all materials on one plot
function fig = plotAll(Ms, useExtensometer, opt)
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];
    fig = figure('Name', 'All materials stress-strain', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [70 70 1200 750]);
    ax = axes(fig);
    hold(ax, 'on');
    h = gobjects(0);
    leg = {};
    curves = cell(size(Ms));
    for m = 1:numel(Ms)
        if useExtensometer, r = Ms{m}.r; else, r = Ms{m}.rD; end
        curves{m} = r;
        cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
        leg{end+1} = Ms{m}.name;
        h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, opt.nErrorBars, 0.5*cm + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', Ms{m}.name);
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Axial Strain (in/in)');
    ylabel(ax, Ms{1}.sLabel);
    if useExtensometer, ttl = 'Stress vs. Strain - All Materials';
    else, ttl = 'Stress vs. Strain - All Materials (MTS displacement)'; end
    title(ax, ttl);
    xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.e), curves))]);
    ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.s), curves))]);
    legend(ax, h, leg, 'Location', 'eastoutside', 'Interpreter', 'none');
end

%% Local function: material from the cache, or read and analyze it once
function M = loadMaterial(name, S, cache)
    path = findSubfolder(findSubfolder(resolveBaseFolder(S.baseFolder), S.testFolder), name);
    if isKey(cache, path)
        M = cache(path);
    else
        M = processMaterial(name, S);
        cache(path) = M;
    end
end

%% Local function: save a figure as .png and .fig
function saveFig(fig, outBase)
    if isempty(fig), return; end
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: label height just below a point, stepped further down
% only if it would overlap a label already placed
function yLab = placeLabel(yLab, labY, sMax)
    while any(abs(labY - yLab) < 0.06*sMax)
        yLab = yLab - 0.06*sMax;
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
```

```
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    M.path   = findSubfolder(testPath, material);
    [~, M.name] = fileparts(M.path);
    M.datFile = findDatFile(M.path, S.datFileName);
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
```

```
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);

    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end
```

```
%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && contains(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
```

```
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name
% Tries each '|'-separated alternative: exact match, then partial match,
% then partial match ignoring case, spaces and punctuation.
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
    normNames = cellfun(norm, names, 'UniformOutput', false);
    hit = false(size(names));
    for alt = strtrim(strsplit(name, '|'))
        a = alt{1};
        if isempty(a), continue; end
        hit = strcmpi(names, a);
        if ~any(hit), hit = contains(names, a, 'IgnoreCase', true); end
        if ~any(hit), hit = contains(normNames, norm(a)); end
        if any(hit), break; end
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
```

```
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dex = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dex, x/10^ex, char(177), dex, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

Steel strain values: the steel ultimate and rupture strains in the steel/aluminum comparison table (0.157 / 0.188) are larger than in the steel single-material table (0.136 / 0.162). That's expected: the comparison uses MTS displacement only, the same as your `SteelandAlumcode.m`. The single-material figure uses the extensometer while it was on.

You 2026-10-14 23:43 UTC

📎 `AllPlastic.m` add this last one to the code

Claude 2026-10-14 23:43 UTC

Now the edits: split `processMaterial` so the plastic samples reuse the same analysis, add section 6, and bring in `naturalKey`, `keyFound` and the plastic table function.

Claude 2026-10-14 23:43 UTC

Now the settings and header.

Claude 2026-10-14 23:43 UTC

My throwaway third replacement was wrong; removing it and re-running.

Claude 2026-10-14 23:47 UTC

I added `AllPlastic.m` to the merged script as section 6. It's pushed to `claude/brave-johnson-5v00jz`. I ran all six sections in Octave on the sample data: sections 1–5 gave the same results as before, and section 6 gave the same tables as `AllPlastic.m`.

What section 6 does (no plot, same as your file):

- It finds every folder in M003 whose name contains `plasticKey = 'M003 Plastic'`, and treats every .dat file inside as one sample.
- It shows a summary table with the number of samples, average ± uncertainty, standard deviation, and the minimum and maximum with the sample name.
- It also shows a table of each sample's ultimate stress.
- It turns on and off with `runPlastic`.
- It uses the main `specimens` table. Samples without their own row use the `'plastic'` row; a commented-out `'plastic 2'` row shows how to give one sample its own measurements.
- With `saveFigure = true`, it saves `plastic_ultimate_stress` in the M003 folder.

Other changes:

- **Same calculation for every material:** I split the old `processMaterial` so the plastic samples and the other materials go through the same analysis code (`processSample`). The results for the other sections didn't change.
- **Name matching:** I used your plastic file's rule, so a key like `plastic1` no longer matches `Plastic 12`.

Full code to copy and paste:

```
% Stress-strain analysis of the M003 tensile tests - all figures in one script.
%
% Sections (each can be switched on/off below, and each runs on its own, so a
% problem in one section does not stop the others):
%   1. Single material   - for every folder in "singleMaterials": stress-strain
%                          curve with a 0.2% offset line, uncertainty bars,
%                          yield, ultimate and rupture points, and a table of the
%                          lab measurements and results with uncertainties.
%   2. Extensometer vs   - for "extVsMtsMaterial" (steel): the extensometer and
%      MTS displacement    MTS displacement readings on one plot, each with its
%                          own offset line, uncertainty bars and yield point,
%                          and a table of E, yield and resilience with the
%                          percent difference (extensometer = reference).
%   3. Material          - "compareMaterials" (steel and aluminum) on one plot:
%      comparison          engineering and true stress-strain curves, offset
%                          lines, uncertainty bars, yield and true rupture
%                          points, plus tables of each reading and of the
%                          materials vs. published values.
%   4. Carbon fiber      - "carbonMaterials" on one plot with uncertainty bars,
%                          and a table of ultimate, rupture and toughness.
%   5. All materials     - "allMaterials" on one plot with uncertainty bars.
%   6. Plastic samples   - every .dat file in the folders matching "plasticKey"
%                          (no plot): a table of each sample's ultimate stress
%                          and a summary of the average, minimum and maximum.
%
% Folder layout:
%   MAE Solids datasets\M003\<material folder>\...\<file>.dat
% Material folders are found by name (case, spaces and punctuation are
% ignored, and '|' separates alternative names). Enter each specimen's
% measurements once in the "specimens" table.
%
% DAT file (MTS 793 export): a few header lines, then a tab-delimited line of
% channel names (Axial Force, Axial Displacement, Axial Strain), a line of
% units (lbf, in, in/in), then the data.
%
% Works for brittle (carbon fiber), ductile (aluminum, steel) and plastic
% samples:
%  - strain comes from the extensometer while it was on the specimen, and
%    from crosshead displacement after it was removed (or if it never was on)
%  - the slack "toe" at the start is removed so the curve starts at 0
%  - nothing after the rupture point is plotted
%
% Uncertainties (propagated as independent errors, root-sum-square):
%   area       A = w*t            dA/A  = sqrt((dw/w)^2 + (dt/t)^2)
%              A = pi*d^2/4       dA/A  = 2*dd/d
%   stress     s = F/A            ds/s  = sqrt((dF/F)^2 + (dA/A)^2)
%   strain     e = dL/L0          de    = sqrt((d(dL)/L0)^2 + (e*dL0/L0)^2)
%              (extensometer)     de    = extensometer accuracy
%   modulus    E = slope of fit   dE/E  = sqrt((SE/E)^2 + (ds/s)^2 + (de/e)^2)
%   toughness  UT = area under curve      dUT/UT = sqrt((ds/s)^2 + (deR/eR)^2)
%   resilience Ur = sY^2/(2E)             dUr/Ur = sqrt((2*dsY/sY)^2 + (dE/E)^2)
%   true rupture stress  sTR = F_R/Af     dsTR/sTR = sqrt((dF/F)^2 + (dAf/Af)^2)
%   true rupture strain  eTR = ln(Ai/Af)  deTR     = sqrt((dAi/Ai)^2 + (dAf/Af)^2)
%   true curve (to the ultimate, before necking)  sT = s*(1 + e),  eT = ln(1 + e)
%   percent difference   |measured - reference| / reference * 100
%   plastic average      sAvg = sum(sU)/n     dsAvg = sqrt(sum(dsU.^2)) / n
clear; clc; close all;

%% ===================== USER SETTINGS =====================
% ---- Which sections to run ----
runSingle     = true;          % 1. single-material figure(s)
runExtVsMTS   = true;          % 2. extensometer vs. MTS displacement (steel)
runComparison = true;          % 3. steel vs. aluminum comparison
runCarbon     = true;          % 4. carbon fiber samples
runAll        = true;          % 5. all materials on one plot
runPlastic    = true;          % 6. plastic samples - ultimate stress tables

% ---- Materials for each section (folders inside M003) ----
singleMaterials   = {'m003alum', 'm003_steel'};         % 1. one figure each
extVsMtsMaterial  = 'm003_steel';                       % 2. needs extensometer data
compareMaterials  = {'m003_steel', 'm003alum'};         % 3.
includeExtensometer = false;   % 3. false = MTS displacement curves only;
                               %    true = also the extensometer curve where the file has one
carbonMaterials   = {'M003 Carbon Fiber 90', 'm003 cf45'};   % 4.
allMaterials      = {'steel', 'alum', 'cf90|carbon fiber 90', 'cf45|carbon fiber 45|cf40|carbon fiber 40', 'plastic'};
                               % 5. (if you have several plastic folders, use the full
                               %    name of the one to plot)
useExtensometer   = false;     % 5. false = MTS displacement strain for every material;
                               %    true  = extensometer while it was on, where a file has it
plasticKey        = 'M003 Plastic';   % 6. every folder in M003 whose name contains this
                                      %    (case, spaces and punctuation ignored); every
                                      %    .dat file inside is one sample

baseFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Solids datasets';
testFolder  = 'M003';
datFileName = '';              % '' = find the .dat file automatically, or e.g. 'specimen.dat'

% ---- Specimen measurements (one row per specimen) [in] ----
% keys:  matched against the material folder name, ignoring case, spaces
%        and punctuation; separate alternatives with '|'. The longest key
%        found in the folder name wins, so 'plastic2' beats 'plastic'
%        (and 'plastic1' does not match 'plastic12'). Plastic samples
%        without their own row use the 'plastic' row.
% shape: 'rect' (width x thickness) or 'round' (diameter in the W columns,
%        NaN in the t columns)
```

```
% i = initial (before the test), f = final (after fracture)
% NaN = not measured -> initial width/thickness: the stress axis falls
%       back to axial force (lbf); final values: N/A in the table
specimens = {
%   keys                          shape   Li      Wi      ti      Lf      Wf      tf
    'steel',                      'rect', 2.7535, 0.5100, 0.0650, 3.0065, 0.4355, 0.0575
    'alum',                       'rect', 2.6430, 0.5160, 0.0615, 2.8455, 0.4650, 0.0550
    'cf90|carbonfiber90',         'rect', 2.5110, 0.5240, 0.0605, 2.5640, 0.5205, 0.0600
    'cf45|carbonfiber45|cf40|carbonfiber40', ...
                                  'rect', 2.8900, 0.5145, 0.0565, 2.9610, 0.4705, 0.0615
    'plastic',                    'rect', 3.0070, 0.5060, 0.1365, NaN,    NaN,    NaN      % <-- final measurements not entered
%   'plastic 2',                  'rect', 3.0000, 0.5000, 0.1350, NaN,    NaN,    NaN      % <-- example: own row for one plastic sample
    };

% ---- Published values for the percent differences (section 3) [psi] ----
% Same key matching as the specimens table. NaN = no published value (N/A).
published = {
%   keys       name                        yield    ultimate  rupture  Young's modulus
    'steel',   'AISI 1018 cold drawn',     53700,   63800,    79770,   29.7e6
    'alum',    '6061-T6',                  40000,   45000,    42060,   10.0e6
    };

% ---- Measurement uncertainties ----
dimUnc     = 0.00025;          % +/- in, width/thickness/diameter (calipers)
gaugeUnc   = 0.00025;          % +/- in, gauge length (initial and final)
loadUncPct = 1.0;              % +/- percent of reading, load cell
dispUnc    = 0.001;            % +/- in, crosshead displacement
extUnc     = 5e-5;             % +/- in/in, extensometer

% ---- Strain ----
% 'auto'         = extensometer (Axial Strain) while it was recording, then
%                  Axial Displacement / gauge length after it was removed;
%                  displacement only if the extensometer was never on
% 'strain'       = extensometer only
% 'displacement' = Axial Displacement / gauge length only
strainSource    = 'auto';
toeCompensation = true;        % remove the slack at the start so the elastic line starts at 0

% ---- 0.2% offset / elastic fit ----
offset    = 0.002;             % offset strain (0.002 = 0.2%)
fitRange  = [];                % [] = automatic (steepest straight part of the curve), or
                               % fractions of ultimate stress, e.g. [0.10 0.40]
breakDrop = 0.10;              % a sudden load drop bigger than this fraction of ultimate
                               % is the break; nothing after the rupture is plotted

nErrorBars = 15;               % number of points along each curve that get uncertainty bars
saveFigure = false;            % true = save every figure as .png and .fig (single-material
                               % figures in their material folder, the rest in M003)
%% ============================================================

S = struct('baseFolder', baseFolder, 'testFolder', testFolder, 'datFileName', datFileName, ...
    'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc, 'loadUncPct', loadUncPct, 'dispUnc', dispUnc, ...
    'extUnc', extUnc, 'strainSource', strainSource, 'toe', toeCompensation, ...
    'offset', offset, 'fitRange', fitRange, 'breakDrop', breakDrop);
S.specimens = specimens;
opt = struct('offset', offset, 'nErrorBars', nErrorBars, 'dimUnc', dimUnc, 'gaugeUnc', gaugeUnc);
cache = containers.Map();      % each material is read and analyzed only once
testPath = findSubfolder(resolveBaseFolder(baseFolder), testFolder);

%% ================ 1. Single material ================
if runSingle
    for m = 1:numel(singleMaterials)
        try
            M = loadMaterial(singleMaterials{m}, S, cache);
            fig = plotSingle(M, opt);
            if saveFigure, saveFig(fig, fullfile(M.path, [M.name '_stress_strain'])); end
        catch err
            warning('Single-material figure for "%s" skipped: %s', singleMaterials{m}, err.message);
        end
    end
end

%% ============ 2. Extensometer vs. MTS displacement ============
if runExtVsMTS && ~isempty(extVsMtsMaterial)
    try
        M = loadMaterial(extVsMtsMaterial, S, cache);
        fig = plotExtVsMTS(M, opt);
        if saveFigure && ~isempty(fig)
            saveFig(fig, fullfile(M.path, [M.name '_stress_strain_ext_vs_MTS']));
        end
    catch err
        warning('Extensometer vs. MTS figure skipped: %s', err.message);
    end
end

%% ============ 3. Material comparison (steel vs. aluminum) ============
if runComparison && ~isempty(compareMaterials)
    try
        Ms = cellfun(@(m) loadMaterial(m, S, cache), compareMaterials, 'UniformOutput', false);
        fig = plotComparison(Ms, published, includeExtensometer, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'comparison_stress_strain')); end
    catch err
        warning('Material comparison figure skipped: %s', err.message);
    end
end

%% ============ 4. Carbon fiber samples ============
if runCarbon && ~isempty(carbonMaterials)
    try
        Cs = cellfun(@(m) loadMaterial(m, S, cache), carbonMaterials, 'UniformOutput', false);
        fig = plotCarbon(Cs, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'carbon_fiber_stress_strain')); end
    catch err
        warning('Carbon fiber figure skipped: %s', err.message);
    end
end

%% ============ 5. All materials ============
```

```
if runAll && ~isempty(allMaterials)
    try
        Ms = cellfun(@(m) loadMaterial(m, S, cache), allMaterials, 'UniformOutput', false);
        fig = plotAll(Ms, useExtensometer, opt);
        if saveFigure, saveFig(fig, fullfile(testPath, 'all_materials_stress_strain')); end
    catch err
        warning('All-materials figure skipped: %s', err.message);
    end
end

%% ============ 6. Plastic samples - ultimate stress tables ============
if runPlastic && ~isempty(plasticKey)
    try
        [labels, files] = findPlasticSamples(testPath, plasticKey);
        Ps = cellfun(@(n, f) processSample(n, f, S), labels, files, 'UniformOutput', false);
        fig = plotPlastic(Ps);
        if saveFigure, saveFig(fig, fullfile(testPath, 'plastic_ultimate_stress')); end
    catch err
        warning('Plastic tables skipped: %s', err.message);
    end
end

%% Local function: section 1 - one material with its results table
function fig = plotSingle(M, opt)
    r = M.r;
    p = M.p;
    e = r.e;  s = r.s;  eUnc = r.eUnc;  sUnc = r.sUnc;
    E = r.E;  sY = r.sY;  eY = r.eY;
    iU = p.iU;  iR = numel(s);
    sU = p.sU;  eU = p.eU;  sR = p.sR;  eR = p.eR;

    fig = figure('Name', sprintf('%s stress-strain', M.name), 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [60 60 1500 700]);
    ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
    hold(ax, 'on');

    hCurve = plot(ax, e, s, '-', 'LineWidth', 1.5, 'Color', [0 0.3 0.6]);

    % uncertainty bars on evenly spaced points along the curve
    hErr = drawUncertainty(ax, e, s, eUnc, sUnc, opt.nErrorBars, [0.5 0.5 0.5]);

    % 0.2% offset line, from zero stress up to a bit past yield
    hOff = gobjects(0);
    if isfinite(E) && E > 0
        [eLine, sLine] = offsetLine(r, opt.offset, sU);
        hOff = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
    end

    % key points
    hY = gobjects(0);
    if isfinite(sY)
        hY = plot(ax, eY, sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
            'MarkerFaceColor', [0.47 0.67 0.19], 'MarkerEdgeColor', 'k');
        text(ax, eY, sY, sprintf('  Yield (%.4g, %.5g)', eY, sY), ...
            'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
    end
    hU = plot(ax, eU, sU, '^', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.93 0.69 0.13], 'MarkerEdgeColor', 'k');
    hR = plot(ax, eR, sR, 's', 'MarkerSize', 9, 'LineWidth', 1.5, ...
        'MarkerFaceColor', [0.64 0.08 0.18], 'MarkerEdgeColor', 'k');
    if iU == iR                         % brittle: breaks at the ultimate
        text(ax, eU, sU, sprintf('Ultimate = Rupture (%.4g, %.5g)  ', eU, sU), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
    else
        text(ax, eU, sU, sprintf('Ultimate (%.4g, %.5g)', eU, sU), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
        text(ax, eR, sR, sprintf('Rupture (%.4g, %.5g)  ', eR, sR), ...
            'VerticalAlignment', 'top', 'HorizontalAlignment', 'right');
    end
    hold(ax, 'off');

    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
    ylabel(ax, M.sLabel);
    title(ax, sprintf('%s - Stress vs. Strain', M.name), 'Interpreter', 'none');
    eSpan = max(e) - min(0, min(e));
    sSpan = sU - min(0, min(s));
    if eSpan > 0, xlim(ax, [min(0, min(e)), max(e) + 0.05*eSpan]); end
    if sSpan > 0, ylim(ax, [min(0, min(s)), sU + 0.12*sSpan]); end

    hLeg = [hCurve hErr hOff hY hU hR];
    legTxt = {'Stress-strain curve', 'Uncertainty'};
    if ~isempty(hOff), legTxt{end+1} = sprintf('%g%% offset line', opt.offset*100); end
    if ~isempty(hY),   legTxt{end+1} = 'Yield (0.2% offset)'; end
    legTxt = [legTxt {'Ultimate', 'Rupture'}];
    legend(ax, hLeg, legTxt, 'Location', 'best');

    % results table
    if strcmpi(M.shape, 'round')
        dimRows = {'Initial diameter', pm(M.Wi, opt.dimUnc), 'in'
                   'Final diameter',   pm(M.Wf, opt.dimUnc), 'in'};
    else
        dimRows = {'Initial width',     pm(M.Wi, opt.dimUnc), 'in'
                   'Initial thickness', pm(M.ti, opt.dimUnc), 'in'
                   'Final width',       pm(M.Wf, opt.dimUnc), 'in'
                   'Final thickness',   pm(M.tf, opt.dimUnc), 'in'};
    end
    rows = [
        {'LAB MEASUREMENTS', '', ''}
        {'Initial gauge length', pm(M.L0, opt.gaugeUnc), 'in'}
        {'Final gauge length',   pm(M.Lf, opt.gaugeUnc), 'in'}
        dimRows
        {'RESULTS', '', ''}
        {'Ultimate stress',          pm(p.sU, p.sUu),  M.sUnit}
        {'Strain at ultimate',       pm(p.eU, p.eUu),  M.eUnit}
        {'Rupture stress',           pm(p.sR, p.sRu),  M.sUnit}
        {'Strain at rupture',        pm(p.eR, p.eRu),  M.eUnit}
```

```
        {'Yield stress (0.2%)',      pm(r.sY, r.sYu),  M.sUnit}
        {'Strain at yield',          pm(r.eY, r.eYu),  M.eUnit}
        {'Young''s modulus E',       pm(r.E, r.EUnc),  M.sUnit}
        {'Modulus of toughness',     pm(p.UT, p.UTu),  M.uUnit}
        {'Modulus of resilience',    pm(r.Ur, r.Uru),  M.uUnit}
        ];
    uitable(fig, 'Data', rows, 'ColumnName', {'Quantity', 'Value', 'Units'}, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.58 0.10 0.40 0.82], ...
        'ColumnWidth', {175, 260, 110}, 'FontSize', 11);
    printTable(M.name, {'Quantity', 'Value', 'Units'}, rows);
end

%% Local function: section 2 - extensometer vs. MTS displacement of one material
% Both readings analyzed separately on one plot. The extensometer curve stops
% where the extensometer was removed.
function fig = plotExtVsMTS(M, opt)
    fig = [];
    if isempty(M.rX)
        warning('"%s" has no extensometer data - extensometer vs. MTS figure skipped.', M.name);
        return;
    end
    rX = M.rX;
    rD = M.rD;

    fig = figure('Name', sprintf('%s extensometer vs MTS', M.name), 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [90 90 1500 700]);
    ax = axes(fig, 'Position', [0.05 0.10 0.50 0.82]);
    hold(ax, 'on');
    cX = [0 0.3 0.6];               % extensometer
    cD = [0.85 0.33 0.1];           % MTS displacement
    readings = {rX, cX, 'Extensometer', 'ext.'; rD, cD, 'MTS displacement', 'MTS'};
    sMax = max([rX.s; rD.s]);
    h = gobjects(0);
    leg = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:2
        [rq, cq, nameq, shortq] = readings{q, :};
        h(end+1) = plot(ax, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg{end+1} = nameq;
        h(end+1) = drawUncertainty(ax, rq.e, rq.s, rq.eUnc, rq.sUnc, opt.nErrorBars, 0.5*cq + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, opt.offset, max(rq.s));
            h(end+1) = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg{end+1} = sprintf('%g%% offset line (%s)', opt.offset*100, shortq);
        end
        if isfinite(rq.sY)
            h(end+1) = plot(ax, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg{end+1} = sprintf('Yield (%s)', shortq);
            yLab = placeLabel(rq.sY - 0.04*sMax, labY, sMax);
            labY(end+1) = yLab;
            text(ax, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'BackgroundColor', 'w', 'Margin', 1);
        end
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, sprintf('Axial Strain (%s)', M.eUnit));
    ylabel(ax, M.sLabel);
    title(ax, sprintf('%s - Extensometer vs. MTS Displacement', M.name), 'Interpreter', 'none');
    xlim(ax, [0, 1.05*max([rX.e; rD.e])]);
    ylim(ax, [0, 1.12*sMax]);
    legend(ax, h, leg, 'Location', 'southeast');

    % percent difference, extensometer taken as the accurate value
    rows = {
        ['Young''s modulus E (' M.sUnit ')'],      pm(rX.E,  rX.EUnc), pm(rD.E,  rD.EUnc), pctDiff(rD.E,  rX.E)
        ['Yield stress, 0.2% (' M.sUnit ')'],      pm(rX.sY, rX.sYu),  pm(rD.sY, rD.sYu),  pctDiff(rD.sY, rX.sY)
        ['Yield strain (' M.eUnit ')'],            pm(rX.eY, rX.eYu),  pm(rD.eY, rD.eYu),  pctDiff(rD.eY, rX.eY)
        ['Modulus of resilience (' M.uUnit ')'],   pm(rX.Ur, rX.Uru),  pm(rD.Ur, rD.Uru),  ''
        };
    cols = {'Quantity', 'Extensometer', 'MTS displacement', '% difference'};
    uitable(fig, 'Data', rows, 'ColumnName', cols, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.57 0.40 0.42 0.30], ...
        'ColumnWidth', {215, 165, 165, 95}, 'FontSize', 11);
    printTable(sprintf('%s - extensometer vs. MTS displacement', M.name), cols, rows);
end

%% Local function: section 3 - materials compared on one plot
function fig = plotComparison(Ms, published, includeExtensometer, opt)
    sUnit = Ms{1}.sUnit;
    uUnit = Ms{1}.uUnit;

    % readings to plot: MTS displacement, plus the extensometer if asked for
    % and the file has it
    useExt = cellfun(@(c) includeExtensometer && ~isempty(c.rX), Ms);
    R = cell(0, 4);                 % {analyzed curve, legend name, short label, material (0 = extensometer)}
    for m = 1:numel(Ms)
        if useExt(m)
            R(end+1, :) = {Ms{m}.rX, [Ms{m}.name ' - extensometer'],     [Ms{m}.name ' ext.'], 0};
        end
        R(end+1, :) = {Ms{m}.rD, [Ms{m}.name ' - MTS displacement'], [Ms{m}.name ' MTS'], m};
    end

    % true stress-strain up to the ultimate (valid until necking starts)
    for m = 1:numel(Ms)
        iUm = Ms{m}.pD.iU;
        eEng = Ms{m}.rD.e(1:iUm);
        Ms{m}.eT = log(1 + eEng);
        Ms{m}.sT = Ms{m}.rD.s(1:iUm) .* (1 + eEng);
    end
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];

    fig = figure('Name', 'Stress-strain comparison', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [40 40 1650 820]);
```

```
    ax = axes(fig, 'Position', [0.055 0.08 0.415 0.85]);
    hold(ax, 'on');
    sMax = max([cellfun(@(c) max(c.s), R(:, 1)); cellfun(@(c) max([c.sT; c.sTR]), Ms(:))]);
    eMax = max([cellfun(@(c) max(c.e), R(:, 1)); cellfun(@(c) max([c.eT; c.eTR]), Ms(:))]);
    h = gobjects(0);
    leg = {};
    labY = [];                      % heights of the yield labels placed so far
    for q = 1:size(R, 1)
        [rq, nameq, shortq, mq] = R{q, :};
        cq = colors(mod(q - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, rq.e, rq.s, '-', 'LineWidth', 1.5, 'Color', cq);
        leg{end+1} = nameq;
        h(end+1) = drawUncertainty(ax, rq.e, rq.s, rq.eUnc, rq.sUnc, opt.nErrorBars, 0.5*cq + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', shortq);
        if rq.E > 0
            [eLine, sLine] = offsetLine(rq, opt.offset, max(rq.s));
            h(end+1) = plot(ax, eLine, sLine, '--', 'LineWidth', 1.2, 'Color', cq);
            leg{end+1} = sprintf('%g%% offset line (%s)', opt.offset*100, shortq);
        end
        if isfinite(rq.sY)
            h(end+1) = plot(ax, rq.eY, rq.sY, 'o', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
            leg{end+1} = sprintf('Yield (%s)', shortq);
            yLab = placeLabel(rq.sY - 0.04*sMax, labY, sMax);
            labY(end+1) = yLab;
            text(ax, rq.eY, yLab, sprintf('   Yield, %s (%.4g, %.5g)', shortq, rq.eY, rq.sY), ...
                'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', 'Color', cq, ...
                'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
        end
        if mq > 0                   % true curve and true rupture point, same color as the MTS curve
            Mm = Ms{mq};
            h(end+1) = plot(ax, Mm.eT, Mm.sT, '-.', 'LineWidth', 1.5, 'Color', cq);
            leg{end+1} = sprintf('True stress-strain (%s)', Mm.name);
            if isfinite(Mm.sTR) && isfinite(Mm.eTR)
                h(end+1) = plot(ax, Mm.eTR, Mm.sTR, 'd', 'MarkerSize', 9, 'LineWidth', 1.5, ...
                    'MarkerFaceColor', cq, 'MarkerEdgeColor', 'k');
                leg{end+1} = sprintf('True rupture (%s)', Mm.name);
                text(ax, Mm.eTR - 0.012*eMax, Mm.sTR, sprintf('True rupture, %s (%.4g, %.0f)', Mm.name, Mm.eTR, Mm.sTR), ...
                    'VerticalAlignment', 'middle', 'HorizontalAlignment', 'right', 'Color', cq, ...
                    'Interpreter', 'none', 'BackgroundColor', 'w', 'Margin', 1);
            end
        end
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Engineering / True Strain (in/in)');
    ylabel(ax, strrep(Ms{1}.sLabel, 'Engineering Stress', 'Engineering / True Stress'));
    if any(useExt), ttl = 'Stress vs. Strain - '; else, ttl = 'Stress vs. Strain (MTS displacement) - '; end
    title(ax, [ttl strjoin(cellfun(@(c) c.name, Ms, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax, [0, 1.05*eMax]);
    ylim(ax, [0, 1.12*sMax]);
    legend(ax, h, leg, 'Location', 'southeast', 'Interpreter', 'none');

    % ---- Table 1: each reading ----
    cols1 = {'Quantity'};
    T1 = {['Young''s modulus E (' sUnit ')']; ['Yield stress, 0.2% (' sUnit ')']; ...
          'Yield strain (in/in)'; ['Modulus of resilience (' uUnit ')']};
    for m = 1:numel(Ms)
        rD = Ms{m}.rD;
        colD = {pm(rD.E, rD.EUnc); pm(rD.sY, rD.sYu); pm(rD.eY, rD.eYu); pm(rD.Ur, rD.Uru)};
        if useExt(m)
            rX = Ms{m}.rX;
            colX = {pm(rX.E, rX.EUnc); pm(rX.sY, rX.sYu); pm(rX.eY, rX.eYu); pm(rX.Ur, rX.Uru)};
            colP = {pctDiff(rD.E, rX.E); pctDiff(rD.sY, rX.sY); pctDiff(rD.eY, rX.eY); ''};
            cols1 = [cols1, {[Ms{m}.name ' - extensometer'], [Ms{m}.name ' - MTS'], '% diff (MTS vs ext.)'}];
            T1 = [T1, colX, colD, colP];
        else
            cols1 = [cols1, {[Ms{m}.name ' - MTS']}];
            T1 = [T1, colD];
        end
    end

    % ---- Table 2: materials and published values ----
    % ultimate, rupture and toughness from the MTS displacement curve (the
    % extensometer comes off before the ultimate); yield and E from the
    % extensometer when it is plotted, MTS displacement otherwise
    labels2 = {['Ultimate stress (' sUnit ')']; 'Strain at ultimate (in/in)'; ...
               ['Rupture stress (' sUnit ')']; 'Strain at rupture (in/in)'; ...
               ['True rupture stress (' sUnit ')']; 'True rupture strain (in/in)'; ...
               ['Yield stress, 0.2% (' sUnit ')']; ['Young''s modulus E (' sUnit ')']; ...
               ['Modulus of toughness (' uUnit ')']};
    pubCol = [2 0 3 0 0 0 1 4 0];   % column of "published" each row compares to (0 = none)
    cols2 = {'Quantity'};
    T2 = labels2;
    pubNames = {};
    for m = 1:numel(Ms)
        Mm = Ms{m};
        pD = Mm.pD;
        if useExt(m), best = Mm.rX; else, best = Mm.rD; end
        vals = [pD.sU pD.sUu; pD.eU pD.eUu; pD.sR pD.sRu; pD.eR pD.eRu; ...
                Mm.sTR Mm.sTRu; Mm.eTR Mm.eTRu; best.sY best.sYu; best.E best.EUnc; pD.UT pD.UTu];
        [pub, pubName] = lookupPublished(published, Mm.name);
        if ~isempty(pubName), pubNames{end+1} = sprintf('%s = %s', Mm.name, pubName); end
        colV = cell(numel(labels2), 1);
        colPub = repmat({''}, numel(labels2), 1);
        colPd  = repmat({''}, numel(labels2), 1);
        for k = 1:numel(labels2)
            colV{k} = pm(vals(k, 1), vals(k, 2));
            if pubCol(k) > 0
                ref = pub(pubCol(k));
                if isfinite(ref)
                    colPub{k} = sprintf('%g', ref);
                    colPd{k}  = pctDiff(vals(k, 1), ref);
                else
                    colPub{k} = 'N/A';
                end
```

```
            end
        end
        cols2 = [cols2, {Mm.name, 'Published', '% diff'}];
        T2 = [T2, colV, colPub, colPd];
    end

    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.93 0.50 0.03], ...
        'String', 'Table 1 - Each reading', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T1, 'ColumnName', cols1, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.70 0.50 0.23], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({150}, 1, numel(cols1) - 1)]);
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.64 0.50 0.03], ...
        'String', 'Table 2 - Materials and published values', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T2, 'ColumnName', cols2, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.49 0.22 0.50 0.42], 'FontSize', 10, ...
        'ColumnWidth', [{210}, repmat({140, 80, 65}, 1, numel(Ms))]);
    if any(useExt)
        strainNote = ['Ultimate, rupture and toughness use the MTS displacement strain; yield and E use ' ...
                      'the extensometer where it is plotted.'];
    else
        strainNote = 'All strains are from the MTS crosshead displacement.';
    end
    note = {strainNote, ...
            ['True curve (dash-dot): stress x (1 + strain) and ln(1 + strain), up to the ultimate (valid until necking). ' ...
             'True rupture: rupture load / final area, ln(initial area / final area).'], ...
            'Percent difference = |measured - reference| / reference x 100 (reference: extensometer or published).', ...
            ['Published: ' strjoin(pubNames, '; ')]};
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.49 0.06 0.50 0.14], ...
        'String', note, 'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Table 1 - Each reading', cols1, T1);
    printTable('Table 2 - Materials and published values', cols2, T2);
end

%% Local function: section 4 - carbon fiber samples with their results table
function fig = plotCarbon(Cs, opt)
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6];
    fig = figure('Name', 'Carbon fiber stress-strain', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [70 70 1500 700]);
    ax = axes(fig, 'Position', [0.06 0.10 0.50 0.82]);
    hold(ax, 'on');
    h = gobjects(0);
    leg = {};
    for m = 1:numel(Cs)
        r  = Cs{m}.r;
        cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
        leg{end+1} = Cs{m}.name;
        h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, opt.nErrorBars, 0.5*cm + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', Cs{m}.name);
    end
    hold(ax, 'off');
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Axial Strain (in/in)');
    ylabel(ax, Cs{1}.sLabel);
    title(ax, ['Stress vs. Strain - ' strjoin(cellfun(@(c) c.name, Cs, 'UniformOutput', false), ' vs. ')], ...
        'Interpreter', 'none');
    xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.r.e), Cs))]);
    ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.r.s), Cs))]);
    legend(ax, h, leg, 'Location', 'southeast', 'Interpreter', 'none');

    % table: ultimate, rupture and modulus of toughness of each sample
    sUnit = Cs{1}.sUnit;
    T = {['Ultimate stress (' sUnit ')']; 'Strain at ultimate (in/in)'; ...
         ['Rupture stress (' sUnit ')']; 'Strain at rupture (in/in)'; ...
         ['Modulus of toughness (' Cs{1}.uUnit ')']};
    cols = {'Quantity'};
    for m = 1:numel(Cs)
        p = Cs{m}.p;
        T = [T, {pm(p.sU, p.sUu); pm(p.eU, p.eUu); pm(p.sR, p.sRu); pm(p.eR, p.eRu); pm(p.UT, p.UTu)}];
        cols = [cols, {Cs{m}.name}];
    end
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.80 0.38 0.04], ...
        'String', 'Carbon fiber results', 'FontWeight', 'bold', 'FontSize', 11, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', T, 'ColumnName', cols, 'RowName', [], 'Units', 'normalized', ...
        'Position', [0.60 0.52 0.38 0.28], 'FontSize', 11, ...
        'ColumnWidth', [{220}, repmat({170}, 1, numel(Cs))]);
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.60 0.38 0.38 0.12], ...
        'String', {'Strain is from the MTS crosshead displacement.', ...
                   'A sample that breaks at its highest load has rupture = ultimate.'}, ...
        'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    printTable('Carbon fiber results', cols, T);
end

%% Local function: section 5 - all materials on one plot
function fig = plotAll(Ms, useExtensometer, opt)
    colors = [0 0.3 0.6; 0.85 0.33 0.1; 0.2 0.6 0.2; 0.5 0.2 0.6; 0.6 0.5 0.1; 0.3 0.3 0.3];
    fig = figure('Name', 'All materials stress-strain', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [70 70 1200 750]);
    ax = axes(fig);
    hold(ax, 'on');
    h = gobjects(0);
    leg = {};
    curves = cell(size(Ms));
    for m = 1:numel(Ms)
        if useExtensometer, r = Ms{m}.r; else, r = Ms{m}.rD; end
        curves{m} = r;
        cm = colors(mod(m - 1, size(colors, 1)) + 1, :);
        h(end+1) = plot(ax, r.e, r.s, '-', 'LineWidth', 1.5, 'Color', cm);
        leg{end+1} = Ms{m}.name;
        h(end+1) = drawUncertainty(ax, r.e, r.s, r.eUnc, r.sUnc, opt.nErrorBars, 0.5*cm + 0.5);
        leg{end+1} = sprintf('Uncertainty (%s)', Ms{m}.name);
    end
    hold(ax, 'off');
```

```
    grid(ax, 'on');
    box(ax, 'on');
    xlabel(ax, 'Axial Strain (in/in)');
    ylabel(ax, Ms{1}.sLabel);
    if useExtensometer, ttl = 'Stress vs. Strain - All Materials';
    else, ttl = 'Stress vs. Strain - All Materials (MTS displacement)'; end
    title(ax, ttl);
    xlim(ax, [0, 1.05*max(cellfun(@(c) max(c.e), curves))]);
    ylim(ax, [0, 1.12*max(cellfun(@(c) max(c.s), curves))]);
    legend(ax, h, leg, 'Location', 'eastoutside', 'Interpreter', 'none');
end

%% Local function: section 6 - every plastic sample (.dat file) found
% Every folder in M003 whose name contains plasticKey (case, spaces and
% punctuation ignored); every .dat file inside it (subfolders included) is
% one sample, so it works whether each sample has its own folder (Plastic 1,
% Plastic 2, ...) or several .dat files share one folder.
function [labels, files] = findPlasticSamples(testPath, plasticKey)
    norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
    d = dir(testPath);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    d = d(contains(cellfun(norm, {d.name}, 'UniformOutput', false), norm(plasticKey)));
    if isempty(d), error('No folder in %s has "%s" in its name.', testPath, plasticKey); end
    [~, order] = sort(naturalKey({d.name}));
    d = d(order);

    labels = {};
    files  = {};
    for k = 1:numel(d)
        folder = fullfile(testPath, d(k).name);
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
        if isempty(f)
            warning('No .dat file in %s - skipped.', folder);
            continue;
        end
        [~, order] = sort(naturalKey(fullfile({f.folder}, {f.name})));
        f = f(order);
        for j = 1:numel(f)
            if numel(f) == 1
                labels{end+1} = d(k).name;
            else
                [~, fn] = fileparts(f(j).name);
                labels{end+1} = sprintf('%s - %s', d(k).name, fn);
            end
            files{end+1} = fullfile(f(j).folder, f(j).name);
        end
    end
    fprintf('Found %d plastic samples.\n', numel(files));
end

%% Local function: section 6 - ultimate stress tables of the plastic samples
% Average uncertainty = sqrt(sum(dsU.^2)) / n (measurement uncertainty);
% standard deviation = sample-to-sample scatter.
function fig = plotPlastic(Ms)
    n   = numel(Ms);
    sU  = cellfun(@(c) c.p.sU,  Ms);
    sUu = cellfun(@(c) c.p.sUu, Ms);
    sUnit = Ms{1}.sUnit;

    [sMin, iMin] = min(sU);
    [sMax, iMax] = max(sU);
    sAvg  = mean(sU);
    sAvgU = sqrt(sum(sUu.^2)) / n;          % measurement uncertainty of the average
    if n > 1, sStd = std(sU); else, sStd = NaN; end

    summary = {
        'Number of samples',            sprintf('%d', n),        '',     ''
        'Average ultimate stress',      pm(sAvg, sAvgU),         sUnit,  ''
        'Standard deviation',           pm(sStd, NaN),           sUnit,  'sample-to-sample scatter'
        'Minimum ultimate stress',      pm(sMin, sUu(iMin)),     sUnit,  Ms{iMin}.name
        'Maximum ultimate stress',      pm(sMax, sUu(iMax)),     sUnit,  Ms{iMax}.name
        };
    perSample = [cellfun(@(c) c.name, Ms(:), 'UniformOutput', false), ...
                 arrayfun(@(k) pm(sU(k), sUu(k)), (1:n)', 'UniformOutput', false)];

    fig = figure('Name', 'Plastic ultimate stress', 'NumberTitle', 'off', ...
        'Color', 'w', 'Position', [70 70 1100 650]);
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.03 0.90 0.50 0.05], ...
        'String', 'Plastic samples - ultimate stress summary', 'FontWeight', 'bold', 'FontSize', 12, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', summary, 'ColumnName', {'Quantity', 'Value', 'Units', 'Sample'}, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.03 0.55 0.55 0.35], 'FontSize', 11, ...
        'ColumnWidth', {190, 150, 60, 190});
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.62 0.90 0.35 0.05], ...
        'String', 'Ultimate stress of each sample', 'FontWeight', 'bold', 'FontSize', 12, ...
        'BackgroundColor', 'w', 'HorizontalAlignment', 'left');
    uitable(fig, 'Data', perSample, 'ColumnName', {'Sample', ['Ultimate stress (' sUnit ')']}, ...
        'RowName', [], 'Units', 'normalized', 'Position', [0.62 0.05 0.35 0.85], 'FontSize', 11, ...
        'ColumnWidth', {170, 170});
    uicontrol(fig, 'Style', 'text', 'Units', 'normalized', 'Position', [0.03 0.30 0.55 0.20], ...
        'String', {'Ultimate stress = highest engineering stress (load / initial area) before rupture.', ...
                   'Average uncertainty = sqrt(sum of squared sample uncertainties) / n.', ...
                   'Standard deviation = scatter between the samples.'}, ...
        'FontSize', 9, 'BackgroundColor', 'w', 'HorizontalAlignment', 'left');

    printTable('Plastic samples - ultimate stress summary', {'Quantity', 'Value', 'Units', 'Sample'}, summary);
    printTable('Ultimate stress of each sample', {'Sample', ['Ultimate stress (' sUnit ')']}, perSample);
end

%% Local function: material from the cache, or read and analyze it once
function M = loadMaterial(name, S, cache)
    path = findSubfolder(findSubfolder(resolveBaseFolder(S.baseFolder), S.testFolder), name);
    if isKey(cache, path)
        M = cache(path);
    else
        M = processMaterial(name, S);
        cache(path) = M;
```

```
    end
end

%% Local function: save a figure as .png and .fig
function saveFig(fig, outBase)
    if isempty(fig), return; end
    saveas(fig, [outBase '.png']);
    savefig(fig, [outBase '.fig']);
    fprintf('Saved %s.png and .fig\n', outBase);
end

%% Local function: label height just below a point, stepped further down
% only if it would overlap a label already placed
function yLab = placeLabel(yLab, labY, sMax)
    while any(abs(labY - yLab) < 0.06*sMax)
        yLab = yLab - 0.06*sMax;
    end
end

%% Local function: find, read and analyze one material
% Returns a struct with the specimen measurements, units, and three
% analyzed curves: r (best strain: extensometer, then displacement),
% rD (MTS displacement only) and rX (extensometer only, [] if none), plus
% ultimate/rupture/toughness (p for r, pD for rD) and the true rupture point.
function M = processMaterial(material, S)
    base     = resolveBaseFolder(S.baseFolder);
    testPath = findSubfolder(base, S.testFolder);
    path     = findSubfolder(testPath, material);
    [~, name] = fileparts(path);
    M = processSample(name, findDatFile(path, S.datFileName), S);
    M.path = path;
end

%% Local function: read and analyze one .dat file
% name is the label for the sample (also used to find its row in the
% specimens table).
function M = processSample(name, datFile, S)
    M.name = name;
    M.datFile = datFile;
    fprintf('\n%s\n  DAT file: %s\n', M.name, M.datFile);

    % specimen geometry
    [M.shape, M.L0, M.Wi, M.ti, M.Lf, M.Wf, M.tf] = lookupSpecimen(S.specimens, M.name);
    L0 = M.L0;
    [M.A,  M.relA ] = sectionArea(M.shape, M.Wi, M.ti, S.dimUnc);
    [M.Af, M.relAf] = sectionArea(M.shape, M.Wf, M.tf, S.dimUnc);
    haveArea = isfinite(M.A) && M.A > 0;
    relA = M.relA;
    if ~haveArea
        relA = 0;
        fprintf('  No specimen dimensions - plotting axial force instead of stress.\n');
    end
    if ~isfinite(L0) || L0 <= 0
        error('Enter the gauge length for "%s" in the specimens table.', M.name);
    end

    % read the data
    [data, names, units] = readMtsDat(M.datFile);
    iF = findCol(names, 'Axial Force');
    iD = findCol(names, 'Axial Displacement');
    iE = find(contains(names, 'Strain', 'IgnoreCase', true), 1);   % extensometer, may be missing
    fUnit = units{iF};
    good = isfinite(data(:, iF)) & isfinite(data(:, iD));
    if ~isempty(iE), good = good & isfinite(data(:, iE)); end
    data = data(good, :);
    if size(data, 1) < 10, error('Not enough data points in %s.', M.datFile); end
    F  = data(:, iF);
    eD = data(:, iD) / L0;
    eD = eD - eD(1);

    % strain
    e = eD;
    eExt = [];                      % extensometer strain, in/in (empty if none)
    isExt = false(size(e));         % which points come from the extensometer
    if ~strcmpi(S.strainSource, 'displacement') && ~isempty(iE)
        eX = data(:, iE);
        if contains(units{iE}, '%'), eX = eX/100; end    % percent -> in/in
        eX = eX - eX(1);
        [~, iRem] = max(eX);                             % last reading before the extensometer came off
        if strcmpi(S.strainSource, 'strain')
            e = eX;
            eExt = eX;
            isExt(:) = true;
        elseif eX(iRem) - min(eX(1:iRem)) >= 1e-3        % a real test goes well past 0.1% strain
            % after removal, continue with displacement, scaled to match the
            % extensometer over the second half of the time it was on
            iA = max(1, round(iRem/2));
            k  = (eX(iRem) - eX(iA)) / (eD(iRem) - eD(iA));
            if ~isfinite(k) || k <= 0, k = 1; end
            e = eX;
            e(iRem+1:end) = eX(iRem) + k*(eD(iRem+1:end) - eD(iRem));
            eExt = eX;
            isExt(1:iRem) = true;
            if iRem < numel(e) - 5
                fprintf('  Extensometer removed at strain %.4g; continuing with displacement.\n', eX(iRem));
            end
        else
            fprintf(['  Axial Strain column is flat - extensometer was not recording.\n' ...
                '  Using Axial Displacement / gauge length (%g in) for strain.\n'], L0);
        end
    end
    M.eUnit = 'in/in';
    eDu  = sqrt((S.dispUnc/L0)^2 + (eD*S.gaugeUnc/L0).^2);
    eUnc = eDu;
    eUnc(isExt) = S.extUnc;

    % stress
    if haveArea
```

```
        s = F / M.A;
        switch lower(fUnit)
            case 'lbf', M.sUnit = 'psi';   M.uUnit = 'in-lbf/in^3';
            case 'kip', M.sUnit = 'ksi';   M.uUnit = 'in-kip/in^3';
            case 'n',   M.sUnit = 'MPa';   M.uUnit = 'MJ/m^3';     % area in mm^2
            otherwise,  M.sUnit = [fUnit '/area']; M.uUnit = M.sUnit;
        end
        M.sLabel = sprintf('Engineering Stress (%s)', M.sUnit);
    else
        s = F;
        M.sUnit  = fUnit;
        M.uUnit  = [fUnit '*in/in'];
        M.sLabel = sprintf('Stress - Axial Force (%s)', M.sUnit);
    end
    relS = sqrt((S.loadUncPct/100)^2 + relA^2);   % relative stress uncertainty
    sUnc = abs(s) * relS;

    % break: biggest sudden load drop after the ultimate (one sample or a
    % few; the load cell may not read 0 afterwards, it can even go negative)
    [sU, iU] = max(s);
    iR = numel(s);
    d  = diff(s(iU:end));           % d(j) = s(iU+j) - s(iU+j-1)
    if ~isempty(d)
        [dMax, j] = min(d);
        if dMax < 0
            a = j;
            z = j;
            while a > 1 && d(a-1) < 0.3*dMax, a = a - 1; end
            while z < numel(d) && d(z+1) < 0.3*dMax, z = z + 1; end
            if -sum(d(a:z)) > S.breakDrop*sU
                iR = iU + a - 1;    % last point before the drop
            end
        end
    end
    if iR == numel(s)
        fprintf('  No break found in the data; rupture is the last data point.\n');
    end

    % rupture = start of the final load drop: ductile samples can tear for
    % a while before the last snap, so walk back from the snap while the
    % curve is still falling steeply (> 5% of E)
    E0 = fitElastic(e(1:iR), s(1:iR), S.fitRange);
    if E0 > 0
        w = max(2, round(0.01*numel(s)));
        j = iR;
        while j - w >= iU && e(j) > e(j-w) && (s(j-w) - s(j))/(e(j) - e(j-w)) > 0.05*E0
            j = j - 1;
        end
        if j < iR
            lo = max(iU, j - w);
            [~, k] = max(s(lo:j));
            iR = lo + k - 1;
        end
    end

    % analyzed curves
    P = struct('fitRange', S.fitRange, 'offset', S.offset, 'toe', S.toe, 'relS', relS, ...
        'extUnc', S.extUnc, 'dispUnc', S.dispUnc, 'gaugeUnc', S.gaugeUnc, 'L0', L0);
    M.r  = analyzeCurve(e(1:iR), s(1:iR), eUnc(1:iR), sUnc(1:iR), isExt(1:iR), P);
    M.p  = curveProps(M.r, relS);
    M.rD = analyzeCurve(eD(1:iR), s(1:iR), eDu(1:iR), sUnc(1:iR), false(iR, 1), P);
    M.pD = curveProps(M.rD, relS);
    M.rX = [];
    if ~isempty(eExt)
        nX = min(find(isExt, 1, 'last'), iR);
        M.rX = analyzeCurve(eExt(1:nX), s(1:nX), S.extUnc*ones(nX, 1), sUnc(1:nX), true(nX, 1), P);
    end
    if ~(M.r.E > 0)
        warning('Elastic slope is not positive (E = %.3g); 0.2%% offset yield cannot be found.', M.r.E);
    elseif isnan(M.r.sY)
        fprintf('  The curve never crosses the 0.2%% offset line (brittle sample); no yield point.\n');
    end

    % true rupture: rupture load over the final area, strain from the area change
    M.sTR  = F(iR) / M.Af;
    M.sTRu = abs(M.sTR) * sqrt((S.loadUncPct/100)^2 + M.relAf^2);
    M.eTR  = log(M.A / M.Af);
    M.eTRu = sqrt(M.relA^2 + M.relAf^2);
end

%% Local function: ultimate, rupture and modulus of toughness of a curve
function p = curveProps(r, relS)
    [p.sU, p.iU] = max(r.s);
    p.eU  = r.e(p.iU);    p.sUu = r.sUnc(p.iU);   p.eUu = r.eUnc(p.iU);
    p.sR  = r.s(end);     p.sRu = r.sUnc(end);
    p.eR  = r.e(end);     p.eRu = r.eUnc(end);
    p.UT  = trapz(r.e, r.s);
    p.UTu = abs(p.UT) * sqrt(relS^2 + (p.eRu/p.eR)^2);
end

%% Local function: elastic fit, toe compensation and 0.2% offset yield
% Returns a struct with the (toe-compensated) curve and its uncertainties,
% E +/- dE, the yield point +/- uncertainties and the modulus of resilience.
function r = analyzeCurve(e, s, eUnc, sUnc, isExt, P)
    [E, b, idx] = fitElastic(e, s, P.fitRange);
    ef  = e(idx);
    sf  = s(idx);
    res = sf - (E*ef + b);
    seE = sqrt(sum(res.^2)/max(1, numel(ef) - 2)) / sqrt(sum((ef - mean(ef)).^2));   % std. error of slope
    span = max(ef) - min(ef);
    if isExt(idx(1))
        relEfit = sqrt(2)*P.extUnc/span;
    else
        relEfit = sqrt((sqrt(2)*P.dispUnc/P.L0/span)^2 + (P.gaugeUnc/P.L0)^2);
    end
    r.E    = E;
    r.EUnc = abs(E) * sqrt((seE/E)^2 + P.relS^2 + relEfit^2);
```

```
    % toe compensation: shift so the elastic line passes through 0
    kStart = idx(1);
    if P.toe && E > 0
        e0   = -b/E;
        e    = [0; e(idx(1):end) - e0];
        s    = [0; s(idx(1):end)];
        sUnc = [0; sUnc(idx(1):end)];
        eUnc = [0; eUnc(idx(1):end)];
        b    = 0;
        kStart = 2;
    end
    r.e = e;  r.s = s;  r.eUnc = eUnc;  r.sUnc = sUnc;  r.b = b;

    % offset line: s = E*(e - offset) + b; yield is where the curve first
    % drops below it
    r.sY = NaN;  r.eY = NaN;  r.sYu = NaN;  r.eYu = NaN;
    if isfinite(E) && E > 0
        g = s - (E*(e - P.offset) + b);
        k = find(g(kStart:end) <= 0, 1) + kStart - 1;
        if ~isempty(k) && k > 1
            t     = g(k-1) / (g(k-1) - g(k));
            r.eY  = e(k-1) + t*(e(k) - e(k-1));
            r.sY  = s(k-1) + t*(s(k) - s(k-1));
            r.sYu = abs(r.sY) * P.relS;
            r.eYu = eUnc(k);
        end
    end

    % modulus of resilience: sY^2 / 2E
    r.Ur  = r.sY^2 / (2*E);
    r.Uru = abs(r.Ur) * sqrt((2*r.sYu/r.sY)^2 + (r.EUnc/E)^2);
end

%% Local function: 0.2% offset line from zero stress to a bit past yield
function [eLine, sLine] = offsetLine(r, offset, sMax)
    if isfinite(r.sY), sTop = 1.15*r.sY; else, sTop = sMax; end
    sLine = [0 sTop];
    eLine = (sLine - r.b)/r.E + offset;
end

%% Local function: uncertainty bars on evenly spaced points along a curve
function h = drawUncertainty(ax, e, s, eUnc, sUnc, n, color)
    ib = unique(round(linspace(1, numel(e), min(n, numel(e)))));
    h  = errorbar(ax, e(ib), s(ib), sUnc(ib), sUnc(ib), eUnc(ib), eUnc(ib), 'LineStyle', 'none', ...
        'Color', color, 'CapSize', 4);
end

%% Local function: percent difference text, |x - ref| / ref * 100
function str = pctDiff(x, ref)
    if isfinite(x) && isfinite(ref) && ref ~= 0
        str = sprintf('%.1f%%', abs(x - ref)/abs(ref)*100);
    else
        str = 'N/A';
    end
end

%% Local function: row of a key table whose key appears in the folder name
% Case, spaces and punctuation are ignored; '|' separates alternative keys.
% A key ending in a number only matches that whole number ('plastic1' does
% not match 'plastic12'). Returns 0 if no key matches.
function best = matchKey(keys, folderName)
    norm = @(x) regexprep(lower(x), '[^a-z0-9|]', '');
    folder = norm(folderName);
    best = 0;
    bestLen = 0;
    for r = 1:numel(keys)
        for key = strsplit(norm(keys{r}), '|')
            if ~isempty(key{1}) && keyFound(folder, key{1}) && numel(key{1}) > bestLen
                best = r;
                bestLen = numel(key{1});
            end
        end
    end
end

%% Local function: key appears in text, not followed by a digit if it ends in one
function tf = keyFound(text, key)
    tf = false;
    for i = strfind(text, key)
        after = i + numel(key);
        if ~isstrprop(key(end), 'digit') || after > numel(text) || ~isstrprop(text(after), 'digit')
            tf = true;
            return;
        end
    end
end

%% Local function: sort key that puts "Plastic 2" before "Plastic 10"
function keys = naturalKey(names)
    keys = cell(size(names));
    for i = 1:numel(names)
        [num, txt] = regexp(lower(names{i}), '\d+', 'match', 'split');
        k = txt{1};
        for j = 1:numel(num)
            k = [k sprintf('%010d', str2double(num{j})) txt{j+1}];
        end
        keys{i} = k;
    end
end

%% Local function: specimen measurements for a material folder
function [shape, Li, Wi, ti, Lf, Wf, tf] = lookupSpecimen(specimens, matName)
    best = matchKey(specimens(:, 1), matName);
    if best == 0
        error(['No row in the specimens table matches the folder "%s".\n' ...
            'Add a row whose key is part of that folder name.'], matName);
    end
    row = specimens(best, :);
```

```
    [shape, Li, Wi, ti, Lf, Wf, tf] = row{2:8};
end

%% Local function: published [yield ultimate rupture E] for a material folder
function [vals, name] = lookupPublished(published, matName)
    vals = nan(1, 4);
    name = '';
    best = matchKey(published(:, 1), matName);
    if best > 0
        name = published{best, 2};
        vals = [published{best, 3:6}];
    end
end

%% Local function: cross-section area and its relative uncertainty
function [A, relA] = sectionArea(shape, w, t, dimUnc)
    if strcmpi(shape, 'round')
        A    = pi*w^2/4;
        relA = 2*dimUnc/w;
    else
        A    = w*t;
        relA = sqrt((dimUnc/w)^2 + (dimUnc/t)^2);
    end
end

%% Local function: elastic slope from a straight-line fit
% fitRange = [] tries 25%-wide stress windows starting at 5%..50% of the
% ultimate and keeps the steepest one (skips the slack toe at the start and
% the curved part after yield).
function [E, b, idx] = fitElastic(e, s, fitRange)
    [sU, iU] = max(s);
    if isempty(fitRange)
        starts = 0.05:0.05:0.50;
        width  = 0.25;
    else
        starts = fitRange(1);
        width  = fitRange(2) - fitRange(1);
    end
    E = NaN;
    b = NaN;
    idx = [];
    for a = starts
        m = find(s(1:iU) >= a*sU & s(1:iU) <= (a + width)*sU);
        if numel(m) < 3, continue; end
        p = polyfit(e(m), s(m), 1);
        if isempty(idx) || p(1) > E
            E = p(1);
            b = p(2);
            idx = m;
        end
    end
    if isempty(idx)
        error('Not enough points to fit the elastic slope. Set fitRange, e.g. [0.10 0.40].');
    end
end

%% Local function: use baseFolder, or find "MAE Solids datasets" near here
function folder = resolveBaseFolder(folder)
    if isfolder(folder), return; end
    [~, name, ext] = fileparts(folder);
    name = [name ext];
    candidates = {fullfile(pwd, name), ...
                  fullfile(fileparts(mfilename('fullpath')), name), ...
                  pwd};
    for c = candidates
        if isfolder(c{1}) && endsWith(c{1}, name, 'IgnoreCase', true)
            folder = c{1};
            return;
        end
    end
    error('Base folder not found: %s\nSet baseFolder at the top of the script.', folder);
end

%% Local function: subfolder by name
% Tries each '|'-separated alternative: exact match, then partial match,
% then partial match ignoring case, spaces and punctuation.
function folder = findSubfolder(parent, name)
    d = dir(parent);
    d = d([d.isdir] & ~startsWith({d.name}, '.'));
    names = {d.name};
    norm = @(x) regexprep(lower(x), '[^a-z0-9]', '');
    normNames = cellfun(norm, names, 'UniformOutput', false);
    hit = false(size(names));
    for alt = strtrim(strsplit(name, '|'))
        a = alt{1};
        if isempty(a), continue; end
        hit = strcmpi(names, a);
        if ~any(hit), hit = contains(names, a, 'IgnoreCase', true); end
        if ~any(hit), hit = contains(normNames, norm(a)); end
        if any(hit), break; end
    end
    if ~any(hit)
        error('No folder matching "%s" in %s\nAvailable folders: %s', ...
            name, parent, strjoin(names, ', '));
    end
    if nnz(hit) > 1
        error('"%s" matches more than one folder in %s: %s\nUse the full folder name.', ...
            name, parent, strjoin(names(hit), ', '));
    end
    folder = fullfile(parent, names{hit});
end

%% Local function: .dat file anywhere inside the material folder
function file = findDatFile(folder, name)
    if isempty(name)
        f = dir(fullfile(folder, '**', '*'));
        f = f(~[f.isdir] & endsWith({f.name}, '.dat', 'IgnoreCase', true));
    else
```

```
        f = dir(fullfile(folder, '**', name));
        f = f(~[f.isdir]);
    end
    if isempty(f)
        error('No .dat file found in %s', folder);
    end
    pick = 1;
    if numel(f) > 1
        pick = find(strcmpi({f.name}, 'specimen.dat'));   % MTS default name
        if numel(pick) ~= 1
            [~, pick] = max([f.datenum]);                 % otherwise the newest
        end
        warning('Found %d .dat files in %s; using %s. Set datFileName to choose another.', ...
            numel(f), folder, f(pick).name);
    end
    file = fullfile(f(pick).folder, f(pick).name);
end

%% Local function: read an MTS .dat file (channel-name line, units line, data)
function [data, names, units] = readMtsDat(file)
    lines = regexp(fileread(file), '\r\n|\n|\r', 'split');
    hdr = find(contains(lines, 'Axial', 'IgnoreCase', true) & contains(lines, char(9)), 1);
    if isempty(hdr) || hdr == numel(lines)
        error('Could not find the channel-name line in %s', file);
    end
    names = strtrim(strsplit(lines{hdr},   char(9)));
    units = strtrim(strsplit(lines{hdr+1}, char(9)));
    nCol  = find(~cellfun('isempty', names), 1, 'last');
    names = names(1:nCol);
    units = [units repmat({''}, 1, max(0, nCol - numel(units)))];
    units = units(1:nCol);

    rows = lines(hdr+2:end);
    data = nan(numel(rows), nCol);
    keep = false(numel(rows), 1);
    for i = 1:numel(rows)
        v = sscanf(rows{i}, '%f');
        if numel(v) == nCol      % skips blank lines and repeated headers
            data(i, :) = v.';
            keep(i) = true;
        end
    end
    data = data(keep, :);
end

%% Local function: column index by channel name
function idx = findCol(names, key)
    idx = find(strcmpi(names, key), 1);
    if isempty(idx)
        idx = find(contains(names, key, 'IgnoreCase', true), 1);
    end
    if isempty(idx)
        error('Column "%s" not found. Columns in file: %s', key, strjoin(names, ', '));
    end
end

%% Local function: print a table to the Command Window
function printTable(titleStr, cols, T)
    fprintf('\n%s\n', titleStr);
    T = [cols; T];
    w = max(cellfun(@numel, T), [], 1);
    for i = 1:size(T, 1)
        for j = 1:size(T, 2)
            fprintf('  %-*s', w(j), T{i, j});
        end
        fprintf('\n');
    end
end

%% Local function: "value +/- uncertainty" text for the table
% Uncertainty to 2 significant figures, value to the same decimal place;
% very large or small numbers as (m +/- dm)e+XX.
function str = pm(x, dx)
    if isnan(x)
        str = 'N/A';
        return;
    elseif isnan(dx) || dx <= 0
        str = sprintf('%.5g', x);
        return;
    end
    p = floor(log10(dx)) - 1;               % place of the 2nd significant figure
    if abs(x) >= 1e5 || (abs(x) < 1e-3 && x ~= 0)
        ex  = floor(log10(abs(x)));
        dec = max(0, ex - p);
        str = sprintf('(%.*f %s %.*f)e%+03d', dec, x/10^ex, char(177), dec, dx/10^ex, ex);
    else
        dec = max(0, -p);
        str = sprintf('%.*f %s %.*f', dec, x, char(177), dec, dx);
    end
end
```

Section 5 and the plastic folders: if your plastic samples are in separate folders (`M003 Plastic 1`, `M003 Plastic 2`, …), section 5 skips with a warning because `'plastic'` matches all of them. Section 6 still runs. To put one plastic sample on the all-materials plot, replace `'plastic'` in `allMaterials` with its full folder name, such as `'M003 Plastic 1'`.

You 2026-10-14 23:55 UTC

make this chat a pdf

Claude 2026-10-14 23:56 UTC

I'm building the PDF from the chat transcript now: converting messages to HTML, then printing with Chromium.

Claude 2026-10-14 23:56 UTC

Lists right after a paragraph line aren't being recognized, so they run together as text. Fixing that.

Claude 2026-10-14 23:57 UTC

Looks good. Copying it into the project folder so you can open it.

Claude 2026-10-14 23:57 UTC

I made the chat into a PDF: `chat_transcript.pdf`, 200 letter-size pages, attached above.

- **What's in it:** all 104 messages from Oct 7–14 in order, including every full MATLAB script I gave you, with page numbers at the bottom.
- **How it's laid out:** your messages have a blue bar and mine have an orange one, each with a timestamp. Your uploaded files appear by filename, such as 📎 `specimen.dat`.
- **What's left out:** my background tool work (test runs, file edits and the preview plot images). Only the conversation text and code are included.
- **Where it is:** in the project folder, not committed to git. If you want it in the repo too, I can commit it.

Claude 2026-10-14 23:57 UTC

I committed `chat_transcript.pdf` and pushed it to `claude/brave-johnson-5v00jz`. Nothing is left uncommitted.

The file was already in the repo from an earlier commit, so this one replaces it with the new version.

You 2026-10-14 23:59 UTC

📎 `MaterialsFinalCodePDF.pdf` put these two pdfs together. put the code first then the chat after

— request interrupted —

You 2026-10-15 00:01 UTC

can you make this chat a pdf but everytime theres code just make it smaller